

ENEL445 Roy Smith Optimisation Block

Learning Guide: Least Squares, Convex Optimisation, LMIs, and MPC

Contents

1	Source Materials	2
2	Study Strategy	3
3	Mindmap-based Learning Path	6
4	Least Squares: Core Formulation	7
5	Nonlinear Least Squares and Gauss/Ceres	30
6	Statistical Properties of Least Squares	34
7	Regularisation and Validation	60
8	Convex Sets and Convex Optimisation	75
9	Problem Classes: LP, QP, SOCP, QCQP, SDP	135
10	LMI and Schur Complement	148
11	Model Predictive Control	175
12	Tutorial and Exercise Route	183
13	Exam Templates and Common Mistakes	199
14	Appendix: Practical EDA for Candidate Φ	204
15	Appendix: Φ as Hidden Structure and Dynamics	212
16	Appendix: Math Background - SVD and Pseudo-inverse	218
17	Appendix: Projection Geometry and Instrumental Variables	223

18 Appendix: Math Background - Variance and Covariance Formula Sheet	229
19 Self Check	235
20 Minimum Memory List	235

1 Source Materials

本学习指导基于本章 Roy Smith 相关课件、练习和 6 节转录内容整理。主线不是逐字复述 lecture，而是按考试复习逻辑重构：

- Least-Squares Problems and Methods/Lecture 6 Least squares regression part 1.pdf
- Least-Squares Problems and Methods/Lecture 7 Least squares regression part 2.pdf
- Convex Optimisation, LMIs, and MPC/Lecture-8-Convex-optimisation-part-1.pdf
- Convex Optimisation, LMIs, and MPC/Lecture-9-Convex-optimisation-part-2.pdf
- Convex Optimisation, LMIs, and MPC/ Lecture-10- Convex- optimisation- part-3- model-predictive-control-(MPC).pdf
- exercises/LS_exercises.pdf
- exercises/Statistics_exercises.pdf
- exercises/Convex_Opt_exercises.pdf
- transcribe/20260504_Lecture_6_least_squares_regression_part_1.md
- transcribe/20260505_Lecture_7_least_squares_statistics_BLUE.md
- transcribe/20260506_Least_squares_regularisation_validation.md
- transcribe/20260511_Lecture_8_convex_sets_cones_ellipsoid_method.md
- transcribe/20260512_Lecture_9_LMI_SDP_control_design.md
- transcribe/20260513_Lecture_10_model_predictive_control_MPC.md

补充理解 Φ 的 cross-course reading：

- DATA423/Lectures/FeatureEngineering-2024.pdf
- DATA423/Lectures/FeatureExtraction-2024.pdf
- DATA423/Lectures/FeatureSelection-2024.pdf
- DATA423/Lectures/10_Model Selection-2025.pdf

补充理解 statistical inference / data mining 视角：

- STAT462-25S1 - Statistical Learning/Lecture/02_statistics.pdf
- STAT462-25S1 - Statistical Learning/Lecture/03_linear_regression.pdf
- STAT462-25S1 - Statistical Learning/Lecture/04_multiple_linear_regression.pdf

补充理解 regularisation / shrinkage 视角：

- STAT448/Lectures/lecture/04_RidgeRegression.pdf
- STAT448/Lectures/lecture/05_LASSO_Regression.pdf

- STAT448/Lectures/lecture/06_ElasticNet.pdf

2 Study Strategy

这组材料是混合型章节：前半部分是 least squares 的计算与统计解释，后半部分是 convex optimisation、LMI 和 MPC 的建模框架。复习时不要按日期机械背，而要按下面的问题链组织：

真实工程问题

- > 数据/系统模型
- > 变量、约束、目标函数
- > 是否是 `least-squares` 或 `convex problem`
- > 可用公式、`solver` 或 `iterative method`
- > 考试题型和易错点

最重要的三类能力：

1. 会把问题写成 optimisation form。
2. 会解释公式背后的统计或几何意义。
3. 会判断一个问题属于 LS、LP、QP、SOCP、SDP/LMI 或 MPC 中哪一类。

2.1 Roy's Warning: From Model-Based to Data-Driven Optimisation

Roy 在 Anscombe dataset 之后强调了一个更大的变化：optimisation problem 在最近几十年里越来越多地从 model-based workflow 走向 data-driven workflow。

传统工程 optimisation 更常见的是：

```
known process / engineering model
-> decision variables
-> constraints
-> objective function
-> solver
```

例如 refinery scheduling、production planning、power dispatch、transport logistics、control design。这些问题的重点通常是：系统模型、约束和目标大致已知，工程师需要把它们写成 optimisation formulation。

现在越来越多问题变成：

```
raw data
-> fit statistical / ML model
-> use fitted model inside optimisation
-> make decisions
```

例如：

- demand forecasting → inventory optimisation;
- load forecasting → power dispatch;
- price prediction → bidding strategy;
- traffic prediction → routing;
- learned surrogate model → design optimisation;
- customer model → recommendation or pricing optimisation。

这带来一个额外风险：如果前面的 fitted model 错了，后面的 solver 仍然会非常认真地优化一个错误的 objective、constraint 或 parameter estimate。

所以 Anscombe dataset 的意义不只是“画图好看”。它提醒我们：

data-driven optimisation requires data diagnostics before optimisation.

旧式 model-based optimisation 的主要风险是：

```
wrong formulation
missing constraints
bad scaling
solver or numerical issues
```

data-driven optimisation 额外增加了：

```
wrong fitted model
biased training data
insufficient features
outlier-driven fit
poor extrapolation
unvalidated learned objective or constraint
```

Roy 进一步指出，现代 data-driven optimisation 的数据经常来自 population，很多时候 literally 是人的 population：customers、drivers、patients、students、users、households、market participants。这样的数据天然带有 noise 和 stochastic variation。

原因不是单一的 measurement error，而是多种不确定性叠加：

- 不同个体有不同偏好和行为习惯；
- 同一个人在不同时间也可能行为不同；
- 有很多未观测变量；
- survey、sensor、transaction logs 会有记录误差；
- sample 只是 population 的一部分；
- 行为模式会随时间、政策、价格、天气、社会事件改变；
- model 总会遗漏某些解释因素。

所以 population data 更像：

$$y = \text{systematic signal} + \text{random variation}$$

在 least-squares 形式里就是：

$$y = \Phi\theta + \epsilon$$

这里的 ϵ 不只是仪器测量误差，也包括 unmodelled behaviour、sampling variation、omitted variables 和 genuinely random shocks。

因此现代 optimisation 常常变成：

```
noisy / stochastic data
-> estimate model and uncertainty
-> validate model
-> optimise under uncertainty
```

这就是为什么 Roy 说我们需要能处理 noisy data 或 stochastic data 的 optimisation methods。它们不是在假装数据干净，而是在承认：

optimisation decisions may depend on quantities estimated from noisy data.

对应到本章工具：

Tool	What uncertainty issue it addresses
residual analysis	checks whether unexplained variation looks structured or random
weighted LS	handles unequal noise variance
BLUE	handles covariance structure and minimum-variance unbiased estimation
validation	tests whether fitted structure predicts unseen data
regularisation	reduces variance and instability when fitting noisy data
stochastic / robust optimisation	later-stage tools when uncertainty enters decisions directly

因此本章 least-squares 的 Φ 、rank、residual、validation、regularisation 不只是统计工具。它们是在回答：

```
Can I trust the data-derived model that I am about to optimise over?
```

复习优先级：

Priority	Topics
Must know	LS formulation, normal equations, BLUE, bias/variance/MSE, regularised LS, convexity definition, LP/QP/SOCP/SDP/LMI classification, MPC formulation
Should know	nonlinear LS linearisation, ellipsoid method idea, Schur complement, stability/state-feedback LMI ideas, explicit MPC
Recognise only	S-procedure details in convex exercise problem 3, deep implementation details of CVX/YALMIP

3 Mindmap-based Learning Path

学习顺序建议如下：

1. **Least squares as optimisation:** 先理解 $y = \Phi\theta + \epsilon$ 、residual、normal equations。
2. **Least squares as statistics:** 再理解 $\hat{\theta}$ 是随机变量，所以要看 bias、variance、MSE。
3. **Regularisation:** 之后看为什么最小 training residual 不等于最好 prediction。
4. **Convex foundations:** 学 convex set、cone、PSD cone、ellipsoid。
5. **Problem classes:** 把 LP、QP、SOCP、QCQP、SDP/LMI 放在同一张分类图里。
6. **LMI tools:** 用 Schur complement 和 PSD cone 理解 matrix constraints。
7. **MPC:** 把前面 optimisation framework 用到 control: finite horizon、constraints、replanning。

3.1 Tomorrow's Reading Route for Lectures 6–7

如果明天直接用这份 guide 复习 least squares，不建议从头到尾等权阅读。建议按下面路线走：

Pass 1: Core mechanics

```

model y = Phi theta + epsilon
-> build Phi
-> normal equations
-> geometric projection
-> basis functions and nonlinear LS

```

Pass 2: Statistical meaning

```

theta_hat is random
-> unbiasedness
-> variance / MSE / consistency
-> standard errors / confidence intervals / significance
-> BLUE and inverse-variance weighting

```

-> correlated-noise BLUE

Pass 3: Prediction and practice

bias-variance tradeoff

-> regularisation

-> validation

-> do the Lecture 6--7 practice set

For exam preparation, the high-frequency least-squares items are:

Priority	You should be able to do
Very high	build Φ , derive normal equations, explain projection/residual orthogonality
Very high	prove OLS unbiased under $E[\epsilon] = 0$ and fixed Φ
Very high	derive inverse-variance weights and explain BLUE
High	handle correlated noise using $(\Phi^T R^{-1} \Phi)^{-1} \Phi^T R^{-1} y$
High	explain $\text{MSE} = \text{bias}^2 + \text{variance}$ and why regularisation can reduce MSE
High	interpret standard error, confidence interval, and coefficient significance
Medium	explain nonlinear LS / Gauss-Newton as repeated linearised LS
Medium	discuss rank, condition number, pseudo-inverse, and why normal equations may be numerically poor

4 Least Squares: Core Formulation

4.1 What Problem Does LS Solve?

Least squares 解决的是：模型无法完全通过所有数据点时，如何选择参数使 residual 最小。

线性模型写成：

$$y = \Phi\theta + \epsilon$$

其中：

- $y \in \mathbb{R}^N$ 是观测输出；
- $\Phi \in \mathbb{R}^{N \times d}$ 是 regressor/design matrix；
- $\theta \in \mathbb{R}^d$ 是待估参数；

- ϵ 是误差或 residual。

这里默认的是 ordinary LS assumption: 误差被建模为加在 y 上, 而 Φ 被当作 known / fixed。如果 Φ 也有 measurement error, 例如

$$y = (\Phi + \Delta\Phi)\theta + \epsilon,$$

那就进入 errors-in-variables / total least-squares 的问题, 不再是普通 OLS 的标准设定。

标准 LS 问题:

$$\min_{\theta} \|y - \Phi\theta\|_2^2$$

等价 constrained form:

$$\begin{aligned} \min_{\theta, \epsilon} \quad & \|\epsilon\|_2^2 \\ \text{subject to} \quad & y - \Phi\theta = \epsilon \end{aligned}$$

考试中常见任务:

- 给出数据和模型, 写出 Φ 、 y 、 θ ;
- 写出 LS optimisation problem;
- 推出 normal equations;
- 判断能不能使用 closed-form inverse。

4.2 Understanding Φ : Regressor Structure, Rank, and Real Data

Φ 是 least-squares 里最重要的对象之一。它不是凭空捏造的矩阵, 而是把“我们认为哪些变量能解释 y ”这件事写成矩阵形式。

可以先记住一句话:

Φ 的每一行对应一个 observation, 每一列对应一个 regressor / feature / basis function。

如果有 N 个观测、 d 个待估参数, 那么:

$$\Phi \in \mathbb{R}^{N \times d}$$

一般写成:

$$\Phi = \begin{bmatrix} \phi_1^T \\ \phi_2^T \\ \vdots \\ \phi_N^T \end{bmatrix}$$

其中第 j 行

$$\phi_j^T = [\phi_{j1} \quad \phi_{j2} \quad \cdots \quad \phi_{jd}]$$

表示第 j 个 observation 下所有 regressors 的取值。预测值是：

$$\hat{y}_j = \phi_j^T \theta$$

把所有 observation 叠起来，就是：

$$\hat{y} = \Phi \theta$$

4.2.1 1. How Φ Represents the Regressor Structure

假设我们想拟合：

$$y \approx \theta_1 + \theta_2 x$$

对数据点 $x_1, \dots, x_N$, regressor matrix 是：

$$\Phi = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_N \end{bmatrix}, \quad \theta = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}$$

如果改成二次多项式：

$$y \approx \theta_1 + \theta_2 x + \theta_3 x^2$$

则：

$$\Phi = \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & \vdots & \vdots \\ 1 & x_N & x_N^2 \end{bmatrix}$$

如果用一般 basis functions:

$$f(x) = \sum_{i=1}^d h_i(x)\theta_i$$

那么:

$$\Phi_{ji} = h_i(x_j)$$

也就是说, Φ 的第 i 列就是第 i 个 basis function 在所有数据点上的取值。

因此, 选择 Φ 本质上就是选择 model structure:

Model choice	Columns of Φ
constant + line	$1, x$
polynomial	$1, x, x^2, \dots$
Chebyshev basis	$T_0(x), T_1(x), T_2(x), \dots$
conic fitting	$x^2, xy, y^2, x, y, 1$
physical linear model	measured inputs, sensitivities, known coefficients

4.2.2
2. Rank, Lose Rank, and Linear Dependence

Rank tells us how many independent columns Φ really has.

如果:

$$\text{rank}(\Phi) = d$$

则 Φ 有 full column rank, 所有 columns 都提供了独立信息。此时:

$$\Phi^T \Phi$$

通常可逆, closed-form LS solution 才能写成:

$$\hat{\theta} = (\Phi^T \Phi)^{-1} \Phi^T y$$

如果 Φ lose rank, 意思是某些 columns 可以由其他 columns 线性组合出来。例如:

$$\phi_3 = 2\phi_2$$

那么第 3 列没有提供新的方向。模型里虽然写了两个参数 θ_2, θ_3 , 但数据无法区分它们的贡献, 因为:

$$\theta_2\phi_2 + \theta_3\phi_3 = \theta_2\phi_2 + 2\theta_3\phi_2 = (\theta_2 + 2\theta_3)\phi_2$$

这会导致参数不唯一: 很多组 θ_2, θ_3 可以给出同样的 prediction。

常见导致 lose rank 的原因:

- 两个 regressors 完全重复;
- 一个 column 是另一个 column 的倍数;
- intercept 和 dummy variables 同时设置不当;
- 数据点太少, $N < d$;
- 某个变量在数据中没有变化, 例如所有 x_j 都一样;
- 工程系统里某些输入总是一起变化, 导致无法分辨单独贡献。

4.2.3 3. Condition Number: Not Singular, But Almost Singular

Rank 讨论的是“是否精确线性相关”。Condition number 讨论的是“是否接近线性相关”。

对 Φ , condition number 可以直观理解为:

$$\kappa(\Phi) = \frac{\text{largest stretching direction}}{\text{smallest stretching direction}}$$

用 singular values 写就是:

$$\kappa(\Phi) = \frac{\sigma_{\max}(\Phi)}{\sigma_{\min}(\Phi)}$$

如果 $\sigma_{\min}(\Phi)$ 很小, 说明有些 column direction 几乎可以由其他 columns 解释。矩阵还没有完全 lose rank, 但已经非常接近。

直观解释:

- $\kappa(\Phi)$ 小: 不同 regressors 提供的方向比较独立, 估计较稳定;
- $\kappa(\Phi)$ 大: 某些 regressors 几乎重复, 参数估计会对 noise 很敏感;
- $\sigma_{\min}(\Phi) = 0$: 真的 lose rank, 某些参数方向不可辨识。

Normal equation 会形成:

$$\Phi^T \Phi$$

这会进一步放大 conditioning 问题：

$$\kappa(\Phi^T \Phi) = \kappa(\Phi)^2$$

这就是为什么 lecture 里强调：closed-form 解适合理论，但实践中直接 invert $\Phi^T \Phi$ 可能数值不稳。

4.2.4 4. Is Φ Symmetric?

通常 Φ 本身不是 symmetric。

首先， Φ 往往不是 square matrix：

$$\Phi \in \mathbb{R}^{N \times d}$$

通常 N 是数据点数量， d 是参数数量，两者不一定相等。所以大多数情况下，问 Φ 是否 symmetric 没有意义。

真正 symmetric 的对象是：

$$\Phi^T \Phi$$

因为：

$$(\Phi^T \Phi)^T = \Phi^T (\Phi^T)^T = \Phi^T \Phi$$

所以 $\Phi^T \Phi$ 一定是 symmetric。

而且它还是 positive semidefinite：

$$z^T \Phi^T \Phi z = (\Phi z)^T (\Phi z) = \|\Phi z\|_2^2 \geq 0$$

如果 Φ full column rank，那么对所有 $z \neq 0$ ：

$$\Phi z \neq 0$$

因此：

$$z^T \Phi^T \Phi z > 0$$

这时 $\Phi^T \Phi$ 是 positive definite, 也就可逆。

所以关键关系是:

$$\Phi \text{ full column rank} \iff \Phi^T \Phi \text{ positive definite} \implies (\Phi^T \Phi)^{-1} \text{ exists}$$

4.2.5 5. Where Does Φ Come From in Real Problems?

Φ 来自 model design, 不是从空气里随便写出来。常见来源有四类。

第一类: 来自你选择的 mathematical basis。

例如 polynomial fitting 里, 选择:

$$1, x, x^2$$

就决定了 Φ 的列是:

$$\begin{bmatrix} 1 \end{bmatrix}, \begin{bmatrix} x_1 \\ x_2 \\ \vdots \end{bmatrix}, \begin{bmatrix} x_1^2 \\ x_2^2 \\ \vdots \end{bmatrix}$$

第二类: 来自 physical model。

例如一个线性化工程模型可能写成:

$$y \approx au_1 + bu_2 + c$$

其中 u_1, u_2 是测到的输入。那么:

$$\Phi = \begin{bmatrix} u_{1,1} & u_{2,1} & 1 \\ u_{1,2} & u_{2,2} & 1 \\ \vdots & \vdots & \vdots \\ u_{1,N} & u_{2,N} & 1 \end{bmatrix}, \quad \theta = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$$

第三类: 来自 linearisation。

如果原模型是 nonlinear:

$$y = H(\theta)$$

在 nominal point θ_i 附近线性化：

$$H(\theta) \approx H(\theta_i) + H_\theta(\theta - \theta_i)$$

这里的 Jacobian：

$$H_\theta$$

就扮演了 local regressor matrix 的角色。

第四类：来自 power-system / DCOPF 这类结构模型。

例如 DC power flow 中，line flow 可能近似写成：

$$f \approx \text{PTDF } p$$

如果我们用一些 observed injections、loads、binding constraints 或 sensitivity quantities 去解释 price、flow 或 dispatch，那么 Φ 的列就来自这些物理量或优化模型导出的 sensitivity。

这时 Φ 不是任意矩阵，而是由：

- 网络拓扑；
- line reactance / admittance；
- PTDF 或 sensitivity matrix；
- load / generation observations；
- constraint status；
- 线性化假设；
- 数据采样方式；

共同决定。

因此建模时要问：

Which physical or statistical mechanism makes these columns relevant to y ?

Are these columns independently informative?

Are any columns nearly duplicated or badly scaled?

Is Φ measured accurately enough to treat it as fixed?

这也把前面讨论的 errors-in-variables 联系起来：如果 Φ 的来源本身有测量误差或模型误差，那么普通 OLS 的“误差只放在 y 上”假设就会变弱。

4.2.6 6. Connecting Φ to Feature Engineering, Extraction, Selection, and Model Selection

First-pass reading note: 这一节主要是帮你把 DATA423 的 feature workflow 和 Roy 的 Φ 串起来。明天如果时间紧，先确保你会 build Φ 、判断 rank/condition、推 normal equations；这一节可以第二遍读，用来加深直觉。

DATA423 里的 feature engineering、feature extraction、feature selection 和 model selection 可以和 least-squares 里的 Φ 完全串起来。它们都在回答同一个问题：

我们到底应该把哪些 columns 放进 Φ ？

可以把流程理解成：

```
raw measurements
-> feature engineering / extraction
-> candidate Phi matrices
-> feature selection
-> model selection
-> final fitted model
```

在 LS 语言里：

$$y \approx \Phi\theta$$

所以任何关于 features 的决定，最后都会变成对 Φ 的列的决定。

4.2.6.1 Feature Engineering: Designing Candidate Columns

Feature engineering 是把原始变量变成更适合建模的变量。放到 Φ 里，就是构造 candidate columns。

例如原始变量是：

$$x_1, \quad x_2$$

最简单的 Φ 可以是：

$$\Phi = \begin{bmatrix} 1 & x_{1,1} & x_{2,1} \\ 1 & x_{1,2} & x_{2,2} \\ \vdots & \vdots & \vdots \\ 1 & x_{1,N} & x_{2,N} \end{bmatrix}$$

如果我们认为有 interaction：

$$x_1 x_2$$

就加一列：

$$\Phi = \begin{bmatrix} 1 & x_{1,1} & x_{2,1} & x_{1,1}x_{2,1} \\ 1 & x_{1,2} & x_{2,2} & x_{1,2}x_{2,2} \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_{1,N} & x_{2,N} & x_{1,N}x_{2,N} \end{bmatrix}$$

模型变成：

$$y \approx \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1 x_2$$

这仍然是 linear least-squares，因为它对参数 θ 是线性的。

常见 feature engineering 操作在 Φ 中的含义：

Feature engineering idea	What happens to Φ
add intercept	add a column of ones
polynomial term x^2	add a column x_j^2
interaction $x_1 x_2$	add a column $x_{1,j} x_{2,j}$
log transform $\log x$	replace/add a column $\log(x_j)$
standardise variable	rescale a column of Φ
one-hot encode category	add indicator columns
cyclic time feature	add sin and cos columns

所以 feature engineering 的直觉是：

用 domain knowledge 决定哪些 transformations 可能让 y 更容易由 $\Phi\theta$ 解释。

4.2.6.2 Feature Extraction: Creating a New Representation

Feature extraction 是从原始变量中创造新的变量。它通常更强调“从已有变量中提取更有信息量的表示”。

假设原始数据矩阵是：

$$X \in \mathbb{R}^{N \times p}$$

Feature extraction 可以看成构造一个变换：

$$Z = g(X)$$

然后用：

$$\Phi = Z$$

或者：

$$\Phi = [\mathbf{1}, X, Z]$$

例如 PCA 会把原始变量变成 principal components：

$$Z = XW$$

其中 W 是 learned projection matrix。此时 Φ 的列不再是原始变量，而是 latent directions：

$$\Phi = \begin{bmatrix} z_{1,1} & z_{1,2} & \cdots & z_{1,k} \\ z_{2,1} & z_{2,2} & \cdots & z_{2,k} \\ \vdots & \vdots & & \vdots \\ z_{N,1} & z_{N,2} & \cdots & z_{N,k} \end{bmatrix}$$

这解释了 latent variables 的概念：这些 variables 不是直接测量的，而是从 raw variables 中推断出来的。

Feature extraction 常见目的：

- 让原始变量变得更适合某类模型；
- 把非线性或周期性结构显式表达出来；
- 降低维度；
- 把多个相关变量压缩成少数 latent variables；
- 改善 Φ 的 conditioning。

例如 cyclic time feature：

$$\text{month}_{\sin} = \sin\left(\frac{2\pi(m-1)}{12}\right)$$

$$\text{month}_{\cos} = \cos\left(\frac{2\pi(m-1)}{12}\right)$$

这比直接把 month 写成 1 到 12 更合理，因为 December 和 January 在周期上很近。

4.2.6.3 Feature Selection: Removing Columns from Φ

Feature selection 是决定哪些 columns 应该保留，哪些应该删除。

如果候选矩阵是：

$$\Phi_{\text{candidate}} = \begin{bmatrix} | & | & | & | \\ \phi_1 & \phi_2 & \phi_3 & \phi_4 \\ | & | & | & | \end{bmatrix}$$

feature selection 可能选择：

$$\Phi_{\text{selected}} = \begin{bmatrix} | & | \\ \phi_1 & \phi_3 \\ | & | \end{bmatrix}$$

也就是删除不重要、重复、噪声大或导致 overfitting 的 columns。

为什么这和 LS 很相关？

- 多余 columns 会增加 variance；
- 无关 columns 会让模型学到 spurious pattern；
- 相关 columns 会让 Φ ill-conditioned；
- 太多 columns 会增加 overfitting 风险；
- 透明模型需要较少且有意义的 columns。

几类 feature selection 方法可以翻译成 Φ 语言：

Feature selection type	Φ interpretation
manual selection	人根据 domain knowledge 删除/保留 columns
filter method	先给 columns 排名，再取 top columns
wrapper method	尝试不同 column subsets，用模型表现选
embedded method	模型训练时自动决定哪些 columns 有用
LASSO / ElasticNet	通过 penalty 让部分 coefficients 变小或变成 0

LASSO 的直觉特别容易和 Φ 连接：

$$\min_{\theta} \|y - \Phi\theta\|_2^2 + \lambda\|\theta\|_1$$

如果某个 coefficient：

$$\theta_i = 0$$

那么 Φ 的第 i 列虽然还在数据里，但模型实际没有使用它。

4.2.6.4 Model Selection: Choosing Between Candidate Φ Designs

Feature engineering、extraction 和 selection 会产生很多可能的 Φ :

$$\Phi_1, \quad \Phi_2, \quad \Phi_3, \quad \dots$$

例如:

$$\Phi_1 = [1, x_1, x_2]$$

$$\Phi_2 = [1, x_1, x_2, x_1x_2]$$

$$\Phi_3 = [1, x_1, x_2, x_1^2, x_2^2, x_1x_2]$$

$$\Phi_4 = [1, T_1(x), T_2(x), T_3(x)]$$

Model selection 就是在这些 candidate models 之间选择:

$$\mathcal{M}_1, \quad \mathcal{M}_2, \quad \mathcal{M}_3, \dots$$

在 regression 里, 常用指标包括:

- RMSE;
- MAE;
- R^2 ;
- validation error;
- cross-validation error;
- AIC / BIC when appropriate.

一个重要原则是: 不要只看 training residual。

如果只看:

$$\|y_{\text{train}} - \Phi_{\text{train}} \hat{\theta}\|_2^2$$

更复杂的 Φ 通常会更好, 因为 columns 更多、模型更灵活。但这不代表它在 unseen data 上更好。

所以要看 validation/test performance:

$$\|y_{\text{valid}} - \Phi_{\text{valid}} \hat{\theta}\|_2^2$$

这正好连接 Roy Smith 后面讲的 regularisation / validation: 最小 training error 不等于最好 prediction。

4.2.6.5 One Unified View

可以把这些概念压缩成一张表:

Concept	Question	Effect on Φ
feature engineering	raw variables 应该如何变形?	add/transform columns
feature extraction	能否从 raw variables 中提取更好的表示?	create new columns $Z = g(X)$
feature selection	哪些 columns 应该保留?	remove or downweight columns
model selection	哪个 candidate Φ 泛化最好?	choose between $\Phi_1, \Phi_2, \dots$
regularisation	如何控制复杂度和 instability?	penalise coefficients linked to columns

所以你可以把 Φ 想成一个“建模决策的账本”:

每一列都是一个假设:

`this feature/basis/interaction/latent variable helps explain y.`

如果 Φ 构造得好, LS 会在这些 columns 的 span 里找到合理 projection。如果 Φ 构造得差, LS 仍然会给你一个最小 residual 的解, 但那个解可能没有解释力、泛化差、数值不稳, 或者参数不可辨识。

实践中, 我们通常会在拟合前对 candidate Φ 做一些 EDA, 例如检查 variance、correlation、VIF、rank 和 condition number。这部分不是 Roy Smith 这组课的考试主线, 但它能帮助你把 DATA423 的 feature workflow 和这里的 Φ 融合起来。详细流程放在 Appendix: Practical EDA for Candidate Φ 。

4.3 Normal Equations

令

$$J(\theta) = \|y - \Phi\theta\|_2^2$$

其中维度通常是:

$$y \in \mathbb{R}^N, \quad \Phi \in \mathbb{R}^{N \times d}, \quad \theta \in \mathbb{R}^d$$

所以:

$$\Phi\theta \in \mathbb{R}^N, \quad y - \Phi\theta \in \mathbb{R}^N$$

目标函数是 residual 的平方长度：

$$J(\theta) = (y - \Phi\theta)^T(y - \Phi\theta) = \|y - \Phi\theta\|_2^2$$

有时 lecture 写成：

$$J(\theta) = (\Phi\theta - y)^T(\Phi\theta - y) = \|\epsilon\|_2^2$$

这两个完全等价，因为：

$$\Phi\theta - y = -(y - \Phi\theta)$$

平方长度不受负号影响。

4.3.1 Matrix Expansion Rules

推导 normal equations 时，最常用的是下面几条规则。

第一，矩阵乘法展开和普通代数很像，但顺序不能乱：

$$(a - b)^T(a - b) = a^T a - a^T b - b^T a + b^T b$$

令

$$a = \Phi\theta, \quad b = y$$

则：

$$\begin{aligned} J(\theta) &= (\Phi\theta - y)^T(\Phi\theta - y) \\ &= (\Phi\theta)^T(\Phi\theta) - (\Phi\theta)^T y - y^T(\Phi\theta) + y^T y \end{aligned}$$

第二，转置乘积要反过来：

$$(AB)^T = B^T A^T$$

所以：

$$(\Phi\theta)^T = \theta^T \Phi^T$$

于是第一项变成：

$$(\Phi\theta)^T(\Phi\theta) = \theta^T\Phi^T\Phi\theta$$

第三，中间两项其实是同一个 scalar。因为

$$(\Phi\theta)^Ty$$

是 1×1 标量，而标量等于自己的转置：

$$(\Phi\theta)^Ty = ((\Phi\theta)^Ty)^T = y^T(\Phi\theta)$$

并且：

$$(\Phi\theta)^Ty = \theta^T\Phi^Ty$$

所以：

$$-(\Phi\theta)^Ty - y^T(\Phi\theta) = -2\theta^T\Phi^Ty$$

完整展开为：

$$J(\theta) = \theta^T\Phi^T\Phi\theta - 2\theta^T\Phi^Ty + y^Ty$$

这就是截图里那一步：

$$(\Phi\theta - y)^T(\Phi\theta - y) = \theta^T\Phi^T\Phi\theta - 2(\Phi\theta)^Ty + y^Ty$$

也可以把中间项写成 $-2\theta^T\Phi^Ty$ ，因为 $(\Phi\theta)^Ty = \theta^T\Phi^Ty$ 。

4.3.2 Differentiation Rules

接着对 θ 求导。常用规则是：

$$\frac{\partial}{\partial x}(x^T Ax) = (A + A^T)x$$

如果 A 是 symmetric，则：

$$\frac{\partial}{\partial x}(x^T Ax) = 2Ax$$

在这里：

$$A = \Phi^T \Phi$$

而

$$(\Phi^T \Phi)^T = \Phi^T \Phi$$

所以 $\Phi^T \Phi$ 是 symmetric。

另外：

$$\frac{\partial}{\partial x}(x^T b) = b$$

以及不含 x 的常数项求导为 0。

因此：

$$\begin{aligned} \frac{\partial J}{\partial \theta} &= \frac{\partial}{\partial \theta} (\theta^T \Phi^T \Phi \theta - 2\theta^T \Phi^T y + y^T y) \\ &= 2\Phi^T \Phi \theta - 2\Phi^T y + 0 \end{aligned}$$

令 gradient 为 0：

$$2\Phi^T \Phi \theta - 2\Phi^T y = 0$$

两边除以 2：

$$\Phi^T \Phi \theta - \Phi^T y = 0$$

所以：

$$\Phi^T \Phi \theta = \Phi^T y$$

如果 $\Phi^T \Phi$ 可逆：

$$\hat{\theta} = (\Phi^T \Phi)^{-1} \Phi^T y$$

4.3.3 Why the Closed Form Is Not Always How We Compute It

这个公式是 least-squares 的 theoretical solution。它很适合：

- 证明 theorem；
- 理解 estimator 的结构；
- 在小规模、well-conditioned 的例子中手算或教学；
- 帮助看清楚 $\Phi^T\Phi$ 、rank、variance 等概念。

但在实际计算中，不一定应该直接写：

$$\hat{\theta} = (\Phi^T\Phi)^{-1}\Phi^Ty$$

原因是 normal equation 先形成了：

$$\Phi^T\Phi$$

这一步会把 singular values 大致平方。直观上：

```
small numbers become smaller
large numbers become larger
```

如果 Φ 本来 scaling 就不好，或者不同列的量级差很多，那么 $\Phi^T\Phi$ 会更 ill-conditioned。

用 condition number 表示，通常有近似关系：

$$\kappa(\Phi^T\Phi) = \kappa(\Phi)^2$$

所以如果 Φ 的 condition number 已经大，normal equation 会把数值问题放大。

接着公式还要求：

$$(\Phi^T\Phi)^{-1}$$

这意味着小的 singular values 在 inverse 里会变成很大的数。于是 tiny numerical errors 或 noisy data 可能被放大，最后影响 $\hat{\theta}$ 。

这就是 lecture 里说的：

```
squared things
small numbers get smaller
big numbers get bigger
then inverted
small numbers may get very big
```


从实践角度，通常更稳的做法是：

- 不显式计算 inverse；
- 用 QR decomposition 或 SVD 解 least-squares；
- 在 MATLAB 里用 backslash，例如 `theta = Phi \ y`；
- 在 Python/NumPy 里用 `lstsq`，而不是手写 inverse；
- 如果问题 ill-conditioned，考虑 scaling、regularisation 或 SVD truncation。

如果是 under-determined case，也就是参数数量比有效数据约束更多：

$$d > N$$

或者 Φ 没有 full column rank，那么：

$$\Phi^T \Phi$$

不可逆或接近不可逆。此时 closed-form inverse 不再是可靠计算方式，甚至不存在。后面讲 regularisation、minimum-norm solution、pseudo-inverse 或 SVD 时，就是在处理这类问题。

Pseudo-inverse 小提示：

如果 Φ 不是 full column rank，不能直接用普通 inverse，但可以用 Moore-Penrose pseudo-inverse：

$$\hat{\theta} = \Phi^+ y$$

其中 Φ^+ 可以通过 SVD 定义。若：

$$\Phi = U \Sigma V^T$$

则：

$$\Phi^+ = V \Sigma^+ U^T$$

Σ^+ 的做法是：把非零 singular values 取倒数，零 singular values 保持为零。

直觉：

- ordinary inverse 只适用于 square 且 nonsingular 的矩阵；
- pseudo-inverse 可以处理 rectangular、under-determined 或 rank-deficient 的 LS 问题；
- 在有多组 exact/least-squares solutions 时，pseudo-inverse 通常给出 minimum-norm solution；
- 如果 singular values 很小，pseudo-inverse 仍可能放大 noise，所以实践中常和 SVD truncation 或 regularisation 一起考虑。

所以 Roy 说 rank deficient 时可以用 pseudo-inverse, 意思是: normal-equation inverse 不存在时, 仍然可以用 SVD-based generalized inverse 找到一个合理的 LS 解。

所以可以把 normal-equation 解记成:

theoretical solution: useful for understanding and proofs, but not always the numerical algorithm.

注意事项总结:

- normal equations 总是 consistent;
- 但 inverse form 需要 Φ full column rank;
- 如果 $\Phi^T \Phi$ condition number 很大, 解会对噪声敏感;
- 实际计算时避免显式求 inverse, 优先使用 QR、SVD 或 solver 提供的 least-squares routine;
- under-determined 或 rank-deficient 时, 要转向 pseudo-inverse、regularisation 或其他约束。

4.4 Geometric Interpretation

LS 的几何意义是: 把 y 投影到 Φ 的 column space 上。残差

$$r = y - \Phi \hat{\theta}$$

与 column space 正交, 因此:

$$\Phi^T r = 0$$

也就是:

$$\Phi^T (y - \Phi \hat{\theta}) = 0$$

这正是 normal equations。

记忆方式: **LS** 不是让每个点都正确, 而是让误差向量无法再沿模型空间方向减少。

4.5 Linear Function Parametrisation and Basis Functions

Lecture 6 里说的 linear least-squares 不等于 “只能拟合直线”。更准确的说法是:

模型对参数 θ 是 linear combination, 但每个 basis function $h_i(x)$ 可以是非线性的。

一般写成:

$$y = f(x) = \sum_{i=1}^d h_i(x) \theta_i$$

这里 $h_i(x)$ 是 basis functions, θ_i 是要估计的系数。只要模型对 θ_i 是线性的, 就仍然可以放进 linear least-squares 框架。

对数据点 $x_1, \dots, x_N$, design matrix 是:

$$\Phi = \begin{bmatrix} h_1(x_1) & h_2(x_1) & \cdots & h_d(x_1) \\ h_1(x_2) & h_2(x_2) & \cdots & h_d(x_2) \\ \vdots & \vdots & & \vdots \\ h_1(x_N) & h_2(x_N) & \cdots & h_d(x_N) \end{bmatrix}$$

于是:

$$y \approx \Phi\theta$$

重点是: 非线性可以在 $h_i(x)$ 里, linear 只要求 θ 是线性出现的。

4.5.1 Power Basis

最直接的多项式 basis 是 powers of x :

$$h_1(x) = 1, \quad h_2(x) = x, \quad h_3(x) = x^2, \quad h_4(x) = x^3, \dots$$

因此三次多项式可以写成:

$$f(x) = \theta_1 + \theta_2 x + \theta_3 x^2 + \theta_4 x^3$$

这不是关于 x 的线性函数, 但它仍然是关于参数 θ 的线性组合。

4.5.2 Chebyshev Basis

Chebyshev polynomials 是另一组多项式 basis。它们在区间 $[-1, 1]$ 上特别常用, 因为数值性质通常比普通 powers of x 更稳定。

前几项是:

$$T_0(x) = 1$$

$$T_1(x) = x$$

$$T_2(x) = 2x^2 - 1$$

递推关系是：

$$T_{i+1}(x) = 2xT_i(x) - T_{i-1}(x)$$

所以也可以写：

$$f(x) = \theta_1 T_0(x) + \theta_2 T_1(x) + \theta_3 T_2(x) + \dots$$

它和 power basis 的区别不是“能不能用 LS”，而是 Φ 的列长什么样、这些列之间是否容易数值相关、拟合曲线在区间上如何摆动。

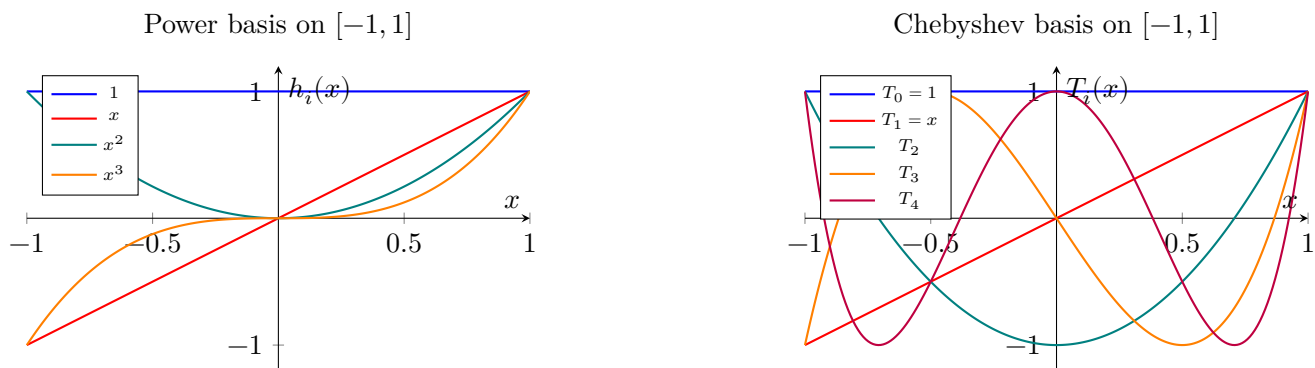


图 1: Power basis 和 Chebyshev basis 都可以形成 Φ 的列，但它们产生的曲线形状和数值性质不同。

4.5.3 Why Polynomial Bases Are Not Good for Orbits

多项式 basis 很适合局部拟合曲线，但不适合作为行星或彗星轨道的自然模型。原因是：行星和彗星在经典二体问题下走的是圆锥曲线，而不是一般多项式图像。

圆锥曲线的一般隐式形式是：

$$Ax^2 + Bxy + Cy^2 + Dx + Ey + F = 0$$

这个式子对 x, y 是二次的，但对参数

$$\theta = [A \ B \ C \ D \ E \ F]^T$$

仍然是线性的。把 basis functions 写出来：

$$h(x, y) = \begin{bmatrix} x^2 & xy & y^2 & x & y & 1 \end{bmatrix}$$

则每个观测点满足：

$$h(x_j, y_j)\theta \approx 0$$

所以“用圆锥曲线拟合轨道”仍然保留了 linear combination 的思想，只是 basis functions 从

$$1, x, x^2, \dots$$

变成了

$$x^2, xy, y^2, x, y, 1$$

小心一点：如果直接最小化 $\|\Phi\theta\|^2$ ，会得到 trivial solution $\theta = 0$ 。实际 conic fitting 通常需要加 normalization 或 constraint，例如固定某个系数、令 $\|\theta\| = 1$ ，或加入与 ellipse/parabola/hyperbola 类型相关的约束。这里要抓住的主线是：**basis function 改变了可表达的几何形状。**

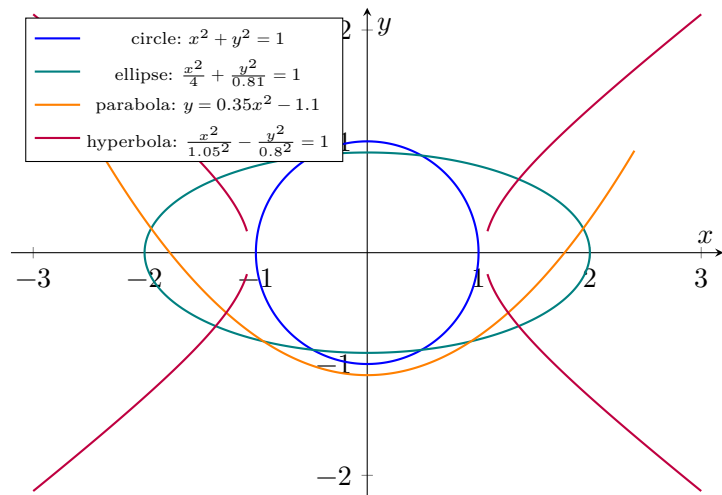


图 2: 圆、椭圆、抛物线、双曲线都是圆锥曲线。它们不是普通的一元多项式图像，但可以由二次隐式 basis functions 表示。

从图像直观上看：

- power basis 让函数图像以 $y = f(x)$ 的形式上下弯曲；
- Chebyshev basis 也是多项式，但在 $[-1, 1]$ 上有更规整的振荡结构；
- conic basis 不一定把曲线写成单值函数 $y = f(x)$ ，因此可以表达闭合轨道形状，例如 circle 和 ellipse；
- 对 Ceres/planet orbit 这类问题，conic section 比普通 polynomial 更符合物理几何。

5 Nonlinear Least Squares and Gauss/Ceres

Lecture 6 用 Gauss 预测 Ceres 轨道说明: least squares 不只是 line fitting, 也可以处理 nonlinear model。

非线性模型:

$$y_{\text{obs}} = H(\theta, t) + \epsilon$$

目标:

$$\min_{\theta} \frac{1}{2} \epsilon^T \epsilon$$

其中:

$$\epsilon = y_{\text{obs}} - H(\theta, t)$$

非线性 LS 的基本做法:

1. 从 nominal parameter θ_i 开始。
2. 对 $H(\theta)$ 做 Taylor linearisation:

$$H(\theta) \approx H(\theta_i) + H_{\theta}(\theta - \theta_i)$$

3. 定义:

$$\delta y = y_{\text{obs}} - H(\theta_i), \quad \delta \theta = \theta - \theta_i$$

4. 解线性 LS:

$$\min_{\delta \theta} \|\delta y - H_{\theta} \delta \theta\|_2^2$$

5. normal equations:

$$H_{\theta}^T H_{\theta} \delta \theta = H_{\theta}^T \delta y$$

6. 更新:

$$\theta_{i+1} = \theta_i + \delta \theta$$

5.1 Why This Is a Gradient / Gauss-Newton Update

Roy 说 Gauss 做的事情 “basically solving the gradient update step”, 意思不是普通 gradient descent 那种简单更新:

$$\theta_{i+1} = \theta_i - \alpha \nabla J(\theta_i)$$

而是更接近 **Gauss-Newton**:

在当前点附近把 nonlinear model 线性化, 然后解一个 local least-squares problem, 得到一个更聪明的 step $\delta\theta$ 。

原始 nonlinear cost 是:

$$J(\theta) = \frac{1}{2} \|y_{\text{obs}} - H(\theta)\|_2^2$$

定义 residual:

$$r(\theta) = y_{\text{obs}} - H(\theta)$$

在当前估计 θ_i 处:

$$r_i = r(\theta_i) = y_{\text{obs}} - H(\theta_i)$$

对模型做一阶 Taylor expansion:

$$H(\theta_i + \delta\theta) \approx H(\theta_i) + H_\theta \delta\theta$$

因此 residual 近似为:

$$r(\theta_i + \delta\theta) = y_{\text{obs}} - H(\theta_i + \delta\theta) \approx y_{\text{obs}} - H(\theta_i) - H_\theta \delta\theta$$

也就是:

$$r(\theta_i + \delta\theta) \approx r_i - H_\theta \delta\theta$$

于是 nonlinear cost 在当前点附近被近似成一个 quadratic cost:

$$J_{\text{quad}}(\delta\theta) = \frac{1}{2} \|r_i - H_\theta \delta\theta\|_2^2$$

这就是 Roy 说的: linearise the nonlinear model, 然后因为 error squared, 所以得到 quadratic approximation to the cost function。

现在问题变成:

$$\min_{\delta\theta} \frac{1}{2} \|r_i - H_\theta \delta\theta\|_2^2$$

这就是一个 ordinary linear least-squares problem, 只不过 unknown 不是 θ , 而是 update step:

$$\delta\theta$$

对应关系是:

Ordinary LS	Gauss-Newton local LS
y	r_i or δy
Φ	H_θ
θ	$\delta\theta$
$\min_\theta \ y - \Phi\theta\ ^2$	$\min_{\delta\theta} \ r_i - H_\theta \delta\theta\ ^2$

对 J_{quad} 求导:

$$\nabla_{\delta\theta} J_{\text{quad}} = -H_\theta^T (r_i - H_\theta \delta\theta)$$

令 gradient 为 0:

$$-H_\theta^T (r_i - H_\theta \delta\theta) = 0$$

所以:

$$H_\theta^T H_\theta \delta\theta = H_\theta^T r_i$$

这就是 local normal equations。

如果 $H_\theta^T H_\theta$ 可逆:

$$\delta\theta = (H_\theta^T H_\theta)^{-1} H_\theta^T r_i$$

然后更新:

$$\theta_{i+1} = \theta_i + \delta\theta$$

再在新的 θ_{i+1} 处重新计算:

$$H(\theta_{i+1}), \quad H_\theta(\theta_{i+1}), \quad r(\theta_{i+1})$$

然后重复。

所以 Gauss 的迭代可以写成:

```
start with theta_i
-> linearise nonlinear model around theta_i
-> solve a linear LS problem for delta theta
-> update theta_{i+1} = theta_i + delta theta
-> repeat until residual/update is small
```

5.1.1 Gradient Descent vs Gauss-Newton

普通 gradient descent 只用 gradient direction:

$$\theta_{i+1} = \theta_i - \alpha \nabla J(\theta_i)$$

它需要选择 step size α , 而且只知道“往哪里下降”。

Gauss-Newton 更进一步: 它用 Jacobian H_θ 构造一个 local quadratic approximation:

$$J(\theta_i + \delta\theta) \approx J_{\text{quad}}(\delta\theta)$$

然后直接最小化这个 local quadratic model, 得到 step $\delta\theta$ 。

所以它不是“随便走一步”, 而是:

choose the step that would solve the linearised least-squares problem.

这就是 Roy 说 least squares is solving the gradient update step 的意思: 每一次 nonlinear optimisation 的 update step, 本身就是一个 linear least-squares solve。

5.1.2 Why This Matters for Ceres

Gauss 的 Ceres problem 是 nonlinear, 因为 orbital elements 和 observations 的关系不是线性的。直接一次 LS 不能解决完整问题。

但在当前 orbit estimate 附近，小范围内可以线性化：

$$\text{change in predicted observations} \approx H_\theta \delta\theta$$

于是每次 iteration 都是：

$$\text{observation mismatch} \approx \text{Jacobian} \times \text{parameter update}$$

也就是：

$$\delta y \approx H_\theta \delta\theta$$

通过 LS 找到最能减少 mismatch 的 $\delta\theta$ ，再更新 orbit estimate。重复很多次后，得到高精度 orbit。

这就是：

```
nonlinear orbit estimation
-> repeated linear least-squares updates
-> Gauss-Newton style iteration
```

考试理解重点：

- 三个观测点给 Gauss 一个 nominal orbit，但完整 22 个观测点用于 LS refinement；
- nonlinear LS 每一步只是 local quadratic/linear approximation；
- LS solve 给出的不是最终答案本身，而是当前 iteration 的 update step；
- 不能把一次 linearised solution 当成 global exact solution。

6 Statistical Properties of Least Squares

6.1 Why LS Estimate Is Random

Roy 在 Lecture 7 这里做了一个重要转向：least squares 不再只是 deterministic optimisation，而是 statistical estimation。

Deterministic view 是：

```
given y and Phi
solve for theta_hat
```

也就是：

$$\hat{\theta} = \arg \min_{\theta} \|y - \Phi\theta\|_2^2$$

但实际 measurement data 有误差。Gauss 测 Ceres 的角度时，观测本身就有有限精度；现代 data-driven optimisation 里的 population data 也有 stochastic variation。

所以更合理的 model 是：

$$y = \Phi\theta_0 + \epsilon$$

其中：

- θ_0 是 underlying true parameter;
- ϵ 是 measurement noise / random error;
- y 因为包含 ϵ ，所以是 random;
- Φ 在 standard LS 中先当作 fixed design matrix。

如果 ϵ 是随机噪声，那么

$$\hat{\theta} = (\Phi^T \Phi)^{-1} \Phi^T y$$

也是随机变量。

直接代入：

$$\begin{aligned}\hat{\theta} &= (\Phi^T \Phi)^{-1} \Phi^T (\Phi \theta_0 + \epsilon) \\ &= (\Phi^T \Phi)^{-1} \Phi^T \Phi \theta_0 + (\Phi^T \Phi)^{-1} \Phi^T \epsilon \\ &= \theta_0 + (\Phi^T \Phi)^{-1} \Phi^T \epsilon\end{aligned}$$

所以：

$$\hat{\theta} - \theta_0 = (\Phi^T \Phi)^{-1} \Phi^T \epsilon$$

这说明 $\hat{\theta}$ 的误差完全由 measurement noise 通过 LS estimator 传播而来。

如果重复同一个 experiment，即使 Φ 不变，新的 measurement noise 会产生新的 y ，因此会产生新的 $\hat{\theta}$ 。

这就是 Roy 说：

`theta_hat is a statistical quantity`

的意思。

把 $\hat{\theta}$ 当作 random variable 后，我们才有理由问：

因此考试会关心：

- $\hat{\theta}$ 是否 unbiased;

- variance/covariance 有多大；
- MSE 是否最小；
- correlated noise 时如何调整。

6.2 Bias, Variance, and MSE

Roy 这里强调：MSE 是一个 expectation。它不是某一次 experiment 算出来的单个 squared error，而是在理论上对同一个 data-generating mechanism 下的 repeated sampling / measurement noise 取平均。

实践中，我们当然不会只停留在“想象”。因为真实 data-generating distribution P 不知道，也不能无限重复实验，所以通常用 train/validation/test split、cross-validation 或 bootstrap 去估计这个 expectation。

单次实验中你看到的是：

$$(\hat{\theta} - \theta_0)^2$$

但 MSE 是：

$$E[(\hat{\theta} - \theta_0)^2]$$

这个 expectation 是对 measurement noise / random sampling / repeated experiments 的分布取的。也就是说，MSE 问的是：

on average, how far is my estimator from the true parameter?

6.2.1 Metric Should Follow the Purpose

Roy 在这里提醒一个很重要的 modelling principle：选 metric 之前，先问自己做 estimation 的初衷是什么。MSE 很常用，因为它把 bias 和 variance 合在一个 expected squared error 里；但它不是所有问题的默认最优目标。

例如 Roy 提到的物理学家场景：如果 physicist 真正关心的是某个 variable 是否有真实 physical effect，那么他可能更在意 coefficient bias，而不是只看 variance 或 MSE。因为一个 variance 很小但 systematically biased 的 estimate，会让人对物理机制作出错误解释。

可以这样建立直觉：

场景	更关心什么	为什么不一定优先 MSE
物理机制 / 科学解释	bias 小，coefficient 方向和大小可信	如果 estimator 有 systematic bias，即使 MSE 不大，也可能把某个 variable 的真实作用解释错

场景	更关心什么	为什么不一定优先 MSE
政策评估 / treatment effect	causal effect 的 bias 小	低 MSE prediction model 可能预测很好，但由于 confounding，不能给出可信 causal interpretation
参数报告 / confidence interval	variance / standard error 小，CI 窄且假设合理	目标是说明 estimate 有多 uncertain；此时 estimator distribution 和 interval coverage 比单一 MSE 更直接
工程监测 / sensor calibration	bias 接近 0	稳定但偏移的 sensor 会持续报错；长期 systematic offset 可能比 random fluctuation 更危险
控制和安全约束	worst-case error / constraint violation	MSE 是 average error；但 safety-critical system 更怕 rare but large error
预测任务 / model selection	validation/test MSE 或 RMSE	如果目标只是预测 unseen data，MSE 很自然，因为 bias 和 variance 都会伤害 prediction
高噪声小样本估计	variance 稳定性	可以接受一点 bias 换低 variance；regularisation 在这里有价值
变量筛选 / sparse interpretation	false inclusion / false exclusion, sparsity	LASSO 的目标不只是 MSE，还包括找出哪些 variables 可能真的有用
不想被 outliers 主导	MAE / median error / robust loss	Squared error 会放大 outliers；如果 outliers 不代表主要系统行为，MSE 可能误导
分类或风险筛查	recall, precision, F1, AUC	这些问题的错误类型不对称；例如漏诊和误报的代价不同，MSE 不是核心 metric

所以 metric 的选择应该是：

scientific explanation	-> care about bias and interpretability
statistical inference	-> care about SE, CI, coverage, significance
prediction	-> care about validation/test error
safety/control	-> care about worst-case or constraint violation
feature discovery	-> care about sparsity and selection reliability

这也是为什么 Roy 不只是说 “minimise MSE 就完了”。MSE 是一个很好的 total-error metric，但它隐含的价值判断是：

bias and variance should be traded off through squared average error.

如果你的真正目标不是 average squared error, 而是 scientific truth、causal interpretation、safe operation、或者 reliable variable selection, 那么你需要选择更贴近目标的 metric。

如果关心 prediction error, 实践中常用 held-out test set 估计:

$$\widehat{\text{MSE}}_{\text{test}} = \frac{1}{n_{\text{test}}} \sum_{i \in \text{test}} (y_i - \hat{y}_i)^2$$

Cross-validation 则反复切分数据, 计算多个 validation errors, 再取平均:

$$\widehat{\text{CV}} = \frac{1}{K} \sum_{k=1}^K \widehat{\text{MSE}}_{\text{valid},k}$$

所以要区分:

theoretical MSE

-> expectation under the data-generating process

estimated MSE

-> finite-data approximation using validation/test/CV/bootstrap

这里要区分三件事:

- θ_0 是 true parameter, 通常固定但未知;
- $\hat{\theta}$ 是 estimator, 是由 noisy data 算出来的 random variable;
- MSE 是 $\hat{\theta}$ 围绕 θ_0 的 expected squared error。

Bias:

$$\text{Bias}(\hat{\theta}) = E[\hat{\theta}] - \theta_0$$

所以 unbiased 的意思是:

$$E[\hat{\theta}] = \theta_0$$

也就是重复实验很多次后, 估计值的平均会落在 true parameter 上。它比较的是 estimator 的 expectation 和 true parameter, 不是某一次 $\hat{\theta}$ 和 θ_0 是否相等。

6.2.2 Derivation: Why Ordinary LS Is Unbiased

这里的前提是 ordinary least squares 的标准统计模型:

$$y = \Phi\theta_0 + \epsilon$$

其中:

- θ_0 是 true but unknown parameter;
- Φ 被当作 fixed / known design matrix;
- error 加在 y 上;
- $E[\epsilon] = 0$;
- Φ full column rank, 所以 $\Phi^T\Phi$ 可逆。

OLS estimator 是:

$$\hat{\theta} = (\Phi^T\Phi)^{-1}\Phi^Ty$$

把 true data-generating model 代进去:

$$\begin{aligned}\hat{\theta} &= (\Phi^T\Phi)^{-1}\Phi^T(\Phi\theta_0 + \epsilon) \\ &= (\Phi^T\Phi)^{-1}\Phi^T\Phi\theta_0 + (\Phi^T\Phi)^{-1}\Phi^T\epsilon \\ &= \theta_0 + (\Phi^T\Phi)^{-1}\Phi^T\epsilon.\end{aligned}$$

现在对 repeated experiments 取 expectation:

$$\begin{aligned}E[\hat{\theta}] &= E[\theta_0 + (\Phi^T\Phi)^{-1}\Phi^T\epsilon] \\ &= \theta_0 + (\Phi^T\Phi)^{-1}\Phi^TE[\epsilon] \\ &= \theta_0 + (\Phi^T\Phi)^{-1}\Phi^T0 \\ &= \theta_0.\end{aligned}$$

所以:

$$\text{Bias}(\hat{\theta}) = E[\hat{\theta}] - \theta_0 = 0.$$

这就是 ordinary LS unbiased 的推导。

需要注意: unbiasedness 不是说每一次估计都会等于真值, 而是说在同一个 data collection mechanism 下重复很多次, $\hat{\theta}$ 的长期平均值等于 θ_0 。

如果 $E[\epsilon] \neq 0$, 则:

$$E[\hat{\theta}] = \theta_0 + (\Phi^T\Phi)^{-1}\Phi^TE[\epsilon],$$

此时 estimator 一般会 biased。类似地，如果 Φ 本身有 measurement error，或者 ϵ 和 Φ 相关，那么 ordinary LS 的 unbiasedness 推导也会断掉；这就是 errors-in-variables / instrumental variables 要处理的问题之一。

Variance:

$$\text{Var}(\hat{\theta}) = E[(\hat{\theta} - E[\hat{\theta}])^2]$$

Variance 衡量的是 $\hat{\theta}$ 在重复实验中围绕自己的平均值 $E[\hat{\theta}]$ 波动多大。

Mean square error:

$$\text{MSE} = E[(\hat{\theta} - \theta_0)^2]$$

MSE 衡量的是 $\hat{\theta}$ 在重复实验中围绕 true parameter θ_0 的 squared error 平均有多大。Validation/test MSE 则是用 held-out data 估计 prediction error；它更接近 DATA423 model selection 里的实践问题。

注意：MSE 本身是一个 expectation，因此在理论上是一个确定的 quantity，不是 random variable。它包含 variance 部分，但不能简单说“MSE 具有 variance”。如果我们用有限 validation set、test set 或 repeated resampling 去估计 MSE，那么那个 estimated MSE 才会因为样本有限而有 sampling variability。

核心关系：

$$\text{MSE} = \text{bias}^2 + \text{variance}$$

6.2.3 Derivation: Why MSE = Bias Squared + Variance

先看 scalar estimator 的情况。令：

$$\mu_{\hat{\theta}} = E[\hat{\theta}]$$

MSE 定义为：

$$\text{MSE} = E[(\hat{\theta} - \theta_0)^2]$$

在括号里加减同一个量 $E[\hat{\theta}]$ ：

$$\hat{\theta} - \theta_0 = (\hat{\theta} - E[\hat{\theta}]) + (E[\hat{\theta}] - \theta_0)$$

所以：

$$\begin{aligned}\text{MSE} &= E \left[\left((\hat{\theta} - E[\hat{\theta}]) + (E[\hat{\theta}] - \theta_0) \right)^2 \right] \\ &= E[(\hat{\theta} - E[\hat{\theta}])^2] + 2(E[\hat{\theta}] - \theta_0)E[\hat{\theta} - E[\hat{\theta}]] + (E[\hat{\theta}] - \theta_0)^2\end{aligned}$$

中间项为 0，因为：

$$E[\hat{\theta} - E[\hat{\theta}]] = E[\hat{\theta}] - E[\hat{\theta}] = 0$$

因此：

$$\text{MSE} = E[(\hat{\theta} - E[\hat{\theta}])^2] + (E[\hat{\theta}] - \theta_0)^2$$

也就是：

$$\text{MSE} = \text{Var}(\hat{\theta}) + \text{Bias}(\hat{\theta})^2$$

这就是 Roy 提到的 result。

向量参数时，常见写法是看 squared norm：

$$\text{MSE} = E[\|\hat{\theta} - \theta_0\|_2^2]$$

对应分解为：

$$\text{MSE} = \text{tr}(\text{Cov}(\hat{\theta})) + \|\text{Bias}(\hat{\theta})\|_2^2$$

所以同样的直觉仍然成立：

```
total expected squared error
= variance part
+ systematic bias part
```

易错点：unbiased 不等于 minimum MSE。Unbiased 只是 bias 为 0，但 MSE 还包括 variance。

6.3 Statistical Consistency

Roy 这里提醒：consistency 有两个不同意思。

前面讲 normal equations 时，consistent 是线性代数里的意思：

the equation has at least one solution

例如 normal equations:

$$\Phi^T \Phi \theta = \Phi^T y$$

总是 algebraically consistent, 因为右边 $\Phi^T y$ 一定在 $\Phi^T \Phi$ 的 range 里。

但在 statistics 里, consistency 是 estimator 的性质。它问的是:

as sample size grows, does the estimator concentrate around the true value?

如果 $\hat{\theta}_N$ 是用 N 个 observations 算出来的 estimator, 那么 consistency 通常写成:

$$\hat{\theta}_N \xrightarrow{p} \theta_0$$

意思是对任意 fixed tolerance $\delta > 0$:

$$P(\|\hat{\theta}_N - \theta_0\| > \delta) \rightarrow 0 \quad \text{as } N \rightarrow \infty.$$

用 Roy 的话说:

as you take more and more data,
the probability of making a big error goes to zero.

也就是说, $\hat{\theta}_N$ 的 distribution 会越来越集中在 true parameter θ_0 附近。不是说每个 finite-sample estimate 都等于 θ_0 , 而是说随着 data 增加, large error 的概率消失。

6.3.1 Relation to Bias and Variance

Unbiasedness 和 consistency 不是同一个概念。

Unbiasedness 是 finite-sample expectation statement:

$$E[\hat{\theta}_N] = \theta_0.$$

Consistency 是 large-sample probability statement:

$$P(\|\hat{\theta}_N - \theta_0\| > \delta) \rightarrow 0.$$

一个 estimator 可以 finite-sample biased, 但仍然 consistent, 只要 bias 随 N 增加消失, variance 也收缩。Regularised estimators 就经常要从这个角度看: finite sample 里可能 biased, 但如果 regularisation 随 data size 合理减小, 仍可能 consistent。

对 ordinary LS, 在标准条件下, 如果:

- $E[\epsilon] = 0$;
- regressors 和 error 没有 problematic correlation;
- $\Phi^T \Phi$ 随着 N 增加提供越来越多 information;
- 没有 rank deficiency 持续存在;

那么更多 data 会让 estimator distribution 收缩, $\hat{\theta}_N$ 会越来越接近 θ_0 。

6.3.2 One-line Distinction

algebraic consistency:

an equation has a solution

statistical consistency:

an estimator converges to the true value as data increases

6.4 From Roy to STAT462: Confidence Intervals and Significance

Roy 的框架到这里已经给了关键对象:

$$\hat{\theta} = (\Phi^T \Phi)^{-1} \Phi^T y$$

以及在 white-noise assumptions 下:

$$\text{Cov}(\hat{\theta}) = \sigma^2 (\Phi^T \Phi)^{-1}.$$

STAT462 里讲的 confidence interval、standard error、coefficient significance, 本质上就是继续问:

we have `theta_hat`,

but how uncertain is each coefficient estimate?

也就是说, Roy 讲的是 estimator 的 distribution; STAT462 inference 是从这个 distribution 里读出 coefficient uncertainty。

6.4.1 Standard Error of a Coefficient

令:

$$C = (\Phi^T \Phi)^{-1}.$$

如果：

$$\text{Cov}(\hat{\theta}) = \sigma^2 C,$$

那么第 j 个 coefficient 的 variance 是 covariance matrix 的第 j 个 diagonal entry：

$$\text{Var}(\hat{\theta}_j) = \sigma^2 C_{jj}.$$

所以 standard error 是：

$$\text{se}(\hat{\theta}_j) = \sigma \sqrt{C_{jj}}.$$

实践中 σ^2 通常未知，所以用 residual variance estimate：

$$\hat{\sigma}^2 = \frac{\|y - \Phi \hat{\theta}\|_2^2}{N - d},$$

其中 N 是 observations 数量， d 是 coefficients 数量。因此：

$$\widehat{\text{se}}(\hat{\theta}_j) = \hat{\sigma} \sqrt{C_{jj}}.$$

这就是 R 的 `lm()` summary 里 **Std. Error** 的来源。

6.4.2 Confidence Interval for a Coefficient

在 Gaussian noise / standard linear-model assumptions 下，第 j 个 coefficient 的 confidence interval 可以写成：

$$\hat{\theta}_j \pm t_{1-\alpha/2, N-d} \widehat{\text{se}}(\hat{\theta}_j).$$

其中：

- α 是 significance level, 例如 $\alpha = 0.05$ 对应 95% confidence interval;
- $t_{1-\alpha/2, N-d}$ 是 t-distribution 的 quantile;
- $N - d$ 是 residual degrees of freedom.

解释时要小心：

The interval is random because it depends on the sampled data.

The true parameter is fixed but unknown.

Across repeated samples, 100(1-alpha)% of such intervals cover the true parameter.

不要说“这个具体区间有 95% 概率包含 true parameter”。更准确是：用同样 procedure 重复抽样，长期有约 95% 的 intervals 会 cover true parameter。

6.4.3 Significance of Coefficients

STAT462 的 MLR 里，R 的 `lm()` 会输出：

```
Estimate
Std. Error
t value
Pr(>|t|)
```

这和 Roy 的 $\hat{\theta}$ covariance 直接相连。

检验某个 feature 是否有显著线性贡献，通常看：

$$H_0 : \theta_j = 0$$

against:

$$H_1 : \theta_j \neq 0.$$

t-statistic 是：

$$t_j = \frac{\hat{\theta}_j - 0}{\widehat{\text{se}}(\hat{\theta}_j)}.$$

如果 p-value 小于 chosen significance level α ，或者等价地， $100(1 - \alpha)\%$ confidence interval 不包含 0，那么说 coefficient is statistically significant at level α 。

```
CI excludes 0
-> coefficient distinguishable from zero at that significance level

CI contains 0
-> data do not provide strong evidence that the coefficient differs from zero
```

这不是说这个 feature “一定没用”。它只是在当前 model、当前 data、当前 assumptions 下，coefficient 的 uncertainty 大到无法稳定地区分它和 0。

6.4.4 MLR Interpretation: “Given Other Features”

在 multiple linear regression 中：

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p + \epsilon.$$

第 j 个 coefficient 的意义是:

holding the other features fixed,
how does y change when x_j changes by one unit?

所以 coefficient significance 也是 conditional on the other columns in Φ .

这正好连接 Roy 讲的 Φ :

- 如果 two columns are highly correlated, C_{jj} 可能很大;
- C_{jj} 大会导致 standard error 大;
- standard error 大会导致 CI 宽、p-value 大;
- coefficient 可能看起来 insignificant。

因此 multicollinearity 不只是数值问题, 也会影响 statistical interpretation。

6.4.5 Confidence Interval vs Prediction Interval

STAT462 还区分 confidence interval 和 prediction interval。

Coefficient confidence interval answers:

How uncertain is θ_{j^*} ?

Mean-response confidence interval answers:

How uncertain is the fitted mean response $E[y \mid x_{j^*}]$?

Prediction interval answers:

How uncertain is a new observed y_{j^*} ?

Prediction interval usually wider, because it includes both:

uncertainty in fitted mean
+ new observation noise

This is why prediction intervals are not just coefficient CIs reused on y_{j^*} .

6.4.6 MSE, R-squared, and Model Selection

STAT462 uses MSE/RSS/R-squared for model comparison. In Roy's framework:

$RSS = ||y - \Phi \hat{\theta}||_2^2$
 $MSE = RSS / N$ or estimated prediction error on held-out data

Training RSS almost always decreases when you add more columns to Φ . But this can be overfitting.

So the data-mining workflow is:

```
fit candidate models on training data
compare validation MSE
choose model/hyperparameter
report test MSE as final performance estimate
```

This is the same idea already used in the regularisation section:

```
training fit is not the same as future prediction performance
```

6.4.7 How This Fits Roy's Big Picture

Roy's least-squares story:

```
choose Phi
-> estimate theta_hat
-> understand estimator bias/variance/MSE
-> maybe use BLUE or regularisation
```

STAT462 inference adds:

```
use Cov(theta_hat)
-> compute standard errors
-> build confidence intervals
-> test coefficient significance
-> decide whether features are statistically credible
```

So the views are complementary:

Roy / ENEL445 view	STAT462 view
estimator covariance	standard errors
uncertainty in $\hat{\theta}$	confidence intervals
regressor rank / conditioning	coefficient stability
Φ columns nearly dependent	large SE / insignificant coefficients
validation MSE	model selection / generalisation

6.5 BLUE

BLUE = Best Linear Unbiased Estimator.

如果噪声 zero mean 且 covariance 为 R :

$$E[\epsilon\epsilon^T] = R$$

则 BLUE:

$$\hat{\theta}_{\text{BLUE}} = (\Phi^T R^{-1} \Phi)^{-1} \Phi^T R^{-1} y$$

其 covariance:

$$\text{Cov}(\hat{\theta}_{\text{BLUE}}) = (\Phi^T R^{-1} \Phi)^{-1}$$

当 $R = \sigma^2 I$ 时, BLUE 退化为普通 LS。

6.6 Variance and Signal-to-Noise Ratio

Roy 说 estimator variance 本质上是在告诉你 inverse signal-to-noise ratio。可以从最简单的一参数模型看出来。

假设:

$$y = \phi\theta + \epsilon$$

其中:

$$\phi = \begin{bmatrix} \phi_1 \\ \phi_2 \\ \vdots \\ \phi_N \end{bmatrix}, \quad E[\epsilon] = 0, \quad \text{Cov}(\epsilon) = \sigma^2 I.$$

OLS estimator 是:

$$\hat{\theta} = (\phi^T \phi)^{-1} \phi^T y$$

把 true model 代入:

$$\begin{aligned} \hat{\theta} &= (\phi^T \phi)^{-1} \phi^T (\phi\theta + \epsilon) \\ &= (\phi^T \phi)^{-1} \phi^T \phi\theta + (\phi^T \phi)^{-1} \phi^T \epsilon \\ &= \theta + (\phi^T \phi)^{-1} \phi^T \epsilon. \end{aligned}$$

所以估计误差是:

$$\hat{\theta} - \theta = (\phi^T \phi)^{-1} \phi^T \epsilon.$$

现在计算 variance。使用规则：

$$\text{Var}(A\epsilon) = A\text{Cov}(\epsilon)A^T$$

这里：

$$A = (\phi^T \phi)^{-1} \phi^T$$

因此：

$$\begin{aligned} \text{Var}(\hat{\theta}) &= (\phi^T \phi)^{-1} \phi^T (\sigma^2 I) \phi (\phi^T \phi)^{-1} \\ &= \sigma^2 (\phi^T \phi)^{-1} \phi^T \phi (\phi^T \phi)^{-1} \\ &= \sigma^2 (\phi^T \phi)^{-1}. \end{aligned}$$

因为这里是 scalar one-regressor case：

$$\phi^T \phi = \sum_{i=1}^N \phi_i^2$$

所以：

$$\text{Var}(\hat{\theta}) = \frac{\sigma^2}{\sum_{i=1}^N \phi_i^2}.$$

这就是 signal-to-noise ratio 的直觉：

```
noise power      = sigma^2
regressor signal = sum phi_i^2
estimator variance = noise power / regressor signal
```

因此：

$$\text{Var}(\hat{\theta}) \propto \frac{1}{\text{SNR}}.$$

如果 ϕ 的 magnitude 大、variation 充分，那么：

$$\sum_i \phi_i^2 \text{ large} \Rightarrow \text{Var}(\hat{\theta}) \text{ small}.$$

如果 ϕ 很小、变化不足，或者所有 observation 都集中在很窄的范围内，那么：

$$\sum_i \phi_i^2 \text{ small} \Rightarrow \text{Var}(\hat{\theta}) \text{ large.}$$

直觉上，如果你想估计 slope，但 input/regressor 几乎不动，那么一点点 output noise 就会让 slope estimate 大幅摆动。反过来，如果 regressor 覆盖范围很大，signal 很强，同样的 noise 对 estimate 的影响就小很多。

对多参数模型：

$$\text{Cov}(\hat{\theta}) = \sigma^2 (\Phi^T \Phi)^{-1}$$

扮演同样角色。 $\Phi^T \Phi$ 描述 regressor signal 在不同 parameter directions 上有多强；如果某些方向 signal weak 或 columns nearly dependent，那么 $(\Phi^T \Phi)^{-1}$ 会很大，estimate variance 也会很大。

6.7 Worked Exercise: Inverse-variance Weighting

Roy 说 “things that have small variance are valuable estimates”，对应的标准推导就是 inverse-variance weighting。这个很可能作为 BLUE 的基础推导题出现。

假设我们有 n 个 independent unbiased estimates：

$$\hat{\theta}_1, \dots, \hat{\theta}_n$$

其中：

$$E[\hat{\theta}_i] = \theta, \quad \text{Var}(\hat{\theta}_i) = \sigma_i^2.$$

我们把它线性组合：

$$\hat{\theta}_w = \sum_{i=1}^n w_i \hat{\theta}_i.$$

6.7.1 Step 1: Unbiasedness Constraint

先计算 expectation：

$$\begin{aligned}
E[\hat{\theta}_w] &= E\left[\sum_{i=1}^n w_i \hat{\theta}_i\right] \\
&= \sum_{i=1}^n w_i E[\hat{\theta}_i] \\
&= \sum_{i=1}^n w_i \theta \\
&= \theta \sum_{i=1}^n w_i.
\end{aligned}$$

如果希望 $\hat{\theta}_w$ unbiased, 就要:

$$E[\hat{\theta}_w] = \theta.$$

因此:

$$\theta \sum_{i=1}^n w_i = \theta$$

所以 constraint 是:

$$\sum_{i=1}^n w_i = 1.$$

6.7.2 Step 2: Variance of the Weighted Estimate

因为 estimates independent:

$$\text{Var}\left(\sum_{i=1}^n w_i \hat{\theta}_i\right) = \sum_{i=1}^n w_i^2 \text{Var}(\hat{\theta}_i).$$

所以:

$$\text{Var}(\hat{\theta}_w) = \sum_{i=1}^n w_i^2 \sigma_i^2.$$

BLUE 的问题就是:

$$\min_{w_1, \dots, w_n} \sum_{i=1}^n w_i^2 \sigma_i^2 \quad \text{subject to} \quad \sum_{i=1}^n w_i = 1.$$

6.7.3 Step 3: Lagrangian

写 Lagrangian:

$$L(w, \lambda) = \sum_{i=1}^n w_i^2 \sigma_i^2 + \lambda \left(\sum_{i=1}^n w_i - 1 \right).$$

对 w_i 求偏导:

$$\frac{\partial L}{\partial w_i} = 2w_i \sigma_i^2 + \lambda.$$

令一阶条件为 0:

$$2w_i \sigma_i^2 + \lambda = 0.$$

所以:

$$w_i = -\frac{\lambda}{2\sigma_i^2}.$$

现在令:

$$c = -\frac{\lambda}{2}.$$

那么:

$$w_i = \frac{c}{\sigma_i^2}.$$

这一步说明, optimal weight 一定和 $1/\sigma_i^2$ 成比例:

$$w_i \propto \frac{1}{\sigma_i^2}.$$

6.7.4 Step 4: Use the Constraint to Find c

我们还不知道 c 。用 unbiasedness constraint:

$$\sum_{i=1}^n w_i = 1.$$

代入:

$$w_i = \frac{c}{\sigma_i^2}$$

得到:

$$\sum_{i=1}^n \frac{c}{\sigma_i^2} = 1.$$

因为 c 对所有 i 都相同, 所以可以提出来:

$$c \sum_{i=1}^n \frac{1}{\sigma_i^2} = 1.$$

因此:

$$c = \frac{1}{\sum_{j=1}^n 1/\sigma_j^2}.$$

把 c 放回:

$$w_i = \frac{c}{\sigma_i^2}$$

得到:

$$w_i = \frac{1}{\sigma_i^2} \frac{1}{\sum_{j=1}^n 1/\sigma_j^2}.$$

也就是:

$$w_i = \frac{1/\sigma_i^2}{\sum_{j=1}^n 1/\sigma_j^2}.$$

这就是 inverse-variance weighting。

6.7.5 Step 5: Interpretation

如果 σ_i^2 很大:

$$\frac{1}{\sigma_i^2} \text{ is small} \quad \Rightarrow \quad w_i \text{ is small.}$$

如果 σ_i^2 很小:

$$\frac{1}{\sigma_i^2} \text{ is large} \Rightarrow w_i \text{ is large.}$$

所以:

```
high variance estimate -> unreliable -> small weight
low variance estimate  -> valuable   -> large weight
```

注意, 不是把 high-variance estimates 扔掉, 而是降低它们的权重。它们仍然包含信息, 只是信息质量较低。

6.8 Why Least Squares Does the Weighting in One Shot

Roy 接着说: 我们可以先对每个 measurement 单独算一个 slope estimate, 再用 inverse-variance weighting 平均; 但 least squares 可以一次性做到同样的事。

这句话的背景是一个 scalar gain model:

$$y_i = x_i g + \epsilon_i, \quad \epsilon_i \sim (0, \sigma^2).$$

如果只用第 i 个 measurement, 可以得到一个单点 slope estimate:

$$\hat{g}_i = \frac{y_i}{x_i}.$$

因为:

$$y_i = x_i g + \epsilon_i$$

所以:

$$\hat{g}_i = \frac{x_i g + \epsilon_i}{x_i} = g + \frac{\epsilon_i}{x_i}.$$

因此:

$$\text{Var}(\hat{g}_i) = \text{Var}\left(\frac{\epsilon_i}{x_i}\right) = \frac{\sigma^2}{x_i^2}.$$

这说明:

```
large |x_i| -> small slope-estimate variance -> more informative
small |x_i| -> large slope-estimate variance -> less informative
```

如果手工做 inverse-variance weighting, 权重是:

$$w_i = \frac{1/\text{Var}(\hat{g}_i)}{\sum_j 1/\text{Var}(\hat{g}_j)}.$$

代入:

$$\text{Var}(\hat{g}_i) = \frac{\sigma^2}{x_i^2}$$

得到:

$$\begin{aligned} w_i &= \frac{1/(\sigma^2/x_i^2)}{\sum_j 1/(\sigma^2/x_j^2)} \\ &= \frac{x_i^2/\sigma^2}{\sum_j x_j^2/\sigma^2} \\ &= \frac{x_i^2}{\sum_j x_j^2}. \end{aligned}$$

所以手工 BLUE estimate 是:

$$\begin{aligned} \hat{g}_{\text{BLUE}} &= \sum_i w_i \hat{g}_i \\ &= \sum_i \frac{x_i^2}{\sum_j x_j^2} \frac{y_i}{x_i} \\ &= \frac{\sum_i x_i y_i}{\sum_j x_j^2}. \end{aligned}$$

现在看 ordinary least squares。把所有 observations 写成:

$$y = xg + \epsilon$$

其中:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}.$$

OLS 直接给:

$$\hat{g}_{\text{LS}} = (x^T x)^{-1} x^T y.$$

因为:

$$x^T x = \sum_i x_i^2, \quad x^T y = \sum_i x_i y_i,$$

所以:

$$\hat{g}_{\text{LS}} = \frac{\sum_i x_i y_i}{\sum_i x_i^2}.$$

这和手工 inverse-variance weighted average 完全一样:

$$\hat{g}_{\text{LS}} = \hat{g}_{\text{BLUE}}.$$

这就是 Roy 说 LS “does it in one shot” 的意思:

```
manual route:
    individual slope estimates
    -> compute each variance
    -> inverse-variance weights
    -> weighted average

least-squares route:
    write y = xg + epsilon
    -> solve normal equation
    -> same weighted average appears automatically
```

6.8.1 What Does “Automatically” Mean?

这里的 automatically 不是说 LS 神奇地知道所有 noise variance, 而是说:

under the standard homoskedastic noise assumption, the algebra of LS already gives each observation influence proportional to its information content.

在这个 scalar gain example 里, information content 来自 x_i^2 。同样的 output noise σ^2 下, larger $|x_i|$ 给出的 slope estimate variance 更小, 所以 LS 解中它的 effective weight 更大。

也可以把 LS 写成:

$$\hat{g}_{\text{LS}} = \sum_i \left(\frac{x_i^2}{\sum_j x_j^2} \right) \left(\frac{y_i}{x_i} \right).$$

括号里的第一项就是 weight, 第二项就是单点 slope estimate。

6.8.2 Is This a General Property?

Yes, but with an important qualification.

For ordinary LS, the Gauss-Markov result says: 如果 errors zero mean、uncorrelated、same variance, 并且 Φ full column rank, 那么 OLS is BLUE。也就是说, 在这些假设下, OLS 自动给出 best linear unbiased estimator。

但如果 measurement noise 的 variance 不相同:

$$\text{Cov}(\epsilon) = R \neq \sigma^2 I$$

ordinary LS 不再自动给正确的 noise weighting。这时需要 weighted / generalized least squares:

$$\hat{\theta}_{\text{BLUE}} = (\Phi^T R^{-1} \Phi)^{-1} \Phi^T R^{-1} y.$$

这里的 R^{-1} 才是显式的 inverse-noise-covariance weighting:

```
low noise variance -> larger weight
high noise variance -> smaller weight
correlated noise   -> account for correlation structure
```

所以要分清两层:

ordinary LS:

```
automatically weights information through Phi under equal-noise assumptions
```

weighted / generalized LS:

```
explicitly weights measurements using  $R^{-1}$  when noise variances/correlations differ
```

这也是为什么 Roy 先用 inverse-variance weighting 讲直觉, 再说 LS 一步就做到: 在标准条件下, normal equations 已经把同一个 BLUE 结果编码进去了。

6.9 Correlated Noise: White Noise vs General BLUE

Roy 后面说, 如果 noise 之间有 correlation, electrical engineer 会说 noise has frequency content。直觉是:

white noise:

```
no time/sample-to-sample correlation
no preferred frequency structure
```

correlated noise:

```
errors have pattern, memory, drift, oscillation, or frequency content
```

在 ordinary LS 的标准 white-noise case:

$$\text{Cov}(\epsilon) = \sigma^2 I.$$

这里的 identity matrix 表示:

- 每个 measurement noise variance 相同;
- 不同 measurements 的 errors uncorrelated;
- covariance matrix 没有 off-diagonal structure。

这时 ordinary LS:

$$\hat{\theta}_{\text{OLS}} = (\Phi^T \Phi)^{-1} \Phi^T y$$

就是 BLUE。

但如果:

$$\text{Cov}(\epsilon) = R$$

其中 R 不是 $\sigma^2 I$, 例如:

$$R = \begin{bmatrix} \sigma_1^2 & r_{12} & \cdots \\ r_{21} & \sigma_2^2 & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix},$$

那么 off-diagonal entries 表示:

$$E[\epsilon_i \epsilon_j] \neq 0.$$

也就是说, 第 i 个 error 和第 j 个 error 不是独立乱动, 而是有共同变化结构。

普通 LS 在这个情况下通常仍然 unbiased, 但不再是 minimum-variance unbiased estimator。BLUE 变成:

$$\hat{\theta}_{\text{BLUE}} = (\Phi^T R^{-1} \Phi)^{-1} \Phi^T R^{-1} y.$$

这里的 R^{-1} 是 inverse covariance weighting。它的作用不是简单地给每个 point 一个 scalar weight, 而是按整个 noise covariance structure 重新度量 residual:

$$\min_{\theta} (y - \Phi\theta)^T R^{-1} (y - \Phi\theta).$$

如果 $R = \sigma^2 I$, 则:

$$R^{-1} = \frac{1}{\sigma^2} I.$$

这个 scalar factor 会在 normal equations 中 cancel 掉, 所以退化回 ordinary LS。

6.9.1 Correlation vs Independence

Roy 提到可以查 correlation 和 independence 的区别。大致关系是:

`independence is stronger than zero correlation.`

如果两个 random variables independent, 那么它们没有任何 probabilistic dependence。知道一个变量的值, 不会改变另一个变量的 distribution。

如果两个变量 uncorrelated, 只表示它们的 linear covariance 为 0:

$$\text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])] = 0.$$

这只排除了 linear relationship, 不排除 nonlinear relationship。

例如:

$$X \sim \text{symmetric around } 0, \quad Y = X^2.$$

则 Y 完全由 X 决定, 所以它们绝对不 independent。但由于 symmetry, 可能有:

$$\text{Cov}(X, Y) = \text{Cov}(X, X^2) = 0.$$

所以:

`independent -> uncorrelated`

`uncorrelated -> not necessarily independent`

在 Gaussian/noise modelling 里有一个特殊好处: 如果 variables jointly Gaussian, 那么 zero covariance implies independence。这也是为什么 Gaussian white noise 是一个很强、很方便的假设。

6.9.2 Exam Risk

这部分中高概率会考, 因为它直接连接 Roy 的 BLUE 结论。

高概率考法:

- 识别 ordinary LS 什么时候是 BLUE;
- 写出 correlated noise 下的 BLUE;
- 解释 R^{-1} 的作用;
- 说明如果 $R = \sigma^2 I$, BLUE 退化为 ordinary LS;
- 解释为什么实践中难点是估计 R 。

较低概率考法:

- 完整证明该 estimator 在所有 unbiased linear estimators 中 covariance 最小。

考试题型:

- 证明 $Z^T \Phi = I$ 是 linear estimator $\hat{\theta} = Z^T y$ unbiased 的条件;
- 推导 $\text{Cov}(\hat{\theta}) = Z^T R Z$;
- 写出 BLUE;
- 解释为什么 low variance measurement 权重更大。

7 Regularisation and Validation

7.1 Why Regularisation Is Needed

普通 LS 给出 best linear unbiased estimate, 但未必最小化 MSE。Regularisation 的作用:

- 降低 variance;
- 改善 ill-conditioned problem;
- 用一点 bias 换更好的 prediction。

Regularised LS:

$$\min_{\theta} \|y - \Phi\theta\|_2^2 + \eta\|\theta\|_2^2$$

解为:

$$\hat{\theta} = (\Phi^T \Phi + \eta I)^{-1} \Phi^T y$$

当 $\eta = 0$, 回到 ordinary LS。随着 η 增大:

- solution 被 shrink;
- variance 下降;
- bias 上升;
- conditioning 变好。

7.2 Old Memory Hook: Ridge, LASSO, ElasticNet, AIC, BIC

Roy 这里的 general template 是:

$$\min_{\theta} \underbrace{\|y - \Phi\theta\|_2^2}_{\text{fit the data}} + \underbrace{\eta J(\theta)}_{\text{penalise parameter complexity}}.$$

如果只保留第一项，就是 ordinary LS / BLUE under standard assumptions。加上第二项之后，我们就在说：
do not only fit the data;
also prefer a parameter vector theta that is simpler / smaller / more plausible.

这正好连接 STAT448 / data mining 里学过的 Ridge、LASSO、ElasticNet、AIC、BIC。

7.2.1 正则化的好处：Roy + STAT448 总结

Roy 在这一段把 regularisation 的好处说得很工程化。它不只是为了让 training error 看起来更小，而是在 ordinary LS 解决不了的地方加约束：

来源	核心点	什么时候重要	实际含义
Roy Smith	改善 conditioning，让 inverse 更可用	$\Phi^T\Phi$ singular、rank-deficient、under-determined，或者 condition number 很大	加上 ηI 或有结构的 R 后， $(\Phi^T\Phi + \eta I)$ 更容易稳定求逆， $\hat{\theta}$ 对小噪声或小数据扰动不那么敏感
Roy Smith	在 poor data 或 measurement 不够时稳定估计	实验设计不好、 x 取值很差，或者 measurements 不足以识别所有参数	ordinary LS 可能给出很 noisy / unreliable 的参数；regularisation 会把参数往更合理、更小、更稳定的方向拉
Roy Smith	用来研究 bias-variance tradeoff	BLUE / ordinary LS 虽然 unbiased，但 variance 可能很大	接受一点 bias，换取 variance 明显下降；目标从“只要 unbiased”变成“让 MSE 更小”
Roy Smith	需要用 validation 找平衡	η 选得太大或太小	太少 regularisation 会留下 instability；太多 regularisation 会让 penalty 主导答案，而不是 data 主导答案

来源	核心点	什么时候重要	实际含义
STAT448	Ridge	很多 variables 都有 useful signal	把很多 coefficients 一起 shrink; 通常保留所有 predictors, 适合 correlated / collinear predictors
STAT448	LASSO	只有少数 variables 有 useful signal	shrink 的同时可以把一些 coefficients 精确变成 0, 所以等价于做 feature selection
STAT448	Elastic Net	signal 强弱不一, 而且 predictors 之间有 correlation	结合 Ridge 的稳定性和 LASSO 的 sparsity, 适合 mixed / correlated signals
STAT448	三个都要试	真实 signal pattern 事先不知道	guideline 只能帮助判断方向; 最终还是要用 validation / cross-validation 比较哪个效果最好

一句话记忆:

Roy: 正则化主要解决 `unstable estimation`, 并用一点 `bias` 换更低 `variance` / MSE。

STAT448: Ridge 适合 `many useful predictors`; LASSO 适合 `few useful predictors`; ElasticNet 适合 `mixed` /

7.2.2 Ridge Regression: L_2 Penalty

Ridge uses squared Euclidean norm:

$$J(\theta) = \|\theta\|_2^2.$$

Objective:

$$\min_{\theta} \|y - \Phi\theta\|_2^2 + \eta\|\theta\|_2^2.$$

Solution:

$$\hat{\theta}_{\text{ridge}} = (\Phi^T\Phi + \eta I)^{-1}\Phi^T y.$$

Roy's interpretation:

```
penalise large theta
-> shrink coefficients
-> improve conditioning
-> reduce variance
-> accept some bias
```

Ridge is especially natural when:

- many predictors have some useful signal;
- columns of Φ are correlated;
- $\Phi^T \Phi$ is ill-conditioned;
- you want stable prediction rather than sparse interpretation.

Important detail: usually the intercept is not penalised, and features are standardised before applying ridge. Otherwise variables with larger units get unfairly penalised.

7.2.3 Tikhonov Regularisation: Weighted L_2 Penalty

Roy 接着说, Ridge 的 ηI 只是最简单的版本。更一般地, 可以惩罚一个 weighted two-norm:

$$J(\theta) = \theta^T R \theta, \quad R \succ 0.$$

对应 objective:

$$\min_{\theta} \|y - \Phi\theta\|_2^2 + \theta^T R \theta.$$

这通常称为 Tikhonov regularisation。若:

$$R = \eta I,$$

就退化成 ordinary ridge:

$$\|y - \Phi\theta\|_2^2 + \eta \|\theta\|_2^2.$$

如果 R 是一般 positive definite matrix, 那么它表达的是:

```
different parameter directions are penalised differently.
```

例如:

$$R = \begin{bmatrix} \eta_1 & 0 \\ 0 & \eta_2 \end{bmatrix}$$

表示:

- θ_1 的 penalty strength 是 η_1 ;
- θ_2 的 penalty strength 是 η_2 。

如果 R 有 off-diagonal entries, 则 penalty 还会表达 parameter directions 之间的 coupling。

Normal equation 变成:

$$(\Phi^T \Phi + R)\theta = \Phi^T y.$$

所以:

$$\hat{\theta}_{\text{Tik}} = (\Phi^T \Phi + R)^{-1} \Phi^T y.$$

7.2.4 Why Roy Warns About Arbitrary R

如果 $\theta \in \mathbb{R}^d$, 那么一个 arbitrary symmetric matrix R 有大约:

$$\frac{d(d+1)}{2}$$

个 free parameters。

这意味着:

`choosing R itself becomes a high-dimensional model-selection problem.`

如果 d 很大, 直接让 R 是任意 positive definite matrix 会造成:

- hyperparameter dimension 爆炸;
- validation data 不够;
- computation 变大;
- 你可能只是把 overfitting 从 θ 转移到了 R 。

所以 Roy 说更常见的做法是 parameterise R by one or two parameters。也就是:

`do not search over every possible positive definite matrix;`
`choose a structured family R(gamma).`

例如:

$$R(\eta) = \eta I$$

只有一个 hyperparameter, 这就是 ridge。

或者:

$$R(\eta_1, \eta_2) = \eta_1 I + \eta_2 L^T L,$$

其中 L 可以表示某种 smoothness / difference operator。这样只需要调 η_1, η_2 , 但 R 仍然有明确结构。

Roy 的重点是:

regularisation should encode useful structure,
not introduce an enormous number of new unknowns.

7.2.5 Connection to Big Data / Statistical Learning

Big data / data mining 里讲 regularisation 时, 常见语言是:

control model complexity
avoid overfitting
improve validation/test performance

Roy 的 engineering view 更强调:

ill-conditioned inverse
poorly identifiable theta
unrealistically large parameter estimates
need structured prior information about plausible theta

二者本质相同:

Big data view	Roy / engineering view
high-dimensional features	many parameters / insufficient measurements
overfitting	noise creates unstable large θ
regularisation controls complexity	regularisation improves conditioning
tune hyperparameter by CV	choose low-dimensional parameterisation and validate
prior preference for simple model	physical/engineering belief about plausible parameters

因此 Tikhonov regularisation 可以理解成:

Ridge is isotropic shrinkage:

all directions penalised equally

Tikhonov is structured shrinkage:

some directions penalised more than others

但 Roy 的 warning 是: 如果你 make the structure too flexible, then choosing the regulariser becomes another high-dimensional estimation problem.

7.2.6 LASSO: L_1 Penalty

LASSO uses the one-norm:

$$J(\theta) = \|\theta\|_1 = \sum_j |\theta_j|.$$

Objective:

$$\min_{\theta} \|y - \Phi\theta\|_2^2 + \lambda \|\theta\|_1.$$

Key effect:

LASSO can set some coefficients exactly to zero.

So LASSO does two things at once:

- shrinkage;
- feature selection.

In Φ language:

```
theta_j = 0
-> column j of Phi is effectively removed from the model.
```

LASSO is especially useful when:

- many candidate features exist;
- only a few features are expected to matter;
- interpretability / sparse model is valuable.

Compared with ridge:

```
ridge:
  shrinks many coefficients, usually keeps all features
```

LASSO:

shrinks and can delete features by setting coefficients to zero

7.2.7 ElasticNet: Mixture of Ridge and LASSO

ElasticNet combines L_2 and L_1 penalties:

$$\min_{\theta} \|y - \Phi\theta\|_2^2 + \lambda(1 - \alpha)\|\theta\|_2^2 + \lambda\alpha\|\theta\|_1.$$

Here:

```
alpha = 0 -> Ridge
alpha = 1 -> LASSO
0 < alpha < 1 -> ElasticNet
```

ElasticNet is a compromise:

- L_2 part stabilises correlated predictors;
- L_1 part encourages sparsity;
- useful when predictors are correlated and only some groups/features matter.

In sklearn, the same idea appears as:

```
ElasticNet(alpha=..., l1_ratio=...)
```

where `alpha` controls total penalty strength and `l1_ratio` controls the LASSO-vs-Ridge mixture.

7.2.8 AIC and BIC: Model Selection Penalties

AIC/BIC are related but conceptually different.

Ridge/LASSO/ElasticNet add a penalty directly to the coefficient-fitting objective:

fit error + coefficient penalty.

AIC/BIC usually compare whole fitted models using:

model fit + penalty for number of parameters

Typical forms are:

$$\text{AIC} = -2 \log L + 2k$$

and:

$$\text{BIC} = -2 \log L + k \log N,$$

where:

- L is likelihood;
- k is number of estimated parameters;
- N is sample size.

Interpretation:

better fit lowers $-2 \log L$
more parameters increase penalty

AIC/BIC are therefore closer to model selection criteria:

compare model A vs model B
choose lower AIC/BIC

They do not usually shrink coefficients continuously like ridge or LASSO. Instead, they penalise model complexity at the model level.

Rough intuition:

Tool	Penalty target	Main effect
Ridge	$\ \theta\ _2^2$	shrink coefficients, stabilise collinearity
LASSO	$\ \theta\ _1$	shrink and select variables
ElasticNet	mixed L_1/L_2	sparse selection with correlated-feature stability
AIC	parameter count, lighter penalty	predictive model selection tendency
BIC	parameter count, stronger with large N	more conservative model selection tendency

7.2.9 How to Choose the Penalty Strength

Roy's (), STAT448's (), and sklearn's `alpha` all play the same conceptual role:

regularisation strength / hyperparameter

But the scale can differ across software and objective conventions.

In practice:

choose eta/lambda/alpha by validation or cross-validation

because the theoretically optimal value depends on unknown quantities such as:

- true parameter θ_0 ;
- noise level σ^2 ;
- direction structure of $\Phi^T \Phi$;
- prediction target and data-generating mechanism.

7.2.10 One Mental Picture

OLS:

fit data as closely as possible

Ridge:

fit data, but avoid large unstable coefficients

LASSO:

fit data, but prefer many coefficients exactly zero

ElasticNet:

fit data, shrink correlated groups, and allow sparsity

AIC/BIC:

fit data, but penalise model size when comparing candidate models

7.3 Bias-variance Tradeoff

Roy 在这里问了一个很关键的问题:

When choosing an estimator, what are we actually trying to minimise?

候选目标至少有三个:

Objective	What it asks for	Typical tool
minimise bias	make $E[\hat{\theta}]$ close to θ_0	unbiased estimator
minimise variance	make $\hat{\theta}$ stable across repeated experiments	BLUE, inverse-variance weighting
minimise MSE	minimise total expected squared error	regularisation / shrinkage

它们之间的关系是：

$$\text{MSE} = \text{Bias}^2 + \text{Variance}$$

所以 minimum variance 不一定 minimum MSE, zero bias 也不一定 minimum MSE。

7.3.1 What BLUE Optimises

BLUE means:

Best Linear Unbiased Estimator

这里的 “Best” 不是所有意义下都最好，而是在一个受限集合里最好：

**among linear unbiased estimators,
choose the one with minimum variance**

也就是说 BLUE 的逻辑是：

force bias = 0, then minimise variance.

所以 ordinary LS / weighted LS 这条线非常适合：

- 参数 θ 本身有物理、工程或经济解释；
- 你希望长期平均估计值等于 true parameter；
- 你要在 unbiased 这个约束下让 estimate 尽量稳定。

但它不保证 minimum MSE，因为它不允许用 bias 换 variance。

7.3.2 What Minimum MSE Optimises

Minimum MSE 的目标是：

$$\min E[(\hat{\theta} - \theta_0)^2]$$

它允许：

**small bias increase
-> larger variance decrease
-> lower total MSE**

这就是 shrinkage / regularisation 的思想。Ridge regularisation 会把 coefficient 往 0 拉：

$$\hat{\theta}_\eta = (\Phi^T \Phi + \eta I)^{-1} \Phi^T y$$

它通常不再 unbiased，但可能有更小的 MSE，尤其是在：

- data noisy;
- columns of Φ highly correlated;
- $\Phi^T \Phi$ ill-conditioned;
- number of features large;
- prediction on future data 比解释 exact parameter 更重要。

7.3.3 Scalar Gain η vs Vector Signal-to-noise View

Roy 在 scalar gain example 里会写类似：

$$y_i = G_0 x_i + \epsilon_i,$$

其中：

$$G_0 = \theta_0$$

是 true gain / true parameter，不是 Φ 。对应到 OLS notation：

$$\Phi = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}, \quad \theta_0 = G_0.$$

在这个 scalar case 中，minimum-MSE shrinkage 的 tuning scale 会出现类似：

$$\eta \sim \frac{\sigma^2}{|G_0|^2}.$$

它的直觉是：

noise strength / true signal strength

也就是说：

- σ^2 large $\Rightarrow$ noise strong $\Rightarrow$ more shrinkage useful;
- $|G_0|^2$ large $\Rightarrow$ true gain strong $\Rightarrow$ less shrinkage needed;
- $|G_0|^2$ small $\Rightarrow$ true gain close to zero $\Rightarrow$ shrink toward zero more useful.

但这只是 one-dimensional intuition。向量情形中：

$$y = \Phi \theta_0 + \epsilon, \quad \theta_0 \in \mathbb{R}^d,$$

不能简单说:

$$\eta = \frac{\sigma^2}{\|\theta_0\|_2^2}$$

就是一般答案。这个式子只能作为 rough intuition:

overall noise strength / overall parameter magnitude

更准确地说, vector case 的 MSE 是 directional 的。Ridge estimator 是:

$$\hat{\theta}_\eta = (\Phi^T \Phi + \eta I)^{-1} \Phi^T y.$$

在 model $y = \Phi \theta_0 + \epsilon$ 下:

$$E[\hat{\theta}_\eta] = (\Phi^T \Phi + \eta I)^{-1} \Phi^T \Phi \theta_0.$$

所以 bias 是:

$$\begin{aligned} \text{Bias}(\hat{\theta}_\eta) &= E[\hat{\theta}_\eta] - \theta_0 \\ &= [(\Phi^T \Phi + \eta I)^{-1} \Phi^T \Phi - I] \theta_0 \\ &= -\eta (\Phi^T \Phi + \eta I)^{-1} \theta_0. \end{aligned}$$

Variance contribution is:

$$\text{Cov}(\hat{\theta}_\eta) = \sigma^2 (\Phi^T \Phi + \eta I)^{-1} \Phi^T \Phi (\Phi^T \Phi + \eta I)^{-1}.$$

因此 vector MSE 可以理解成:

$$\text{MSE} = \underbrace{\|\text{Bias}(\hat{\theta}_\eta)\|_2^2}_{\text{depends on } \theta_0 \text{ and } \Phi} + \underbrace{\text{tr}(\text{Cov}(\hat{\theta}_\eta))}_{\text{depends on } \sigma^2 \text{ and } \Phi}.$$

这就是更高维的 SNR view:

scalar case:

SNR controlled by $|G_0|^2 / \sigma^2$

vector case:

SNR is directional:

each parameter direction has its own signal strength and noise amplification

如果把：

$$\Phi^T \Phi$$

看成 parameter directions 的 information matrix，那么：

- large eigenvalue direction: data strongly identifies that direction;
- small eigenvalue direction: data weakly identifies that direction;
- ridge shrinkage affects weak directions more strongly;
- θ_0 落在哪些 directions 上，会决定 bias contribution;
- σ^2 决定 noise-driven variance contribution。

因此更准确的 mental model 是：

MSE = signal-direction bias cost + noise-direction variance cost

这也是为什么 practical η 通常不能直接由 formula 算出来： θ_0 unknown，而且 Φ 的 directional information structure matters。实际做法还是 validation / cross-validation。

Roy 后面说的 counterintuitive point 就是：

best fit to the experimental data is not necessarily best expected fit to future data.

普通 LS 很擅长 fit 当前 training data，并且在标准假设下给出 BLUE。但 prediction 目标更关心 future/new data 上的 expected error，这时 minimum MSE 往往比 strict unbiasedness 更重要。

7.3.4 Connection to Data Mining / Model Selection

这和 data mining 里学的 train / validation / test split 是同一条思想线。

training error

-> how well the model fits the data used to estimate theta

validation error

-> how well a chosen model complexity / eta predicts unseen data

test error

-> final estimate of generalisation performance

在 data mining 语言里：

Statistical language	Data mining language
low bias	model flexible enough to capture signal
high bias	underfitting
high variance	overfitting / unstable model

Statistical language	Data mining language
regularisation	complexity control
validation	choose model complexity / hyperparameter
test error	estimate out-of-sample performance

因此 Roy 这里的 tradeoff 可以翻译成:

```

too simple model:
    high bias, low variance, underfit

too flexible model:
    low training error, low bias on training structure, high variance, overfit

regularised / selected model:
    controlled bias, controlled variance, better validation/test error

```

所以如果目标是 scientific parameter estimation, unbiased/BBLUE 很有吸引力; 如果目标是 prediction, minimum MSE 和 validation 通常更关键。

7.3.5 Practical Rule of Thumb

```

If theta itself is the object of interest:
    care strongly about unbiasedness and covariance.

If future prediction is the object of interest:
    care strongly about MSE / validation error.

If Phi is ill-conditioned:
    regularisation may be needed even before prediction quality is considered.

```

Regularisation 的核心不是让 training error 最小, 而是让 expected prediction error 更好。

记忆方式:

```

eta small -> flexible estimate, low bias, high variance
eta large -> shrunk estimate, higher bias, lower variance
best eta  -> balance point for MSE/prediction

```

7.4 Validation

选择 η 时不能只看 fitting data residual。正确思路:

1. 把 data 分成 fitting/training set 和 validation set。
2. 对不同 η 用 fitting set 拟合。
3. 用 validation set 测试 prediction error。
4. 选择 validation error 最小的 η 。

考试可能问：

- 为什么 validation data 不能参与 fitting；
- 为什么 best training fit 不一定 best prediction；
- regularisation 如何连接到 minimum MSE。

8 Convex Sets and Convex Optimisation

这一章开始会出现很多几何 / 拓扑 / 凸分析词汇。下面统一采用：

中文解释 (English term)

这样考试时能认英文，复习时又能先用中文理解。

8.1 Geometry Vocabulary Quick Table

English term	中文	直觉
set	集合	一堆满足某条件的点
point	点	空间中的一个元素 / vector
line segment	线段	两点之间所有 convex combinations
hyperplane	超平面	$\mathbf{a}^T \mathbf{x} = b$ ，把空间切成两边的 flat boundary
halfspace	半空间	$\mathbf{a}^T \mathbf{x} \leq b$ ，超平面一侧的区域
norm ball	范数球	离中心不超过半径 r 的点集
norm cone	范数锥	把 ball 的半径变成变量 t ，即 $\ \mathbf{x}\ \leq t$
polyhedron	多面体	很多个 linear inequalities/ equalities 交出来的区域
polytope	有界多面体 / 多胞体	bounded polyhedron，可以想成封闭的多边形 / 多面体
ellipsoid	椭球	被 matrix 拉伸 / 旋转后的 ball
cone	锥	如果 x 在里面，那么 $\gamma x, \gamma \geq 0$ 也在里面
convex cone	凸锥	同时满足 cone 和 convex set

English term	中文	直觉
PSD cone	正半定锥	所有 positive semidefinite matrices 组成的 cone
affine	仿射	linear plus offset, 例如 $Ax + b$
feasible region	可行域	所有满足 constraints 的 x
boundary	边界	constraint 刚好 active 的边缘
interior	内部	不贴着边界、附近还有空间的点

8.2 Matrix/Vector Form of Geometry (几何体的向量/矩阵解析定义)

Roy 这里的几何语言，其实是在说：

用 **algebraic constraints** (代数约束) 定义 **point sets** (点集)。

在 optimisation 里，一个 geometry object (几何体) 通常不是靠画图定义，而是靠“哪些 $\mathbf{x}$ 满足某些等式 / 不等式”定义：

$$C = \{\mathbf{x} \mid \text{constraints on } \mathbf{x}\}$$

这里用粗体表示 vector (向量)，例如 $\mathbf{x}, \mathbf{a}$ ；普通小写如 b, r 表示 scalar (标量)。

例如二维：

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \in \mathbb{R}^2$$

三维：

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \in \mathbb{R}^3$$

更一般地：

$$\mathbf{x} \in \mathbb{R}^n$$

8.2.1 Hyperplane (超平面)

Hyperplane (超平面) 的标准定义是：

$$\{\mathbf{x} \mid \mathbf{a}^T \mathbf{x} = b\}$$

其中:

- $\mathbf{x}$ 是未知 point / vector (点 / 向量);
- $\mathbf{a}$ 是已知 vector, 叫 normal vector (法向量);
- b 是 scalar offset (标量偏移);
- $\mathbf{a}^T \mathbf{x}$ 是 inner product / dot product (内积 / 点积)。

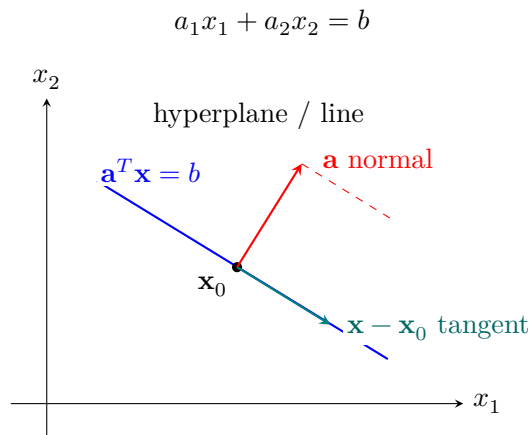
二维时:

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

所以:

$$\mathbf{a}^T \mathbf{x} = a_1 x_1 + a_2 x_2$$

Hyperplane equation (超平面方程) 变成:



在 $\mathbb{R}^2$ 里, 这是一条 line (直线)。在 $\mathbb{R}^3$ 里, 这是一个 plane (平面)。在 $\mathbb{R}^n$ 里, 统一叫 hyperplane (超平面)。

为什么维度会少 1?

在 $\mathbb{R}^n$ 里, 一个点 $\mathbf{x}$ 原本有 n 个自由坐标:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

但 hyperplane 加了一个 independent linear equality (独立线性等式):

$$\mathbf{a}^T \mathbf{x} = b$$

也就是:

$$a_1 x_1 + a_2 x_2 + \cdots + a_n x_n = b$$

这个等式会“用掉”一个自由度。只要 $a_n \neq 0$, 就可以把其中一个变量写成其他变量的函数:

$$x_n = \frac{b - a_1 x_1 - \cdots - a_{n-1} x_{n-1}}{a_n}$$

所以 $x_1, \dots, x_{n-1}$ 可以自由选, 但 x_n 被 equality 决定。自由度从 n 个变成 $n-1$ 个。

例子:

```
R^2:  one equation a1 x1 + a2 x2 = b
      -> 2 variables, 1 constraint
      -> 1-dimensional line

R^3:  one equation a1 x1 + a2 x2 + a3 x3 = b
      -> 3 variables, 1 constraint
      -> 2-dimensional plane

R^n:  one independent linear equality
      -> n variables, 1 constraint
      -> (n-1)-dimensional hyperplane
```

从方向角度看, $\mathbf{a}$ 这个 normal vector (法向量) 规定了一个不能自由移动的方向: 沿着 $\mathbf{a}$ 的方向移动会改变 $\mathbf{a}^T \mathbf{x}$, 所以会离开 $\mathbf{a}^T \mathbf{x} = b$ 。剩下所有与 $\mathbf{a}$ 正交的 directions (方向) 仍然可以自由移动, 因此 hyperplane 里的可移动方向组成一个 $n-1$ dimensional subspace ($n-1$ 维子空间)。

几何直觉:

```
\mathbf{a} tells which direction is perpendicular to the hyperplane.
b tells where the hyperplane is shifted.
```

也就是说, $\mathbf{a}$ 不是平面上的方向, 而是 normal direction (法向方向)。所有满足 $\mathbf{a}^T \mathbf{x} = b$ 的点 $\mathbf{x}$, 都落在同一个 flat boundary (平直边界) 上。

如果 $\mathbf{x}_0$ 是 hyperplane 上某个点, 即:

$$\mathbf{a}^T \mathbf{x}_0 = b$$

那么任何同在 hyperplane 上的点 $\mathbf{x}$ 都满足：

$$\mathbf{a}^T(\mathbf{x} - \mathbf{x}_0) = 0$$

这说明 $\mathbf{x} - \mathbf{x}_0$ 是沿着 hyperplane 走的 direction（切向方向），它和 normal vector $\mathbf{a}$ 正交。

8.2.2 Halfspace（半空间）

把 equality（等式）换成 inequality（不等式），就得到 halfspace（半空间）：

$$\{\mathbf{x} \mid \mathbf{a}^T \mathbf{x} \leq b\}$$

直觉：

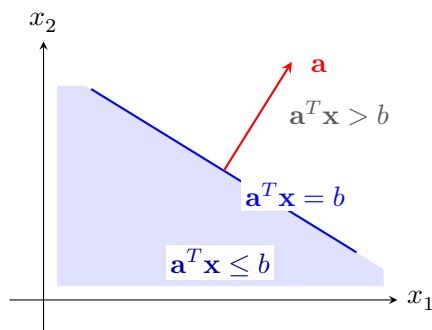
hyperplane boundary: vector $\mathbf{a}$ dot vector $\mathbf{x} = b$

halfspace side: vector $\mathbf{a}$ dot vector $\mathbf{x} \leq b$

另一个方向是：

$$\{\mathbf{x} \mid \mathbf{a}^T \mathbf{x} \geq b\}$$

所以 hyperplane 像一把 knife，把空间切成两个 halfspaces。



判断 feasible side（可行侧）最稳的方法不是凭眼睛猜 left/right/up/down，而是用 test point（测试点）代入不等式。

步骤：

1. 先画 boundary（边界）：把 inequality 改成 equality。
2. 选一个不在 boundary 上、容易算的点，例如 origin（原点） $\mathbf{0}$ 。
3. 把 test point 代入 inequality。
4. 如果满足，不等式可行侧就在 test point 那一侧；如果不满足，可行侧就在另一侧。

例子：

$$x_1 + x_2 \leq 1$$

Boundary 是:

$$x_1 + x_2 = 1$$

选 origin:

$$(0, 0): \quad 0 + 0 \leq 1$$

成立, 所以 feasible side 是包含 origin 的那一侧。

如果是:

$$x_1 + x_2 \geq 1$$

origin 代入后:

$$0 + 0 \geq 1$$

不成立, 所以 feasible side 是不包含 origin 的另一侧。

多个约束时, 对每条线都做同样检查, 然后取所有可行侧的 intersection (交集):

```
one inequality -> one halfspace
many inequalities -> intersection of many halfspaces
feasible region -> region satisfying all tests at once
```

方向词要小心:

```
left/right/up/down depends on how the line is drawn.
test-point substitution works every time.
```

8.2.3 Stacking Many Linear Constraints (把很多线性约束堆成矩阵)

如果有很多 halfspace constraints:

$$\mathbf{a}_1^T \mathbf{x} \leq b_1, \quad \mathbf{a}_2^T \mathbf{x} \leq b_2, \quad \dots, \quad \mathbf{a}_m^T \mathbf{x} \leq b_m$$

可以把每个 $\mathbf{a}_i^T$ 放成矩阵 A 的一行:

$$A = \begin{bmatrix} \mathbf{a}_1^T \\ \mathbf{a}_2^T \\ \vdots \\ \mathbf{a}_m^T \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

于是所有 constraints 可以写成：

$$Ax \leq b$$

这个不等式是 componentwise (逐分量) 的，意思是：

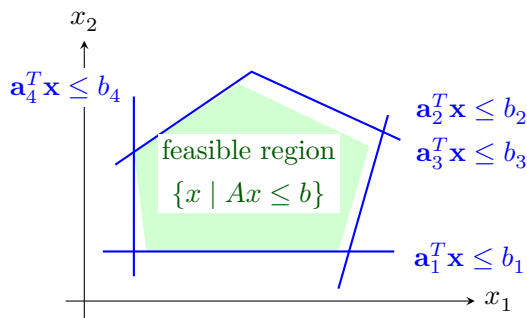
$$(Ax)_i \leq b_i, \quad i = 1, \dots, m$$

所以：

$$\{x \mid Ax \leq b\}$$

就是很多 halfspaces (半空间) 的 intersection (交集)。这就是 polyhedron (多面体) 的基本矩阵定义。

实际画图时，对每一行 $\mathbf{a}_i^T \mathbf{x} \leq b_i$ 都先画 boundary line (边界线) $\mathbf{a}_i^T \mathbf{x} = b_i$ ，再用 test point 判断保留哪一侧。最后剩下的共同区域，就是 feasible region (可行域)。



8.2.4 Two Ways to Describe the Same Polytope (同一个多胞体的两种表示)

Roy 这里讲的是 convex geometry (凸几何) 里非常重要的一对表示法。

第一种是用 halfspaces / inequalities (半空间 / 不等式) 定义：

$$P = \{\mathbf{x} \mid A\mathbf{x} \leq \mathbf{b}\}.$$

这叫 H-representation (halfspace representation, 半空间表示)。每一行

$$\mathbf{a}_i^T \mathbf{x} \leq b_i$$

就是一个 boundary hyperplane 加上它的一侧。很多这样的半空间取交集, 就得到一个 polyhedron / polytope。

第二种是用 vertices (顶点) 定义:

$$P = \text{conv}\{\mathbf{v}_1, \dots, \mathbf{v}_k\}.$$

这叫 V-representation (vertex representation, 顶点表示)。这里的 conv 表示 convex hull (凸包), 意思是所有顶点的 convex combinations (凸组合):

$$\mathbf{x} = \lambda_1 \mathbf{v}_1 + \dots + \lambda_k \mathbf{v}_k, \quad \lambda_i \geq 0, \quad \sum_{i=1}^k \lambda_i = 1.$$

直觉上:

H-representation: keep the side of each wall

用墙 / 边界的内侧定义区域

V-representation: fill in the convex hull of corner points

用角点围起来再填满中间

例如二维 square (正方形) 可以有两种写法。

H-representation:

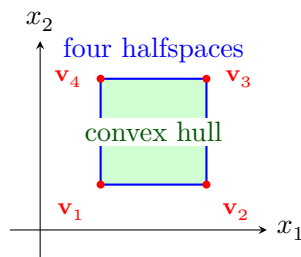
$$0 \leq x_1 \leq 1, \quad 0 \leq x_2 \leq 1.$$

也就是四个 halfspaces 的交集。

V-representation:

$$\text{conv} \left\{ \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right\}.$$

也就是四个 vertices 的 convex hull。



Roy 说 “theoretically they are the same”, 指的是: 对 bounded polytope (有界多胞体) 来说, H-representation 和 V-representation 可以描述同一个集合。一个正方形既可以说成四个半空间的交集, 也可以说成四个顶点的凸包。

但是 computationally (计算上) 它们很不一样。

如果优化问题天然写成 constraints (约束), 比如:

$$\min f(\mathbf{x}) \quad \text{subject to} \quad A\mathbf{x} \leq \mathbf{b},$$

那么 H-representation 很方便, 因为 solver 可以直接检查每个 inequality 是否满足。linear programming (线性规划)、quadratic programming (二次规划)、SOCP、SDP/LMI 都基本沿着这种 “constraints first” 的思路发展。

而 V-representation 看起来几何直观, 但在高维里有一个严重问题: vertices / edges / faces (顶点 / 边 / 面) 的数量可能爆炸。即使 halfspaces 的数量不多, 由它们围出来的 polytope 也可能有非常多 vertices。反过来, 从 vertices 找出所有 supporting halfspaces / facets (支撑半空间 / 面) 也可能非常贵。

所以 Roy 说 “solving things in terms of half-spaces can be easy”, 意思是:

```
constraints as inequalities
-> solver handles them directly
-> no need to enumerate every vertex or edge
```

而 “NP-hard in terms of the edges” 的直觉是:

```
if you describe or solve the geometry by listing vertices/edges/faces,
you may have to deal with a combinatorial object.
The number of edges/faces can grow extremely fast with dimension.
Some associated enumeration/conversion problems become computationally hard.
```

这里不要把 Roy 的话理解成 “顶点表示永远不能用”。在低维画图时, V-representation 非常直观; 但在 optimisation algorithms (优化算法) 里, 特别是高维问题里, 通常更喜欢 H-representation / conic representation, 因为它直接对应 constraints, 而不是先把所有角点和边都找出来。

一句话:

```
H-representation tells the solver what constraints must hold.
V-representation tells the geometry by listing the corners.

They can define the same polytope,
but they are not equally convenient computationally.
```

8.2.5 Why Halfspaces Matter for Classification (为什么半空间对分类有用)

Classification problem (分类问题) 的基本任务是:

```
given features  $\mathbf{x}$ ,
decide which class / label  $\mathbf{x}$  belongs to.
```

如果 feature vector 是:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix},$$

一个 linear classifier (线性分类器) 可以写成:

$$\mathbf{w}^T \mathbf{x} + c = 0.$$

这条 hyperplane / line (超平面 / 直线) 叫 decision boundary (决策边界)。它把 feature space (特征空间) 切成两个 halfspaces:

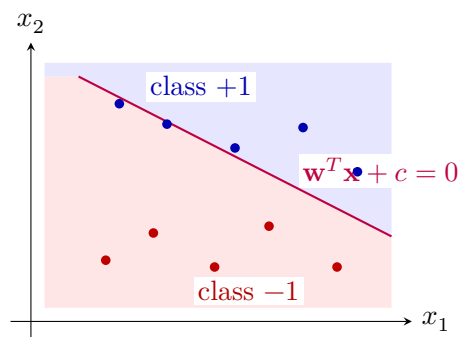
$$\mathbf{w}^T \mathbf{x} + c \geq 0 \quad \text{and} \quad \mathbf{w}^T \mathbf{x} + c < 0.$$

于是分类规则可以写成:

$$\hat{y} = \begin{cases} +1, & \mathbf{w}^T \mathbf{x} + c \geq 0, \\ -1, & \mathbf{w}^T \mathbf{x} + c < 0. \end{cases}$$

也就是说:

```
one hyperplane
-> two halfspaces
-> two predicted classes
```



多个 halfspaces 也有用。比如一个 class 不是“某条线的一侧”，而是要同时满足多个 linear tests:

$$A\mathbf{x} \leq \mathbf{b}.$$

这时 class region (类别区域) 就是一个 polytope / polyhedron (多胞体 / 多面体):

```
inside all selected halfspaces -> class A
outside at least one          -> not class A
```

这解释了 Roy 说 halfspace intersections 对 classification 有用：

- 每条 hyperplane 是一个 decision boundary；
- 每个 halfspace 是 boundary 的一侧；
- 多个 halfspaces 的 intersection 可以定义一个 more structured decision region；
- 线性模型、perceptron、logistic regression、linear SVM 都可以从这个几何视角理解。

注意：classification 里的 Φ / feature engineering 也很重要。如果原始 feature space 中两类不是线性可分，可以通过 nonlinear features / kernels 把数据映射到新的 feature space，再用 hyperplane 分开。

8.2.6 Equality Constraints and Affine Sets (等式约束与仿射集合)

类似地：

$$Ax = b$$

表示很多 hyperplanes (超平面) 同时成立。它定义的是 affine set (仿射集合)。

如果 $b = 0$ ：

$$Ax = 0$$

这个集合经过 origin (原点)，通常是 subspace (子空间)。

如果 $b \neq 0$ ：

$$Ax = b$$

这个集合通常不经过 origin，是 shifted subspace (平移后的子空间)，也就是 affine set (仿射集合)。

8.2.7 Solving Equality Constraints by Null Space (用零空间表示等式约束的所有解)

Roy 这里讲的是：如果 optimisation problem 里有 equality constraint (等式约束)

$$Ax = \mathbf{b},$$

我们可以先把所有满足这个等式的 $\mathbf{x}$ 写出来，再把它 substitute back (代回) 原问题。这样就能 eliminate equality constraints (消去等式约束)。

先复习矩阵乘法的含义。假设：

$$A = \begin{bmatrix} \mathbf{a}_1^T \\ \mathbf{a}_2^T \\ \vdots \\ \mathbf{a}_m^T \end{bmatrix}, \quad \mathbf{x} \in \mathbb{R}^n, \quad \mathbf{b} \in \mathbb{R}^m.$$

那么：

$$A\mathbf{x} = \mathbf{b}$$

其实就是 m 个 linear equations (线性方程)：

$$\mathbf{a}_1^T \mathbf{x} = b_1, \quad \mathbf{a}_2^T \mathbf{x} = b_2, \quad \dots, \quad \mathbf{a}_m^T \mathbf{x} = b_m.$$

每一行 $\mathbf{a}_i^T \mathbf{x} = b_i$ 是一个 hyperplane。所有行同时满足，就是这些 hyperplanes 的 intersection (交集)。

8.2.7.1 Null Space (零空间)

Null space (零空间) 定义为：

$$\mathcal{N}(A) = \{\mathbf{v} \mid A\mathbf{v} = \mathbf{0}\}.$$

读法：

```

null space of A
= all directions v that A maps to zero
= all movements that do not change Ax

```

为什么它重要？

如果 $\mathbf{x}_0$ 是 $A\mathbf{x} = \mathbf{b}$ 的一个 particular solution (特解)，也就是：

$$A\mathbf{x}_0 = \mathbf{b},$$

那么任何 null-space direction (零空间方向) $\mathbf{v}$ 都可以加到 $\mathbf{x}_0$ 上：

$$\mathbf{x} = \mathbf{x}_0 + \mathbf{v}, \quad \mathbf{v} \in \mathcal{N}(A).$$

因为：

$$A(\mathbf{x}_0 + \mathbf{v}) = A\mathbf{x}_0 + A\mathbf{v} = \mathbf{b} + 0 = \mathbf{b}.$$

所以:

```
particular solution x0:
  one point satisfying Ax = b

null space directions:
  directions you can move without violating Ax = b

all solutions:
  x0 + any null-space direction
```

这就是 Roy 说的 “particular solution $\mathbf{x}_0$ and then $F\mathbf{z}$ defines all of the other ones”。

8.2.7.2 Matrix Form: $\mathbf{x} = \mathbf{x}_0 + F\mathbf{z}$

实际计算时，我们不会一个一个列出所有 $\mathbf{v}$ 。我们找一个 matrix F ，它的 columns（列）是一组 basis vectors（基向量），span（张成）整个 null space:

$$\mathcal{N}(A) = \{F\mathbf{z} \mid \mathbf{z} \in \mathbb{R}^k\}.$$

这里:

- $F \in \mathbb{R}^{n \times k}$;
- $\mathbf{z} \in \mathbb{R}^k$ 是新的 free variable（自由变量）;
- k 是 degrees of freedom（自由度）;
- $AF = 0$ ，因为 F 的每一列都在 null space 里。

于是所有满足 $A\mathbf{x} = \mathbf{b}$ 的解都可以写成:

$$\mathbf{x} = \mathbf{x}_0 + F\mathbf{z}.$$

代回原 constraint:

$$A\mathbf{x} = A(\mathbf{x}_0 + F\mathbf{z}) = A\mathbf{x}_0 + AF\mathbf{z}.$$

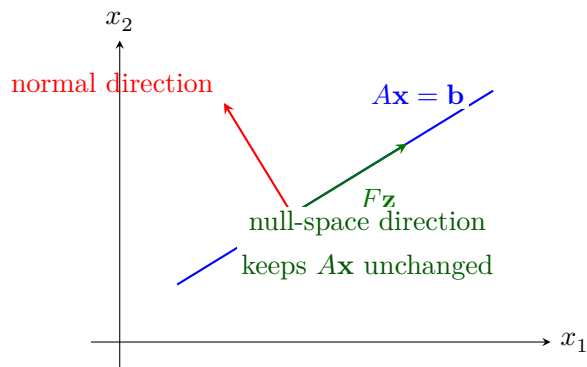
要让它等于 $\mathbf{b}$ ，只需要:

$$A\mathbf{x}_0 = \mathbf{b}, \quad AF = 0.$$

这正是 Roy 写的：

$$AF\mathbf{z} + A\mathbf{x}_0 = \mathbf{b}.$$

由于 $AF = 0$ 且 $A\mathbf{x}_0 = \mathbf{b}$ ，所以无论 $\mathbf{z}$ 怎么选， $\mathbf{x} = \mathbf{x}_0 + F\mathbf{z}$ 都满足 equality constraint。



二维图里， $A\mathbf{x} = b$ 是一条 line。 $\mathbf{x}_0$ 是这条线上一个点。沿着线本身移动，就是 null-space direction；沿着 normal direction 移动，会离开 constraint。

8.2.7.3 Concrete Example (具体例子)

考虑：

$$A = \begin{bmatrix} 1 & 1 \end{bmatrix}, \quad b = 3.$$

constraint 是：

$$x_1 + x_2 = 3.$$

一个 particular solution 是：

$$\mathbf{x}_0 = \begin{bmatrix} 3 \\ 0 \end{bmatrix}, \quad A\mathbf{x}_0 = 3.$$

null space 满足：

$$A\mathbf{v} = 0 \iff v_1 + v_2 = 0.$$

所以一个 null-space basis vector 是：

$$\mathbf{f} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad A\mathbf{f} = 1 - 1 = 0.$$

于是：

$$F = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad z \in \mathbb{R}.$$

所有解可以写成：

$$\mathbf{x} = \mathbf{x}_0 + Fz = \begin{bmatrix} 3 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \end{bmatrix} z = \begin{bmatrix} 3 + z \\ -z \end{bmatrix}.$$

检查：

$$x_1 + x_2 = (3 + z) + (-z) = 3.$$

无论 z 是多少，constraint 都满足。 z 就是剩下的 degree of freedom（自由度）。

8.2.7.4 Why This Helps Optimisation（为什么这对优化有用）

如果原问题是：

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{subject to} \quad A\mathbf{x} = \mathbf{b},$$

我们可以令：

$$\mathbf{x} = \mathbf{x}_0 + F\mathbf{z}.$$

然后原问题变成：

$$\min_{\mathbf{z}} f(\mathbf{x}_0 + F\mathbf{z}).$$

这样 equality constraint 已经自动满足，不需要再写。

直觉：

```
Instead of optimizing over all  $\mathbf{x}$ ,
optimize only over the free directions  $\mathbf{z}$ 
that stay inside  $A\mathbf{x} = \mathbf{b}$ .
```

如果 null space 维度 $k = 0$ ，说明没有 degrees of freedom。那就只有一个 feasible point $\mathbf{x}_0$ ：

```

if k = 0:
    x0 is the only solution
    no optimisation left

if k > 0:
    there are free directions Fz
    still need to optimize over z

```

这就是 Roy 说的:

```

If you don't have any degrees of freedom,
x0 is the answer and you're done.

```

```

But in general,
you still have to do some optimization.

```

一句话总结:

```

Ax = b defines an affine set.
x0 gives one feasible point.
Null(A) gives all directions that keep feasibility.
All solutions are  $x = x_0 + Fz$ .

```

8.2.8 Norm Ball (范数球)

Norm ball (范数球) 定义为:

$$\{\mathbf{x} \mid \|\mathbf{x} - \mathbf{x}_c\|_2 \leq r\}$$

其中:

- $\mathbf{x}_c$ 是 center (中心);
- r 是 radius (半径);
- $\|\mathbf{x} - \mathbf{x}_c\|_2$ 是 $\mathbf{x}$ 到中心的 Euclidean distance (欧氏距离)。

二维里是 disk (圆盘), 三维里是 ball (球), 高维里仍叫 norm ball。

8.2.9 Norm Cone (范数锥)

Roy 接着说的 norm cone (范数锥) 可以看成:

```

norm ball + an extra scalar variable t

```

Norm ball 里 radius (半径) r 是固定常数:

$$\|\mathbf{x}\|_2 \leq r$$

Norm cone 里, 右边不再是固定半径, 而是一个 variable (变量) t :

$$\|\mathbf{x}\|_2 \leq t$$

所以 norm cone 的集合定义是:

$$\{(t, \mathbf{x}) \mid \|\mathbf{x}\|_2 \leq t\}$$

这里:

- $\mathbf{x} \in \mathbb{R}^n$ 是 vector (向量);
- $t \in \mathbb{R}$ 是 scalar (标量);
- 因为 norm 总是非负, 所以这个 constraint 自动要求 $t \geq 0$ 。

直觉:

```
fixed radius r:
    one ball

variable radius t:
    as t grows, the allowed ball grows
    stacking all these balls over t creates a cone
```

在一维 slice (切片) 里, 如果 $\mathbf{x}$ 只有一个 scalar component x , 那么:

$$\|\mathbf{x}\|_2 \leq t \text{ becomes } |x| \leq t$$

也就是:

$$-t \leq x \leq t$$

这里也用 test point / inequality logic 判断区域:

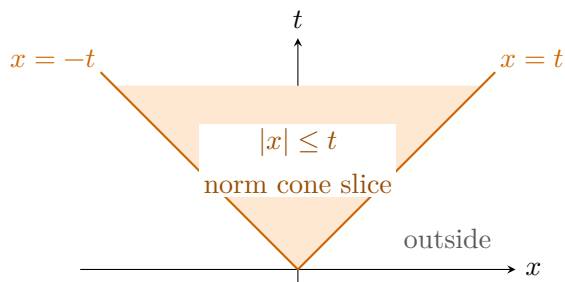
- $x \leq t$ 等价于 $x - t \leq 0$, boundary 是 $x = t$;
- $-t \leq x$ 等价于 $x \geq -t$, boundary 是 $x = -t$;
- norm cone 还隐含 $t \geq 0$, 因为 $|x| \geq 0$ 。

所以 feasible region 必须同时满足:

$$x \leq t, \quad x \geq -t, \quad t \geq 0.$$

这就是两条斜线之间、并且在 t 轴非负方向上的区域。

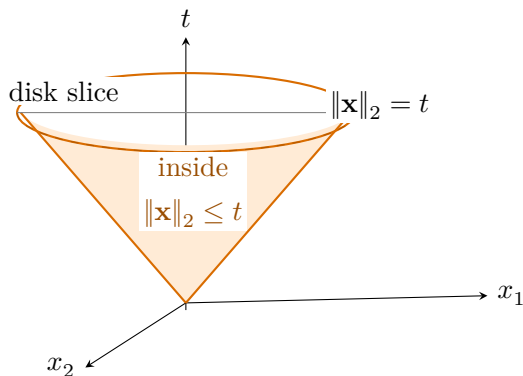
画在 (x, t) 平面里就是一个 cone-shaped region (锥形区域):



Lecture 8 讲义里的 second-order cone 图，就是把 $\mathbf{x} = (x_1, x_2)$ 时的:

$$\sqrt{x_1^2 + x_2^2} \leq t$$

画成三维区域。每一个固定高度 t 的 horizontal slice (水平切片) 都是一个 radius 为 t 的 disk; 这些 disk 从 origin 开始往上叠, 就形成 cone:



为什么叫 cone?

如果 $(t, \mathbf{x})$ 满足 $\|\mathbf{x}\|_2 \leq t$, 那么对任意 $\gamma \geq 0$:

$$\|\gamma \mathbf{x}\|_2 = \gamma \|\mathbf{x}\|_2 \leq \gamma t$$

所以:

$$(\gamma t, \gamma \mathbf{x})$$

仍然在集合里。这正是 cone (锥) 的 scaling property (缩放性质)。

同时它也是 convex cone (凸锥), 因为 norm 是 convex 的, 而且任意两个满足 $\|\mathbf{x}\| \leq t$ 的点做 nonnegative combination 后仍满足同类不等式。

Norm cone 也叫 second-order cone (二阶锥) 或 Lorentz cone (洛伦兹锥):

$$\mathcal{Q}^{n+1} = \{(t, \mathbf{x}) \in \mathbb{R} \times \mathbb{R}^n \mid \|\mathbf{x}\|_2 \leq t\}.$$

这就是 SOCP (second-order cone program, 二阶锥规划) 名字里的 cone。

8.2.10 Ellipsoid (椭球)

Ellipsoid (椭球) 定义为:

$$\{x \mid (x - x_c)^T P^{-1} (x - x_c) \leq 1\}$$

其中 $P \succ 0$ 是 positive definite matrix (正定矩阵)。

直觉:

```
norm ball:
    all directions use the same scale

ellipsoid:
    different directions have different scales
```

如果 $P = I$, 就回到单位球:

$$\|x - x_c\|_2^2 \leq 1$$

如果 P 的某些方向 eigenvalue (特征值) 大, ellipsoid 在那些方向更长; eigenvalue 小, ellipsoid 在那些方向更短。

8.2.11 Ellipsoidal Algorithm (椭球算法)

Roy 截图里的 ellipsoidal algorithm (椭球算法) 是在讲一种 optimisation strategy (优化策略):

```
Keep an ellipsoid that is guaranteed to contain the optimum.
At each step, cut away half that cannot contain the optimum.
Wrap the remaining half with a smaller ellipsoid.
Repeat.
```

它不是 gradient descent。Gradient descent 是:

```
current point -> take a step -> next point
```

Ellipsoid method 更像:

current uncertainty region -> cut it -> smaller uncertainty region

所以它关心的是:

Where can the optimum still be?

而不是:

Which direction should I step right now?

8.2.11.1 Reparametrised Form (重参数化形式)

前面 Roy 已经讲过 equality constraint 的处理:

$$\mathbf{x} = \mathbf{x}_0 + F\mathbf{z}.$$

这里 $\mathbf{x}_0$ 是一个 particular feasible point (特解), $F\mathbf{z}$ 是 null-space directions (零空间方向)。这样 equality constraints 已经自动满足。

所以原来的 problem 可以写成 $\mathbf{z}$ 空间里的问题:

$$\min_{\mathbf{z}} \phi(\mathbf{z}) \quad \text{subject to} \quad \psi_i(\mathbf{z}) \leq 0, \quad i = 1, \dots, m,$$

其中:

$$\phi(\mathbf{z}) = f_0(F\mathbf{z} + \mathbf{x}_0), \quad \psi_i(\mathbf{z}) = f_i(F\mathbf{z} + \mathbf{x}_0).$$

意思是:

$\mathbf{z}$ is the free variable after equality constraints are removed.

$\mathbf{x} = F\mathbf{z} + \mathbf{x}_0$ maps $\mathbf{z}$ back to the original variable $\mathbf{x}$.

$\phi(\mathbf{z})$ is the objective expressed in $\mathbf{z}$.

$\psi_i(\mathbf{z})$ are the inequality constraints expressed in $\mathbf{z}$.

所以椭球算法是在 $\mathbf{z}$ 空间里工作: 它维护一个 ellipsoid, 认为 optimum $\mathbf{z}^*$ 在里面。

8.2.11.2 Step 1: Start With a Big Ellipsoid (先用大椭球包住最优点)

先假设我们知道一个很大的 ellipsoid:

$$E_0 = \{\mathbf{z} \mid (\mathbf{z} - \mathbf{c}_0)^T P_0^{-1} (\mathbf{z} - \mathbf{c}_0) \leq 1\}$$

并且确信:

$$\mathbf{z}^* \in E_0.$$

这里：

- $\mathbf{c}_0$ 是 ellipsoid center (椭球中心);
- P_0 决定 ellipsoid 的 shape / size (形状 / 大小);
- $\mathbf{z}^*$ 是最优解。

直觉：

I do not know where the optimum is,
but I know it is somewhere inside this big ellipsoid.

8.2.11.3 Step 2: Evaluate at the Center (在中心点计算函数与约束)

在当前椭球中心 $\mathbf{c}$ 处，计算：

$$\phi(\mathbf{c}), \quad \psi_i(\mathbf{c}).$$

如果暂时只看 unconstrained problem (无约束问题)，就只看 objective ϕ 。

如果有 constraints，则还要看有没有 violated constraint (被违反的约束)：

$$\psi_i(\mathbf{c}) > 0.$$

Roy 这里先说 “focus on just the cost function”，也就是先忽略 constraints，只用 objective 的 gradient / subgradient 来切椭球。

8.2.11.4 Gradient and Subgradient (梯度与次梯度)

如果 ϕ differentiable (可微)，我们用 gradient：

$$\nabla \phi(\mathbf{c}).$$

Gradient 指向 function 增加最快的方向。对 convex function，有一条重要 inequality：

$$\phi(\mathbf{z}) \geq \phi(\mathbf{c}) + \nabla \phi(\mathbf{c})^T (\mathbf{z} - \mathbf{c}).$$

这条式子的意思是：在 $\mathbf{c}$ 点的 tangent plane (切平面) 永远在 convex function 的 graph 下面。

如果函数不可微，比如有 kink (尖点)：

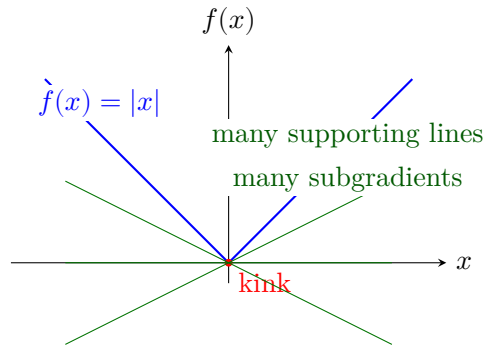
$$f(x) = |x|$$

在 $x = 0$ 没有唯一 gradient。这时可以用 subgradient（次梯度）。

Subgradient g 的定义是：

$$f(y) \geq f(x) + g^T(y - x) \quad \text{for all } y.$$

也就是说， g 定义了一条仍然在函数下面的 supporting line / supporting hyperplane（支撑线 / 支撑超平面）。



所以 Roy 说：

replace the gradient with a set of gradients.

Those are called sub-gradients.

8.2.11.5 Step 3: Use Gradient/Subgradient to Cut the Ellipsoid（用梯度切掉一半）

设当前中心是 $\mathbf{c}$ ，取一个 gradient 或 subgradient：

$$\mathbf{g} \in \partial\phi(\mathbf{c}).$$

对 convex function，如果 $\mathbf{g}^T(\mathbf{z} - \mathbf{c}) > 0$ ，那么由 subgradient inequality：

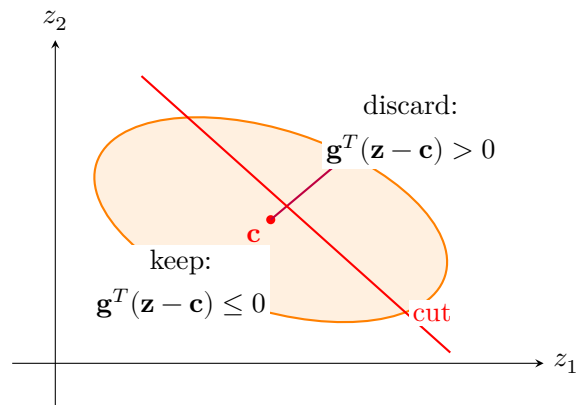
$$\phi(\mathbf{z}) \geq \phi(\mathbf{c}) + \mathbf{g}^T(\mathbf{z} - \mathbf{c}) > \phi(\mathbf{c}).$$

这说明这些点的 objective value 比中心点还大。既然中心点 $\mathbf{c}$ 已经在椭球里，最优点不可能在这个 “higher side”（更高的一侧）。所以 minimizer 必须在另一侧：

$$\mathbf{g}^T(\mathbf{z} - \mathbf{c}) \leq 0.$$

这就是一个 halfspace cut（半空间切割）。

gradient/subgradient gives a separating hyperplane through the center.
 one side cannot contain the minimizer.
 keep the other side.



这张图对应 Roy 的话：

if the gradient points this way,
 cut the ellipsoid in two,
 then the minimizer has to lie in this one.

8.2.11.6 Step 4: Wrap the Remaining Half With a Smaller Ellipsoid (用更小椭球包住剩下的一半)

切完之后，剩下的集合是：

$$E \cap \{\mathbf{z} \mid \mathbf{g}^T(\mathbf{z} - \mathbf{c}) \leq 0\}.$$

它不是一个 ellipsoid，而是 half-ellipsoid（半个椭球）。为了继续用同样算法，我们找一个 new ellipsoid（新椭球）把它包住：

```
old ellipsoid
-> cut by halfspace
-> half ellipsoid
-> smallest-volume enclosing ellipsoid
-> new smaller ellipsoid
```

这个 “smallest volume ellipse enclosing the half ellipse” 就是 Roy 截图第 3 步。

8.2.11.7 A Concrete 2D Cut Example (二维数值例子：椭圆怎么切半空间)

先不要看复杂公式。用最简单的二维例子：

$$E_k = \{\mathbf{z} \mid \mathbf{z}^T \mathbf{z} \leq 1\}.$$

这就是 unit disk (单位圆), 也可以看成一个特殊的 ellipse:

$$\mathbf{c}_k = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad P_k = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

假设在中心点算出来的 gradient 是:

$$\mathbf{g}_k = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

这表示 objective 沿着 positive z_1 direction (向右) 增加最快。对 convex objective 来说, 右半边的点满足:

$$\mathbf{g}_k^T (\mathbf{z} - \mathbf{c}_k) > 0.$$

代入数字:

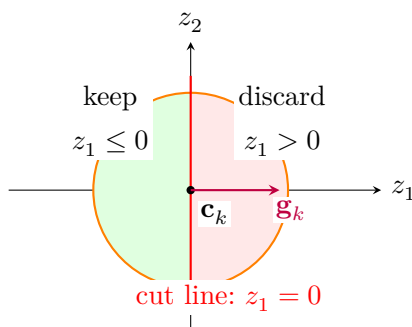
$$\begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} > 0 \iff z_1 > 0.$$

也就是说, 右半边 $z_1 > 0$ 的 objective 一定比中心点更高。既然中心点已经是一个 candidate, minimizer 不需要去右半边找。我们保留:

$$\mathbf{g}_k^T (\mathbf{z} - \mathbf{c}_k) \leq 0 \iff z_1 \leq 0.$$

所以这一步的 halfspace cut 很具体:

```
old region: unit disk
discard:    right half,  $z_1 > 0$ 
keep:      left half,  $z_1 \leq 0$ 
```



这个例子中的 “line” 是 $z_1 = 0$ 。在更高维里，它就是 hyperplane：

$$\mathbf{g}_k^T(\mathbf{z} - \mathbf{c}_k) = 0.$$

而保留下来的那一侧是 halfspace：

$$\mathbf{g}_k^T(\mathbf{z} - \mathbf{c}_k) \leq 0.$$

注意：保留下来的 half-disk 不是 ellipse。所以算法下一步要找一个 new ellipse 把它包起来。

8.2.11.8 Roy's Ellipse Update Formula (Roy 的椭圆更新公式)

Roy 截图里的 ellipsoid 通常写成：

$$E_k = \{\mathbf{z} \mid (\mathbf{z} - \mathbf{v}_k)^T P_k^{-1} (\mathbf{z} - \mathbf{v}_k) \leq 1\}.$$

这里：

- $\mathbf{v}_k$ is the centre (中心)；
- P_k is the shape matrix (形状矩阵)；
- P_k 的 eigenvectors 决定椭圆朝向；
- P_k 的 eigenvalues 决定各方向的半轴长度平方。

Roy 的更新分三步。

1. Normalize the gradient with respect to P_k (按椭圆几何归一化梯度)

$$\tilde{\mathbf{g}} = \frac{\mathbf{g}_k}{\sqrt{\mathbf{g}_k^T P_k \mathbf{g}_k}}.$$

这一步不是普通 Euclidean normalization (不是简单除以 $\|\mathbf{g}\|_2$)。它是在当前 ellipse 的 geometry 下 normalize。

为什么要这样做？因为同一个 gradient direction，在一个长扁椭圆里，沿不同轴方向切，实际切掉的几何尺度不同。 P_k 告诉我们当前椭圆在各方向有多宽。

所以 Roy 说：

you normalize the gradient with respect to P,
because you only care about its direction.

2. Move the centre (移动中心)

$$\mathbf{v}_{k+1} = \mathbf{v}_k - \frac{P_k \tilde{\mathbf{g}}}{n+1}.$$

这里 n 是 dimension (维度)。如果是二维平面, $n = 2$, 所以 denominator 是 3。

直觉:

gradient points toward the discarded side.
new centre moves away from that side.

也就是说, 如果 gradient 指向右边, 而右边被切掉, 新椭圆中心就会往左移。

3. Update the shape matrix (更新椭球形状)

$$P_{k+1} = \frac{n^2}{n^2 - 1} \left(P_k - \frac{2}{n + 1} P_k \tilde{\mathbf{g}} \tilde{\mathbf{g}}^T P_k \right).$$

这条式子看起来复杂, 但含义可以拆成两层:

P_k
old ellipse shape

$P_k \tilde{\mathbf{g}} \tilde{\mathbf{g}}^T P_k$
rank-1 correction in the cut direction

$n^2/(n^2-1)$
rescaling so the new ellipsoid still encloses the kept half

为什么 Roy 说它是 rank 1 update?

因为:

$$\tilde{\mathbf{g}} \tilde{\mathbf{g}}^T$$

是一个 outer product (外积)。一个 vector 乘它自己的 transpose, 会得到一个 rank-1 matrix。它只主要改变一个方向: cut direction (切割方向)。

8.2.11.9 Numerical Update Example (把二维数字代进去算一次)

继续用上面的例子:

$$n = 2, \quad \mathbf{v}_k = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad P_k = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{g}_k = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Step 1:

$$\sqrt{\mathbf{g}_k^T P_k \mathbf{g}_k} = \sqrt{\begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix}} = 1.$$

所以：

$$\tilde{\mathbf{g}} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Step 2:

$$\mathbf{v}_{k+1} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - \frac{1}{3} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -1/3 \\ 0 \end{bmatrix}.$$

新中心移动到了左边，因为我们保留的是 $z_1 \leq 0$ 。

Step 3:

$$\tilde{\mathbf{g}}\tilde{\mathbf{g}}^T = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

因为 $P_k = I$:

$$P_k \tilde{\mathbf{g}}\tilde{\mathbf{g}}^T P_k = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

代入：

$$P_{k+1} = \frac{4}{3} \left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - \frac{2}{3} \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \right).$$

括号里是：

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - \begin{bmatrix} 2/3 & 0 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 1/3 & 0 \\ 0 & 1 \end{bmatrix}.$$

所以：

$$P_{k+1} = \begin{bmatrix} 4/9 & 0 \\ 0 & 4/3 \end{bmatrix}.$$

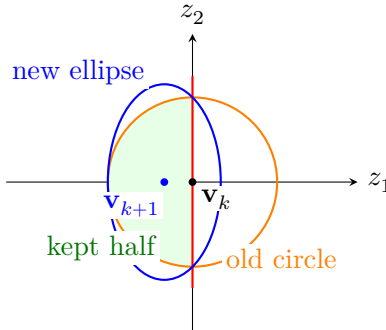
这说明新椭圆是：

$$\frac{(z_1 + 1/3)^2}{4/9} + \frac{z_2^2}{4/3} \leq 1.$$

读这个式子：

- centre is $(-1/3, 0)$;
- horizontal semi-axis 是 $\sqrt{4/9} = 2/3$;
- vertical semi-axis 是 $\sqrt{4/3} \approx 1.155$ 。

所以新椭圆向左移动，并且变得更竖、更窄。它刚好包住左半圆。



这个例子就是 Roy 公式的实际含义：

cut line says which side to keep.

centre update moves toward the kept side.

shape update stretches/shrinks the ellipse to cover the kept half.

8.2.11.10 Volume Reduction (体积缩小保证)

Roy 说：

$$\text{vol}(E_{k+1}) < e^{-\frac{1}{2n}} \text{vol}(E_k).$$

这句话不是说每次都切掉 exactly 50%。它说的是：更新后的 enclosing ellipsoid 的 volume 有一个 guaranteed shrink factor (保证缩小因子)。

当 $n = 2$ ：

$$e^{-1/(2n)} = e^{-1/4} \approx 0.779.$$

所以：

$$\text{vol}(E_{k+1}) < 0.779 \text{vol}(E_k).$$

也就是 new ellipse 的面积最多约为旧面积的 77.9%，大约减少 22.1%。

Roy 口头说 “volume drops by 77%” 容易让人误解。更准确地读应该是：

the remaining volume is about 77% of the old volume for $n = 2$

不是减少 77%，而是变成约 77%。重点是：每一步都有数学保证，不会只是在经验上变小。

算法反复做：

1. Evaluate objective/constraints at ellipsoid center.
2. Use gradient/subgradient or violated constraint to define a cut.
3. Keep the half that must contain the optimum.
4. Enclose that half with a smaller ellipsoid.
5. Repeat.

关键性质：

the volume of each ellipsoid shrinks by at least a specified factor.

也就是说，每一步都能保证 uncertainty region（不确定区域）变小。只要一直切下去，包含最优点的 ellipsoid 会越来越小。

8.2.11.11 What If There Are Constraints? (如果有约束怎么办)

Roy 截图里也有 constraints:

$$\psi_i(\mathbf{z}) \leq 0.$$

如果当前中心 $\mathbf{c}$ violate（违反）某个 constraint:

$$\psi_i(\mathbf{c}) > 0,$$

那么用这个 constraint 的 subgradient 也可以做 cut。

对于 convex constraint function ψ_i ，如果：

$$\psi_i(\mathbf{c}) > 0,$$

则当前中心不可行。最优可行点必须在让 ψ_i 变小的一侧。于是可以用：

$$\mathbf{g}_i^T(\mathbf{z} - \mathbf{c}) \leq 0, \quad \mathbf{g}_i \in \partial\psi_i(\mathbf{c})$$

来切掉不可能含有 feasible optimum（可行最优点）的一半。

所以 ellipsoid method 可以同时处理：

objective cut:
center is feasible, use objective subgradient

```
constraint cut:
    center violates a constraint, use constraint subgradient
```

8.2.11.12 80/20 Intuition (80/20 直觉)

Ellipsoidal algorithm 的核心不是要你手算更新公式，而是理解：

```
Convexity gives valid cuts.
Ellipsoids keep a compact uncertainty region.
Each cut removes a region that cannot contain the optimum.
Each enclosing ellipsoid is smaller.
Repeated cuts converge toward the global optimum.
```

和前面 quasi-convex bisection 的思想类似：

```
Do not only ask:
    what is the gradient step?

Ask:
    what region can still contain the optimum?
```

一句话：

```
Ellipsoid method is geometry-driven optimisation:
use gradients/subgradients as certificates to cut away impossible regions.
```

8.2.11.13 Historical Status and Exam Perspective (历史地位与考试角度)

Ellipsoid method 不是最近的新算法。它是 convex optimisation 里很经典的 old algorithm (老算法)，主要价值偏 theory / foundations (理论基础)，不是现代工程里最常用的高效 solver。

它出名的历史原因是：

```
ellipsoid method was used to prove that linear programming is polynomial-time solvable.
```

更精确地说，通常把这个里程碑归到 Khachiyan 1979 对 linear programming (线性规划) 多项式时间可解的证明。Roy 口头提到 1984，可能是在简化这段历史；1984 更常和 Karmarkar's interior-point method (内点法) 联系在一起。对本课来说，不需要背优化史细节，但要知道：

```
ellipsoid method is historically important because it gave a polynomial-time guarantee.
```

8.2.11.14 What Does Polynomial Time Mean? (什么是多项式时间)

Polynomial time (多项式时间) 是计算复杂性里的概念。它不是说程序一定很快，而是说：

as the problem size grows,
the worst-case running time is bounded by a polynomial function of the input size.

比如 input size 记作 N 。如果一个算法最坏情况下需要:

$$O(N), \quad O(N^2), \quad O(N^3), \quad O(N^{10})$$

这些都叫 polynomial time。因为运行时间像 N 的某个固定次方增长。

相反, 如果最坏情况是:

$$O(2^N), \quad O(3^N), \quad O(N!)$$

这些通常叫 exponential / factorial time。它们随着 N 变大增长得非常快。

一个直观比较:

Problem size N	N^3	2^N
10	1,000	1,024
20	8,000	1,048,576
50	125,000	1.13×10^{15}

所以 polynomial time 的意义是:

the algorithm scales in a controlled way in the worst case.

但注意两点:

1. **Polynomial time does not mean fast in practice.**

$O(N^{10})$ 也是 polynomial time, 但可能完全不好用。

2. **Not polynomial-time proof does not mean useless.**

Simplex method (单纯形法) 在最坏情况下不是 polynomial-time guarantee, 但工程上长期非常好用。

所以 Roy 说 ellipsoid method 重要, 是指它给了 LP 一个 theoretical guarantee (理论保证):

linear programming can be solved in polynomial time.

这在理论上很重大, 因为它说明 LP 不是那种最坏情况会爆炸到不可控的 optimization class。

但这不等于 ellipsoid method 是实际求 LP 时最快、最常用的方法。

现代工程中, 真正常用来解 LP / QP / SOCP / SDP / LMI 的通常是:

simplex method
interior-point methods

active-set methods
first-order / proximal methods
ADMM
commercial solvers: Gurobi, MOSEK, CPLEX
modeling tools: CVX, CVXPY, YALMIP

所以 Roy 讲 ellipsoid method，大概率不是让你以后手写它作为主求解器，而是为了建立这几个思想：

Roy 想传达的思想	中文理解
convexity gives valid cuts	凸性让我们能安全切掉不可能含有最优点的区域
gradient/subgradient as certificate	梯度/次梯度不只是 descent direction，也能作为排除区域的证据
optimisation as shrinking uncertainty region	优化可以理解为不断缩小“最优点可能在哪里”的区域
cutting-plane idea	很多算法不是沿梯度走，而是用 halfspace 切 feasible/optimal region
polynomial-time guarantee	算法复杂度也可以被严格证明

Will this be an exam question? (是否可能作为考题?)

参考 Roy 给的 Convex_Opt_exercises.pdf，exercise 说明里已经说这些题：

generally relevant, but harder than the ENEL445 examination

而 convex exercise 里和 ellipsoid 相关的题主要是：

minimum-volume ellipsoid containing another ellipsoid and finite points

它要求用 SDP / LMI / S-procedure formulation 来表达，不是要求完整手推 ellipsoid method 的迭代公式。

所以我的考试风险判断是：

内容	考试可能性	备注
解释 ellipsoid / halfspace / gradient cut 的几何直觉	High	Roy 课堂讲得很具体，适合短答题
给出 ellipsoid method 的 4-step summary	Medium-High	可能问 “describe the idea”
判断 minimizer 在 cut 的哪一侧	Medium	可用 subgradient inequality 解释
记住完整 P_{k+1} 更新公式并手算	Low-Medium	exercise 难度高于考试；更像 coding / theory extension
证明 LP polynomial time	Low	应知道历史意义，不太可能要求证明

内容	考试可能性	备注
minimum-volume ellipsoid as SDP/LMI	Medium for advanced exercise, lower for exam	可能作为理解 SDP/LMI 建模 的延伸

考试版最小记忆：

Ellipsoid method keeps an ellipsoid containing the optimum.
 At the centre, evaluate objective/constraints.
 A gradient, subgradient, or violated constraint gives a cutting hyperplane.
 Discard the side that cannot contain the optimum.
 Wrap the remaining region in a smaller ellipsoid.
 Repeat until the uncertainty region is small enough.

对 ENEL445 来说，最重要的不是背公式，而是能说清楚：

Convexity turns local gradient/subgradient information into global geometric exclusion.

8.2.12 Why This Matters for Optimisation (为什么这对优化重要)

Optimisation problem 里的 constraint (约束) 本质上就是在定义 feasible region (可行域)：

$$\text{feasible region} = \{x \mid \text{all constraints are satisfied}\}$$

所以 Roy 讲 hyperplane、halfspace、cone、ellipsoid，不是为了抽象几何本身，而是为了让你能看懂：

linear constraints -> intersections of halfspaces
 norm constraints -> norm cones / second-order cones
 quadratic constraints -> balls / ellipsoids
 PSD constraints -> matrix-valued convex geometry
 LMI constraints -> affine matrix expression inside PSD cone

一句话记忆：

Analytic geometry in optimisation:
 describe shapes by equations and inequalities in x .

8.3 Convex Set (凸集)

集合 C 是 convex set (凸集)，如果：

$$x_1, x_2 \in C, \quad \gamma \in [0, 1] \quad \Rightarrow \quad \gamma x_1 + (1 - \gamma)x_2 \in C$$

直觉：任意两点之间的 line segment（线段）都在集合里。

重要例子：

- hyperplane（超平面）： $\{\mathbf{x} \mid \mathbf{a}^T \mathbf{x} = b\}$;
- halfspace（半空间）： $\{\mathbf{x} \mid \mathbf{a}^T \mathbf{x} \leq b\}$;
- norm ball（范数球）： $\{x \mid \|x - x_c\| \leq r\}$;
- polyhedron（多面体）：linear inequalities/equalities（线性不等式 / 等式）的交集；
- ellipsoid（椭球）：

$$\{x \mid (x - x_c)^T P^{-1}(x - x_c) \leq 1\}$$

重要性质：convex sets（凸集）的 intersection（交集）仍然 convex（凸）。

8.4 Convex Function and Convex Domain（凸函数与凸定义域）

是的，Roy 这里的意思是：在标准 convex analysis（凸分析）里，一个 convex function（凸函数）的 domain（定义域）必须是 convex set（凸集）。

函数 f 是 convex function，如果它的 domain $\text{dom } f$ 是 convex set，并且对任意

$$\mathbf{x}, \mathbf{y} \in \text{dom } f, \quad \gamma \in [0, 1],$$

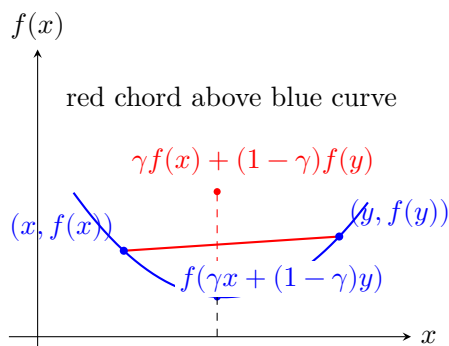
都有：

$$f(\gamma \mathbf{x} + (1 - \gamma) \mathbf{y}) \leq \gamma f(\mathbf{x}) + (1 - \gamma) f(\mathbf{y}).$$

这就是 Roy 说的：

the line between two function values lies above the function

左边是先在 input space（输入空间）里取 $\mathbf{x}$ 和 $\mathbf{y}$ 的 convex combination（凸组合），再计算函数值。右边是直接把两个 function values（函数值）用直线连接起来。Convex function 的图像在这条 chord（弦线）下面。



为什么 domain 也必须 convex?

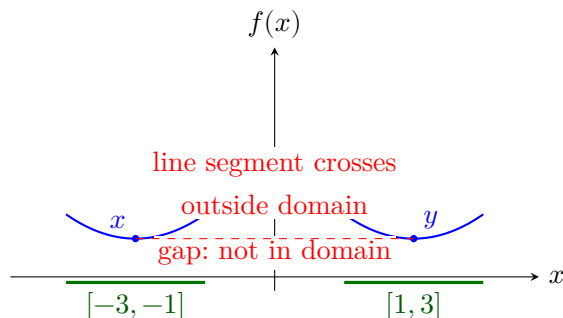
因为上面的定义需要 $\gamma \mathbf{x} + (1 - \gamma) \mathbf{y}$ 这个点仍然在 domain 里。也就是说，如果 $\mathbf{x}$ 和 $\mathbf{y}$ 都是允许输入，那么它们中间的所有 convex combinations 也必须是允许输入。否则下面这个表达式可能根本没有定义：

$$f(\gamma \mathbf{x} + (1 - \gamma) \mathbf{y}).$$

例如，假设一个函数只定义在两个分开的区间上：

$$\text{dom } f = [-3, -1] \cup [1, 3].$$

这个 domain 不是 convex set，因为取 $x = -2$ 和 $y = 2$ ，中点 0 不在 domain 里。即使函数在每一段上看起来都像 bowl-shaped（碗形），整体上也不能叫 convex function，因为连接两个可行输入的 line segment 中间跑出了定义域。



所以 Roy 的逻辑是：

convex set first:

we need line segments between feasible points to stay feasible

convex function second:

along those feasible line segments,

the function value stays below the straight-line interpolation

在 optimisation 里，这一点很重要。A convex optimisation problem 通常要求：

- objective function（目标函数）是 convex；
- feasible region / domain（可行域 / 定义域）是 convex；
- constraints（约束）也以保持 convex feasible set 的方式写出来。

如果 domain 不是 convex，那么即使函数曲线局部看起来很“碗形”，也会破坏 convex optimisation 的核心性质：local optimum（局部最优）不一定是 global optimum（全局最优），线段插值也不再可靠。

8.4.1 Strictly Convex vs Convex (严格凸 vs 凸)

Roy 后面说:

a strictly convex function is always curving.
But one that's just convex could have a flat region.

这句话的数学版本是:

Convex function (凸函数) 要求:

$$f(\gamma \mathbf{x} + (1 - \gamma) \mathbf{y}) \leq \gamma f(\mathbf{x}) + (1 - \gamma) f(\mathbf{y}).$$

Strictly convex function (严格凸函数) 要求更强: 只要 $\mathbf{x} \neq \mathbf{y}$ 且 $\gamma \in (0, 1)$, 就有:

$$f(\gamma \mathbf{x} + (1 - \gamma) \mathbf{y}) < \gamma f(\mathbf{x}) + (1 - \gamma) f(\mathbf{y}).$$

区别就在这个 $\leq$ 和 $<$ 。

直觉:

convex:

graph can touch the chord over an interval
可以有平的一段 / 直的一段

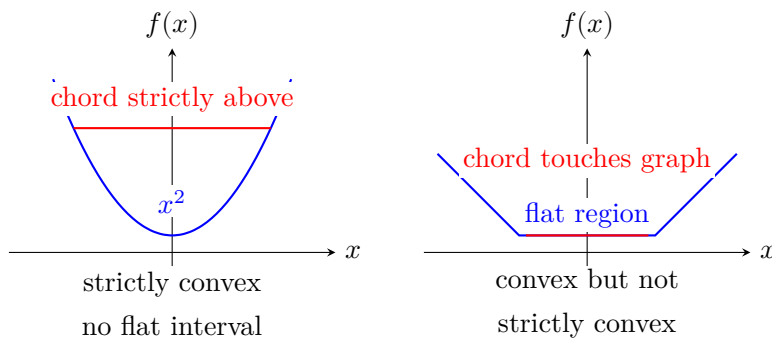
strictly convex:

between any two different points, the graph stays strictly below the chord
中间不能贴着弦线走

所以 $f(x) = x^2$ 是 strictly convex; 而

$$f(x) = \begin{cases} 0, & |x| \leq 1, \\ |x| - 1, & |x| > 1 \end{cases}$$

是 convex 但不是 strictly convex, 因为它在 $[-1, 1]$ 上有 flat region (平坦区域)。



为什么这对 optimisation 很重要?

- 如果 f 是 convex, 任何 local minimum 都是 global minimum。
- 如果 f 是 strictly convex, 并且 minimum 存在, 那么 minimizer (最优解点) 通常是 unique (唯一的)。
- 如果只是 convex 但有 flat bottom (平底), 可能有 infinitely many minimizers (无穷多个最优解)。

例如左边的 x^2 只有一个 minimizer: $x = 0$ 。右边的 flat-bottom convex function 在整个 $[-1, 1]$ 上都是 minimum, 所以最优值唯一, 但最优解不唯一。

在工程里, 这对应一个很实际的问题:

```
convex objective:
    easy to know we found a global optimum

strictly convex objective:
    also helps guarantee a unique solution

flat convex objective:
    many parameter values may be equally optimal
```

所以 Roy 说 “strictly convex is always curving” 是一个直觉说法。更严谨地说, strict convexity 禁止函数在两个不同点之间沿着一条 straight line / flat segment 走; 普通 convexity 允许这种 flat region。

8.4.2 Convex Functions with Vector Inputs (向量输入的凸函数)

Roy 接下来把 x 从 scalar (标量, 一个数) 推广到 vector (向量, 一系列数)。这时不要被符号吓到, 核心规则没有变。

Scalar input (一维输入) 时:

$$x \in \mathbb{R}, \quad f(x) \in \mathbb{R}.$$

Vector input (多维输入) 时:

$$\mathbf{x} \in \mathbb{R}^n, \quad f(\mathbf{x}) \in \mathbb{R}.$$

注意: 这里通常还是 scalar-valued function (标量值函数)。意思是 input 是一个 vector, 但 output 是一个 number。例如:

```
input:
    x = [x1, x2, ..., xn]^T

output:
    f(x) = one number
```

比如二维：

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad f(\mathbf{x}) = x_1^2 + x_2^2.$$

这里 $\mathbf{x}$ 是一个点，也可以理解成 feature vector / decision vector (特征向量 / 决策变量)。 $f(\mathbf{x})$ 是这个点对应的 height / cost / objective value (高度 / 成本 / 目标函数值)。

Convex function 的定义还是同一条：

$$f(\gamma\mathbf{x} + (1-\gamma)\mathbf{y}) \leq \gamma f(\mathbf{x}) + (1-\gamma)f(\mathbf{y}).$$

只是现在 $\mathbf{x}$ 和 $\mathbf{y}$ 是 vectors：

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}.$$

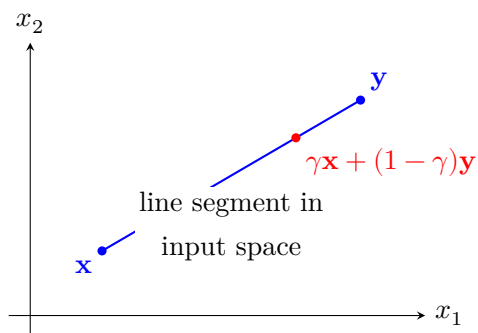
它们的 convex combination 是 componentwise (逐分量) 算的：

$$\gamma\mathbf{x} + (1-\gamma)\mathbf{y} = \begin{bmatrix} \gamma x_1 + (1-\gamma)y_1 \\ \gamma x_2 + (1-\gamma)y_2 \\ \vdots \\ \gamma x_n + (1-\gamma)y_n \end{bmatrix}.$$

所以如果 $\gamma = 0.25$ ，那么这个点就是：

25% of $\mathbf{x}$ + 75% of $\mathbf{y}$

它仍然是 $\mathbf{x}$ 和 $\mathbf{y}$ 之间 line segment (线段) 上的一个点。



这里要区分两个空间：


```

input space:
  where x and y live
  比如  $\mathbb{R}^2$ ,  $\mathbb{R}^3$ ,  $\mathbb{R}^n$ 

output/value space:
  where  $f(x)$  and  $f(y)$  live
  通常是一条 real number axis

```

Roy 说 “you need two points”，指的是：判断 convexity 时要取 domain 里的任意两个 input points $\mathbf{x}$ 和 $\mathbf{y}$ ，然后看它们中间所有 convex combinations 的函数值是否低于 chord。

8.4.2.1 How to Read the Sum Symbol (如何读懂求和符号)

向量函数经常出现 $\sum$ 。它只是 “add many similar terms” (把很多相似项加起来)。

例如：

$$\sum_{i=1}^n x_i^2$$

读作：

```
sum from i = 1 to n of x_i squared
```

展开就是：

$$x_1^2 + x_2^2 + \cdots + x_n^2.$$

如果 $n = 3$ ，并且

$$\mathbf{x} = \begin{bmatrix} 2 \\ -1 \\ 4 \end{bmatrix},$$

那么：

$$\sum_{i=1}^3 x_i^2 = 2^2 + (-1)^2 + 4^2 = 4 + 1 + 16 = 21.$$

所以一重求和符号的读法可以很机械：

```

i = 1: use x1
i = 2: use x2
...

```

```
i = n: use xn
then add them all
```

8.4.2.2 Double Sum and Matrix Form (双重求和与矩阵形式)

Roy 的讲义里还出现了 double sum (双重求和):

$$\sum_{i=1}^m \sum_{j=1}^n A_{ij} X_{ij}.$$

读作:

```
sum over i from 1 to m,
and for each i, sum over j from 1 to n,
of A_ij times X_ij.
```

最机械的理解是: matrix (矩阵) 有 row index (行指标) i 和 column index (列指标) j 。

```
i tells which row.
j tells which column.
A_ij means entry of A at row i, column j.
X_ij means entry of X at row i, column j.
```

如果 $m = 2, n = 3$, 那么:

$$A = \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \end{bmatrix}, \quad X = \begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \end{bmatrix}.$$

双重求和展开就是:

$$\sum_{i=1}^2 \sum_{j=1}^3 A_{ij} X_{ij} = A_{11}X_{11} + A_{12}X_{12} + A_{13}X_{13} + A_{21}X_{21} + A_{22}X_{22} + A_{23}X_{23}.$$

也就是说:

```
go row by row,
inside each row go column by column,
multiply matching entries,
then add everything.
```

这就是 matrices 的 inner product (矩阵内积), 常写成:

$$\langle A, X \rangle = \sum_{i=1}^m \sum_{j=1}^n A_{ij} X_{ij}.$$

Roy 截图中的 affine function on matrices (矩阵上的仿射函数) 是:

$$f(X) = \text{tr}(A^T X) + b = \sum_{i=1}^m \sum_{j=1}^n A_{ij} X_{ij} + b.$$

这里:

- $X \in \mathbb{R}^{m \times n}$ 是 matrix input (矩阵输入);
- $A \in \mathbb{R}^{m \times n}$ 是 fixed matrix (固定矩阵), 类似 vector case 里的 coefficient vector;
- $b \in \mathbb{R}$ 是 scalar offset (标量偏移);
- $f(X) \in \mathbb{R}$ 仍然是 one number (一个数)。

这和 vector affine function 完全平行:

$$f(\mathbf{x}) = \mathbf{a}^T \mathbf{x} + b = \sum_{i=1}^n a_i x_i + b.$$

对比表:

Input type	Affine function	Expanded sum	Meaning
vector $\mathbf{x} \in \mathbb{R}^n$	$\mathbf{a}^T \mathbf{x} + b$	$\sum_{i=1}^n a_i x_i + b$	match each vector entry
matrix $X \in \mathbb{R}^{m \times n}$	$\text{tr}(A^T X) + b$	$\sum_{i=1}^m \sum_{j=1}^n A_{ij} X_{ij} + b$	match each matrix entry

所以 $\text{tr}(A^T X)$ 不要先理解成一个神秘的 trace trick。先把它读成:

```
multiply A and X entry-by-entry,
then add all entries.
```

为什么是 $A^T X$ 而不是直接 AX ? 因为 trace formula 是矩阵内积的紧凑写法:

$$\text{tr}(A^T X) = \langle A, X \rangle.$$

其中 trace (迹) 表示 square matrix (方阵) 对角线元素之和:

$$\text{tr} \left(\begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} \right) = c_{11} + c_{22}.$$

但在读 optimisation 讲义时, 你可以先记住:

vector dot product:

```
a^T x = sum_i a_i x_i
```

matrix dot product:

```
tr(A^T X) = sum_i sum_j A_ij X_ij
```

它们都是 affine functions，所以都是 convex functions and concave functions（既凸也凹）。原因是它们没有 curvature（曲率）：图像是 flat hyperplane（平的超平面）。

和 p-norm 的关系是：

```
affine function:
    weighted sum of entries

norm:
    size of vector/matrix
```

Roy 这一页是在给 $\mathbb{R}^n$ 和 $\mathbb{R}^{m \times n}$ 上的 convex function examples（凸函数例子）：vector 上有 affine functions 和 norms；matrix 上也有 affine functions，以及 spectral norm（最大奇异值 / 谱范数）这类 matrix norm。

8.4.2.3 Norms as Convex Functions（范数是凸函数）

Roy 说 norms are convex functions。Norm（范数）可以理解成 vector 的 size / length（大小 / 长度）。

最常见的是 Euclidean norm / 2-norm：

$$\|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2} = \sqrt{\sum_{i=1}^n x_i^2}.$$

二维时：

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad \|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2}.$$

这就是 Pythagoras（勾股定理）：点 (x_1, x_2) 到 origin（原点）的距离。

更一般的 p-norm 定义为：

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}, \quad p \geq 1.$$

逐步读：

```
1. take each component xi
2. take its magnitude |xi|
3. raise it to power p
4. add all components
5. take the 1/p power
```

例如：

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i| = |x_1| + \cdots + |x_n|$$

叫 1-norm / Manhattan norm。二维里它的 norm ball 是 diamond shape (菱形)。

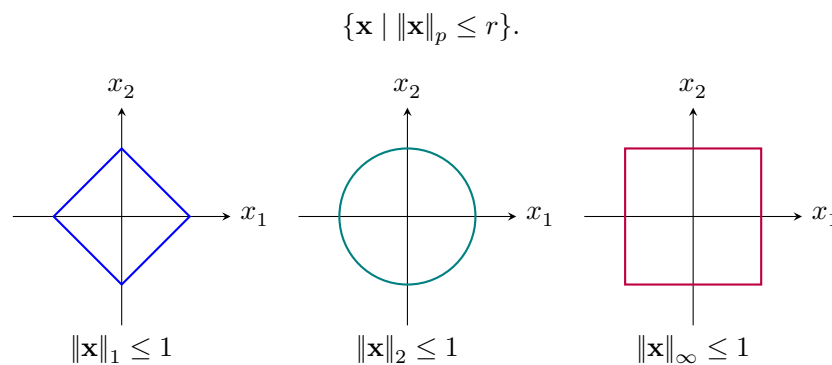
$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$$

叫 2-norm / Euclidean norm。二维里它的 norm ball 是 circle (圆)。

$$\|\mathbf{x}\|_\infty = \max_i |x_i|$$

叫 infinity norm。二维里它的 norm ball 是 square (正方形)。

这些就是 Roy 前面画过的 norm balls:



为什么 norms 是 convex functions? 直觉上, vector 的 length 不会因为在两个 vectors 中间插值而超过两端长度的线性插值:

$$\|\gamma \mathbf{x} + (1 - \gamma) \mathbf{y}\| \leq \gamma \|\mathbf{x}\| + (1 - \gamma) \|\mathbf{y}\|.$$

这其实来自 triangle inequality (三角不等式):

$$\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|.$$

所以当 Roy 说 “P-norm is exactly those balls that I drew before”, 可以理解成:

norm function:

maps each vector $\mathbf{x}$ to its size $\|\mathbf{x}\|_p$

norm ball:

all vectors whose size is less than or equal to r

也就是：

$$\text{function } \|\mathbf{x}\|_p \implies \text{set } \{\mathbf{x} \mid \|\mathbf{x}\|_p \leq r\}.$$

一句话总结：

Vector-input convex functions use the same convexity rule.
The only new difficulty is reading vector components and sums.
Norms are the first major examples:
they turn vectors into scalar lengths, and their sublevel sets are norm balls.

8.5 Quasi-convex Functions (准凸函数)

Roy 后面讲 quasi-convex functions (准凸函数)。它们经常出现在 control systems (控制系统) 和一些 optimisation problems 里。

准凸函数不一定是 convex function。它的定义不是看 graph (函数图像) 是否在 chord 下面，而是看所有 sublevel sets (下水平集) 是否 convex。

首先，domain 仍然必须是 convex set：

$$\text{dom } f \text{ is convex.}$$

然后对任意 real number α ，定义 sublevel set：

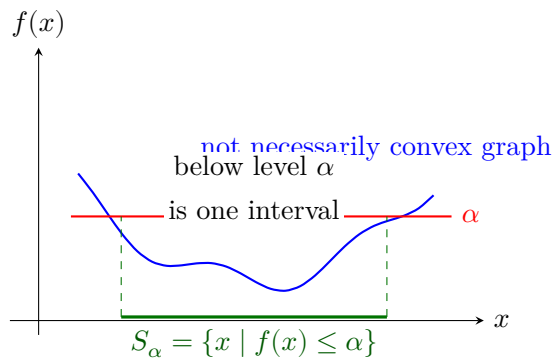
$$S_\alpha = \{\mathbf{x} \in \text{dom } f \mid f(\mathbf{x}) \leq \alpha\}.$$

读法：

S_α is the set of all $\mathbf{x}$ whose function value is no larger than α .
 S_α 是所有函数值不超过 α 的点。

如果对每个 α ， S_α 都是 convex set，那么 f 就是 quasi-convex function：

$$f \text{ is quasi-convex} \iff S_\alpha \text{ is convex for every } \alpha.$$



一维里，convex set 就是 interval（区间）。所以一维 quasi-convex function 的直觉是：

for every horizontal level alpha,
the part below alpha is one interval, not separated pieces.

这允许函数图像有一些不满足 convex graph inequality 的形状，但它不能有多个 separated valleys（分开的谷）。如果某个 level α 下面有两段分开的区域，那么 sublevel set 不是 convex，这个函数就不是 quasi-convex。

8.5.1 Convex vs Quasi-convex（凸 vs 准凸）

所有 convex functions 都是 quasi-convex，但反过来不成立。

原因是：convex function 的 sublevel sets 一定是 convex sets：

$$\{\mathbf{x} \mid f(\mathbf{x}) \leq \alpha\}$$

会形成 convex feasible region。这个性质是 convex optimisation 里非常常用的。

但 quasi-convex 只要求 sublevel sets convex，不要求 graph itself convex。

对比：

Function type	中文	看什么	要求
convex function	凸函数	graph / chord	graph below chord
quasi-convex function	准凸函数	sublevel sets	every S_α is convex

8.5.2 Why Bisection Works（为什么可以用二分法）

Roy 说 quasi-convex functions 给了你一种求解方式：不要直接沿着 gradient descent（梯度下降）走，而是检查 sublevel set 是否可行，然后对 α 做 bisection（二分法）。

关键性质是 nested sublevel sets（嵌套下水平集）：

如果 $\alpha_1 \leq \alpha_2$ ，那么：

$$S_{\alpha_1} \subseteq S_{\alpha_2}.$$

直觉:

```
lower alpha:
    stricter requirement  $f(\mathbf{x}) \leq \alpha$ 
    smaller set

higher alpha:
    looser requirement  $f(\mathbf{x}) \leq \alpha$ 
    larger set
```

所以我们可以把求最小值改写成一系列 feasibility problems (可行性问题):

$$\text{Is there any } \mathbf{x} \text{ such that } f(\mathbf{x}) \leq \alpha?$$

也就是问:

$$S_\alpha \neq \emptyset?$$

如果某个 α 可行, 说明 minimum value (最小函数值) 不超过 α 。如果不可行, 说明 α 太低了。

Bisection 的流程:

```
1. Find lower and upper bounds:  $L \leq f^* \leq U$ .
2. Pick midpoint  $\alpha = (L + U)/2$ .
3. Check feasibility of  $f(\mathbf{x}) \leq \alpha$ .
4. If feasible:
     $U = \alpha$ 
else:
     $L = \alpha$ 
5. Repeat until  $U - L$  is small enough.
```

这里 f^* 表示 global minimum value (全局最小值)。

准凸函数的好处是: 每次检查 $f(\mathbf{x}) \leq \alpha$ 时, sublevel set 是 convex set, 因此这个 feasibility problem 往往可以用 convex optimisation methods 处理。

8.5.3 Bisection Example Exercise (准凸二分法习题)

你提到的 “Amoji” 很可能是 Armijo line search (Armijo 线搜索)。Armijo 是 gradient descent / Newton method 里选择 step size (步长) 的方法; Roy 这里讲的不是 Armijo, 而是 bisection on objective value (对目标函数值做二分)。

区别:

Armijo line search:

choose a step size along a descent direction

沿着下降方向选步长

quasi-convex bisection:

choose a candidate objective value alpha

test whether $f(x) \leq \alpha$ is feasible

对目标值 alpha 做可行性检查

下面做一个具体习题。

Problem. 在 $x > 0$ 上最小化:

$$f(x) = \max \left\{ x, \frac{4}{x} \right\}.$$

也就是:

$$\min_{x>0} \max \left\{ x, \frac{4}{x} \right\}.$$

这个函数是 quasi-convex, 因为它的 sublevel set 是 convex interval (凸区间)。对某个候选值 α , 检查:

$$f(x) \leq \alpha.$$

这等价于同时满足:

$$x \leq \alpha, \quad \frac{4}{x} \leq \alpha.$$

因为 $x > 0$, 第二个不等式可以改写成:

$$x \geq \frac{4}{\alpha}.$$

所以 sublevel set 是:

$$S_\alpha = \left\{ x > 0 \mid \frac{4}{\alpha} \leq x \leq \alpha \right\}.$$

这是一个 interval, 所以是 convex set。

现在 feasibility question (可行性问题) 变成:

Is there any x such that $4/\alpha \leq x \leq \alpha$?

也就是说，区间必须非空：

$$\frac{4}{\alpha} \leq \alpha.$$

因为 $\alpha > 0$ ，所以：

$$4 \leq \alpha^2 \iff \alpha \geq 2.$$

这已经告诉我们 global minimum value 是：

$$f^* = 2.$$

对应的 minimizer 是：

$$x^* = 2,$$

因为：

$$f(2) = \max \left\{ 2, \frac{4}{2} \right\} = 2.$$

但假设我们不知道答案，只用 bisection 来找。

先给一个 bracket（夹逼区间）：

$$0 \leq f^* \leq 5.$$

然后每次取中点 $\alpha = (L + U)/2$ ，检查 S_α 是否非空。

Step	L	U	$\alpha = (L + U)/2$	Feasible?	Reason
1	0	5	2.5	yes	$4/2.5 = 1.6 \leq 2.5$
2	0	2.5	1.25	no	$4/1.25 = 3.2 > 1.25$
3	1.25	2.5	1.875	no	$4/1.875 \approx 2.13 > 1.875$

Step	L	U	$\alpha = (L + U)/2$	Feasible?	Reason
4	1.875	2.5	2.1875	yes	$4/2.1875 \approx 1.83 \leq 2.1875$
5	1.875	2.1875	2.03125	yes	$4/2.03125 \approx 1.97 \leq 2.03125$
6	1.875	2.03125	1.953125	no	$4/1.953125 \approx 2.05 > 1.953125$

现在我们知道：

$$1.953125 \leq f^* \leq 2.03125.$$

继续二分，区间会越来越窄，最后夹到 $f^* = 2$ 。

这个例子的关键不是梯度，而是：

alpha too low:

```
S_alpha is empty
no x can make f(x) <= alpha
```

alpha high enough:

```
S_alpha is nonempty
there exists x with f(x) <= alpha
```

这就是 quasi-convex bisection 的核心。

Exercise for you. 用同样方法做：

$$\min_{x>0} \max \left\{ 2x, \frac{8}{x} \right\}.$$

提示：

$$2x \leq \alpha, \quad \frac{8}{x} \leq \alpha.$$

所以：

$$x \leq \frac{\alpha}{2}, \quad x \geq \frac{8}{\alpha}.$$

问题变成：什么时候区间

$$\left[\frac{8}{\alpha}, \frac{\alpha}{2} \right]$$

非空？

答案应该得到：

$$\alpha^* = 4, \quad x^* = 2.$$

8.5.4 Why Gradient Descent Can Be Misleading (为什么梯度下降可能误导)

Roy 说 flat does not mean you're at the bottom。对 quasi-convex functions，某些区域可能变 flat（梯度很小或为零），但这不一定表示已经到 global minimum。

所以：

```
convex / quasi-convex structure:
  tells us global geometry of level sets

gradient:
  only tells us local slope
```

在 quasi-convex problems 里，用 bisection on alpha 通常比盲目 gradient descent 更稳，因为它利用的是 sublevel sets 的 global convex structure（全局凸集合结构）。

一句话：

```
Convex function:
  graph has convex shape.

Quasi-convex function:
  every below-alpha region has convex shape.

This is weaker than convexity,
but still strong enough to support global search by bisection.
```

8.5.5 80/20 Takeaway: Back to Structure (核心思想：回到结构)

这一段最值得带走的思想不是某个公式，而是：

```
Optimization is not the same thing as gradient descent.
Optimization is about exploiting problem structure.
```

Gradient descent (梯度下降) 只是很多 optimisation algorithms 里的一个 family。它依赖 local slope (局部斜率)。但 Roy 讲 quasi-convexity 的重点是: 有时我们不需要先问 “gradient 是多少”, 而应该先问:

What is the structure of the feasible sets?

What is the geometry of the level sets?

对于 quasi-convex optimisation:

$$\min_{\mathbf{x}} f(\mathbf{x})$$

可以转成:

given α , is there $\mathbf{x}$ such that $f(\mathbf{x}) \leq \alpha$?

也就是:

```
Optimization
=
sequence of feasibility problems.
```

Bisection 搜索的是 objective value (目标函数值) α :

```
guess alpha
check feasibility of f(x) <= alpha

feasible:
    alpha may be too high, lower U

infeasible:
    alpha is too low, raise L
```

Armijo line search 搜索的是 step size (步长) t 。它服务于 gradient-based method (基于梯度的方法):

$$f(\mathbf{x}_k + t\mathbf{d}_k) \leq f(\mathbf{x}_k) + ct \nabla f(\mathbf{x}_k)^T \mathbf{d}_k.$$

含义是: 沿着 descent direction (下降方向) 走 t 这么远, 函数值必须下降得足够明显。

Backtracking line search (回溯线搜索) 的典型形式是:

```
t = 1
while Armijo condition is not satisfied:
    t = beta * t
```

通常 $\beta = 0.5$ 。

对比：

Method	搜索变量	含义	判断标准	主要用途	依赖
quasi-convex bisection	α	candidate objective value (候选目标值)	feasibility: $S_\alpha \neq \emptyset$	global search over value	level-set geometry
Armijo line search	t	step size (步长)	sufficient decrease (足够 下降)	local step selection	gradient

所以这两者不是同一个算法：

Quasi-convex bisection asks:

Is this target value achievable?

Armijo asks:

Is this gradient step good enough?

对 machine learning / data science 的启发是：不要把 “optimization” 简化成 “take gradient”。很多时候真正重要的是 modeling (建模) 和 problem structure recognition (问题结构识别)。

可能有用的 structure 包括：

- convex structure (凸结构)；
- quasi-convex / level-set structure (准凸 / 水平集结构)；
- constraint structure (约束结构)；
- combinatorial structure (组合结构)；
- coverage / facility-location structure (覆盖 / 设施选址结构)。

工程实践里，很多时候你不会手写 quasi-convex bisection，而是调用：

```
problem.solve()
model.fit()
```

但你仍然需要知道自己交给 solver 的是什么结构。因为 solver 能不能可靠地找到 global optimum，往往取决于你有没有把问题建模成正确的 structure。

最值得记忆的一句话：

When gradient information becomes messy,
look at the geometry of feasible sets.

或者更工程一点：

Do not first ask: what is the gradient?
First ask: what is the structure of this problem?

这也是从 “data science 做模型” 走向 “optimisation and decision science 做系统” 的重要思维转变。

8.6 Cones and PSD Cone (锥与正半定锥)

Cone (锥):

$$x \in C, \gamma \geq 0 \Rightarrow \gamma x \in C$$

也就是说，只要某个 direction (方向) 在集合里，沿着这个 direction 从 origin (原点) 往外放大仍然在集合里。Cone 只保证 single ray (单条射线) 上的 scaling，不自动保证两条 ray 中间的点也在集合里。

Convex cone (凸锥) 要再加一个条件：任意 nonnegative linear combination (非负线性组合)，也叫 conic combination (锥组合)，仍然在集合里：

$$x_1, x_2 \in C, \gamma_1, \gamma_2 \geq 0 \Rightarrow \gamma_1 x_1 + \gamma_2 x_2 \in C$$

这就是 Roy 说的：

not just that one stays in,
but every positive combination,
including every line in between.

对比记忆：

Object	中文	要求	直觉
cone	锥	$x \in C \Rightarrow \gamma x \in C, \gamma \geq 0$	每个方向可以从原点往外拉长
convex set	凸集	$\gamma x_1 + (1 - \gamma)x_2 \in C, \gamma \in [0, 1]$	两点之间的线段在集合里
convex cone	凸锥	$\gamma_1 x_1 + \gamma_2 x_2 \in C, \gamma_i \geq 0$	既能沿方向放大，也能在方向之间做非负组合

Positive semidefinite cone / PSD cone (正半定锥):

$$S_+^n = \{X \in S^n \mid X \succeq 0\}$$

Lecture 8 讲义里的 PSD cone 图可以用最小的 2×2 symmetric matrix (对称矩阵) 理解。令：

$$X = \begin{bmatrix} x & y \\ y & z \end{bmatrix}$$

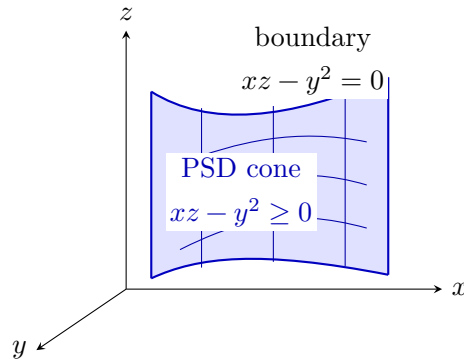
则:

$$X \succeq 0 \iff x \geq 0, \quad z \geq 0, \quad xz - y^2 \geq 0.$$

边界是:

$$xz - y^2 = 0 \iff z = \frac{y^2}{x} \quad (x > 0).$$

PSD cone 是这个 curved boundary (弯曲边界) 以上、且 x, z 非负的区域:



8.6.1 Why Split the Matrix into Basis Matrices?

Roy 接着把:

$$X = \begin{bmatrix} x & y \\ y & z \end{bmatrix}$$

写成:

$$X = x \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} + y \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} + z \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}.$$

这个写法的重点是: 把 matrix variable (矩阵变量) 表示成 decision variables (决策变量) 乘以 fixed basis matrices (固定基矩阵)。

也就是说:

variables:

x, y, z

fixed matrices:

$F1, F2, F3$

matrix expression:

$$X(x,y,z) = x F_1 + y F_2 + z F_3$$

这样做的好处有三个。

第一，它把 PSD constraint 写成 standard LMI form:

$$F(\theta) = F_0 + \theta_1 F_1 + \cdots + \theta_m F_m \succeq 0.$$

在这个例子里 $F_0 = 0$, $\theta = (x, y, z)$ 。

第二，它让 optimisation software 更容易接收问题。Solver 不需要你手工展开:

$$xz - y^2 \geq 0$$

这种 nonlinear-looking scalar inequality。你只要告诉 solver:

this affine matrix expression must lie in the PSD cone

也就是:

$$xF_1 + yF_2 + zF_3 \in S_+^2.$$

第三，它把 LP、SOCP、SDP/LMI 放进同一种 conic modelling language:

linear expression in variables
must lie inside a cone

对 LP, cone 是 $\mathbb{R}_+^m$ 。

对 SOCP, cone 是 second-order cone。

对 LMI/SDP, cone 是 PSD cone。

8.6.2 Is This Matrix Blocking?

这和 matrix block partitioning (矩阵分块) 有关, 但不是同一个操作。

这里的操作是 basis expansion (基矩阵展开):

write a symmetric matrix as
linear combination of fixed matrices.

它的目的: 让 solver 看见 affine matrix expression (仿射矩阵表达式)。

Matrix blocking / partitioning (矩阵分块) 是另一件事:

```
split a large matrix into submatrices,
for example
```

```
[ Q   S ]
[ S^T R ]
```

它的目的通常是做 algebraic manipulation (代数操作), 例如 Schur complement (舒尔补)、elimination、或者把 control LMI 改写成可解形式。

所以两者关系可以这样记:

Operation	中文	在这里的作用
basis matrix expansion	基矩阵展开	把变量放进 fixed matrix basis, 形成 affine matrix expression
block partitioning	矩阵分块	把矩阵按子块切开, 方便 Schur complement / LMI transformation

在本章中, 两者都有用:

- PSD cone example 用 basis expansion 表示 $X = xF_1 + yF_2 + zF_3$;
- Schur complement 和 control LMI 会大量用 block partitioning。

PSD (positive semidefinite, 正半定) 的等价理解:

$$\mathbf{v}^T X \mathbf{v} \geq 0 \quad \text{for all } \mathbf{v}$$

或所有 eigenvalues (特征值) 非负。

易错点:

- positive semidefinite (正半定) : eigenvalues ≥ 0 ;
- positive definite (正定) : eigenvalues > 0 ;
- cone (锥) 不自动 convex (凸)。

8.7 Did We Already Learn This? LP/LMI as Cone Programming

你感觉 “之前好像没有这样学过 cone 或 cone constraints” 是正常的。

通常第一次学 linear programming (线性规划, LP) 时, 写的是 standard inequality form (标准不等式形式):

$$\begin{aligned} \min_{\mathbf{x}} \quad & \mathbf{c}^T \mathbf{x} \\ \text{subject to} \quad & \mathbf{A}\mathbf{x} \leq \mathbf{b}. \end{aligned}$$

这时我们会说：

`linear objective + linear inequalities`

但 Roy 现在换了一个更 general 的语言：cone programming（锥规划）。

核心观察是，componentwise inequality（逐分量不等式）其实等价于 saying a vector lies in the nonnegative orthant cone（非负正交象限锥）。

定义：

$$\mathbb{R}_+^m = \{\mathbf{s} \in \mathbb{R}^m \mid s_i \geq 0, i = 1, \dots, m\}.$$

这是一个 cone，因为如果 $\mathbf{s} \geq 0$ ，那么对任意 $\gamma \geq 0$ ：

$$\gamma \mathbf{s} \geq 0.$$

现在把 LP 约束：

$$A\mathbf{x} \leq \mathbf{b}$$

改写成：

$$\mathbf{b} - A\mathbf{x} \in \mathbb{R}_+^m.$$

这里 $\mathbf{b} - A\mathbf{x}$ 叫 slack vector（松弛向量）。它每个分量都非负，意思就是每条 inequality 都满足。

所以：

LP standard view:

$$A\mathbf{x} \leq \mathbf{b}$$

LP conic view:

$$\mathbf{b} - A\mathbf{x} \text{ lies in the nonnegative orthant cone } \mathbb{R}_+^m$$

这就是 Roy 说：

`linear programs are programming over the positive orthant cone`

的意思。

8.7.1 Why “Greater Than Zero” Means “Inside a Cone”（为什么“大于等于 0”就是在锥里）

Roy 说：

y - f is greater than zero
greater than zero means it belongs to your cone

这里的关键是：他已经把“ ≥ 0 ”的含义推广了。

普通 scalar case（标量情况）：

$$s \geq 0 \iff s \in \mathbb{R}_+.$$

这里 $\mathbb{R}_+$ 就是 nonnegative real line（非负实数轴），它本身是一个 cone。

Vector case（向量情况）：

$$\mathbf{s} \geq 0$$

通常不是说 vector 有大小，而是 shorthand（简写）：

$$\mathbf{s} \in \mathbb{R}_+^m \iff s_i \geq 0, i = 1, \dots, m.$$

也就是说：

vector ≥ 0
means every component is nonnegative
means the vector lies in the positive orthant cone.

Cone programming 更进一步，把这个写法推广成：

$$\mathbf{s} \succeq_K 0 \iff \mathbf{s} \in K.$$

这里 K 是你选定的 cone。因此：

Cone K	“ $s \succeq_K 0$ ” means
$\mathbb{R}_+$	scalar $s \geq 0$
$\mathbb{R}_+^m$	vector entries all nonnegative
second-order cone $\mathcal{Q}$	$(t, \mathbf{x})$ satisfies $\ \mathbf{x}\ _2 \leq t$
PSD cone S_+^n	matrix $X \succeq 0$, i.e. positive semidefinite

所以在 Roy 那句话里，如果约束写成：

$$f(x) \preceq_K y,$$

把 $f(x)$ 移到另一边:

$$y - f(x) \succeq_K 0.$$

这句话的意思不是 “ $y - f(x)$ 是普通数字且大于 0”，而是:

$$y - f(x) \in K.$$

一句话:

In cone language, "nonnegative" means "belonging to the chosen cone K ".

这就是为什么 “greater than zero” 会变成 “inside this cone”。

同理, LMI / SDP 用的不是 $\mathbb{R}_+^m$, 而是 PSD cone (正半定锥):

$$F(\mathbf{x}) \succeq 0.$$

也就是:

```
LP:
    vector inequality
    b - A x in R_+^m
    cone = nonnegative orthant

LMI / SDP:
    matrix inequality
    F(x) in S_+^n
    cone = positive semidefinite cone
```

所以你可以这样理解学习路径:

以前可能学过的形式	Roy 现在用的 cone 语言	本质
$x_i \geq 0$	$\mathbf{x} \in \mathbb{R}_+^n$	nonnegative orthant cone
$A\mathbf{x} \leq \mathbf{b}$	$\mathbf{b} - A\mathbf{x} \in \mathbb{R}_+^m$	LP over positive orthant cone
$\ \mathbf{x}\ _2 \leq t$	$(t, \mathbf{x}) \in \mathcal{Q}^{n+1}$	SOCP over second-order cone
$F(\mathbf{x}) \succeq 0$	$F(\mathbf{x}) \in \mathbb{S}_+^n$	LMI/SDP over PSD cone

一句话记忆:

You may have learned LP constraints before,
but Roy is re-framing constraints as membership in cones.
That cone-language is what lets LP, SOCP, and LMI/SDP sit in one framework.

8.7.2 Why Roy Introduces Cones (Roy 为什么要介绍 cone)

Roy 后面明确解释了为什么要引入 cone language。核心不是为了增加抽象符号，而是两个原因：

1. Cone constraints give more modelling generality.
2. There is code/software that can solve optimisation problems written in cone form.

第一，cone 让 inequality constraints（不等式约束）更 general。以前我们熟悉的是 scalar inequality：

$$f_i(x) \leq 0.$$

这只是一个 number 小于等于 0。Cone view 允许 $f(x)$ 返回：

- vector（向量）；
- symmetric matrix（对称矩阵）；
- norm expression（范数表达式）；
- affine matrix expression（仿射矩阵表达式）。

然后用：

$$y - f(x) \in K$$

统一表达约束。

第二，cone language 不是纯理论，因为 solver 确实能处理这些标准 cone：

Cone	Problem class	What solver understands
nonnegative orthant $\mathbb{R}_+^m$	LP	componentwise inequalities
second-order cone $\mathcal{Q}$	SOCP	norm constraints
PSD cone S_+^n	SDP / LMI	matrix positive semidefinite constraints

所以 Roy 的逻辑是：

If cone constraints only looked elegant but could not be solved, they would not be useful.

They are useful because modern optimisation software can solve problems written in standard cone form.

这也是为什么 CVX / CVXPY / YALMIP 这类 modelling tools 重要：你写的是数学形式，底层 solver 会识别 cone structure。

对 ENEL445 来说，cone 的学习价值是：

LP, SOCP, SDP, and LMI are not separate tricks.
They are different cones inside one modelling language.

8.8 Convex Optimisation Problem (凸优化问题)

标准形式:

$$\begin{aligned} \min_x \quad & f_0(x) \\ \text{subject to} \quad & f_i(x) \leq 0, \quad i = 1, \dots, m \\ & Ax = b \end{aligned}$$

其中:

- f_0 和 f_i convex (凸函数);
- equality constraints (等式约束) 必须 affine (仿射)。

为什么重要: convex problem (凸问题) 的 local information (局部信息) 可以支持 global solution (全局解), solver 更可靠。

9 Problem Classes: LP, QP, SOCP, QCQP, SDP

9.1 Linear Program (线性规划, LP)

LP:

$$\begin{aligned} \min_x \quad & c^T x + d \\ \text{subject to} \quad & Gx \leq h \\ & Ax = b \end{aligned}$$

特征:

- linear cost (线性目标函数);
- linear constraints (线性约束);
- feasible region (可行域) 是 polytope/polyhedron (多胞体 / 多面体)。

例子: diet problem、logistics、Chebyshev center。

9.1.1 Chebyshev Center (切比雪夫中心: 多面体内的最大球)

Roy 这里说的:

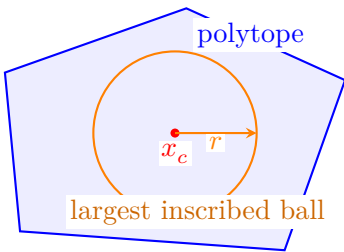
Chebyshev center (largest ball inside a polyhedron)

可以直译成：

在一个 polyhedron / polytope 里面，找一个最大的 ball。

二维里 polyhedron / polytope 可以想成一个 polygon（多边形）。Chebyshev center 就是：

找一个圆心 $\mathbf{x}_c$ 和半径 r ，
让整个圆都放在多边形里面，
并且 r 尽可能大。



在三维里，这个 ball 就是真正的 sphere（球）。在更高维里，我们仍然叫 ball，只是无法画出来。

9.1.1.1 Why Is This Useful?（为什么有用）

Chebyshev center 可以理解成 feasible region 里最“安全”的点：

it maximises the distance to the nearest constraint boundary.

如果 constraints 表示安全限制、设计公差、资源限制，那么 Chebyshev center 给出的点离所有边界都尽量远，所以 robustness / margin（鲁棒性 / 裕度）最大。

例子：

Context	Chebyshev center means
robot motion	在安全区域内找离障碍物边界最远的位置
control / robust design	找一个离所有约束边界都有最大 margin 的 operating point
parameter uncertainty	在允许参数集合中找最不靠边的 nominal parameter
manufacturing tolerance	找一个对误差最不敏感的设计中心

9.1.1.2 Why Is It an LP?（为什么它是线性规划）

设 polyhedron 由 halfspaces 定义：

$$P = \{\mathbf{x} \mid \mathbf{a}_i^T \mathbf{x} \leq b_i, i = 1, \dots, m\}.$$

每一条约束:

$$\mathbf{a}_i^T \mathbf{x} \leq b_i$$

对应一条边界 hyperplane:

$$\mathbf{a}_i^T \mathbf{x} = b_i.$$

如果圆心是 $\mathbf{x}_c$, 半径是 r , 要保证半径为 r 的 ball 全部在 polyhedron 里面, 就需要圆心到每条边界的距离至少是 r 。

点 $\mathbf{x}_c$ 到第 i 条边界的距离是:

$$\frac{b_i - \mathbf{a}_i^T \mathbf{x}_c}{\|\mathbf{a}_i\|_2}.$$

要求这个距离 $\geq r$:

$$\frac{b_i - \mathbf{a}_i^T \mathbf{x}_c}{\|\mathbf{a}_i\|_2} \geq r.$$

整理:

$$\mathbf{a}_i^T \mathbf{x}_c + r\|\mathbf{a}_i\|_2 \leq b_i.$$

注意这里 $\mathbf{a}_i$ 和 b_i 是 problem data (已知数据), 所以 $\|\mathbf{a}_i\|_2$ 是常数。Decision variables (决策变量) 是:

$$\mathbf{x}_c, \quad r.$$

因此约束:

$$\mathbf{a}_i^T \mathbf{x}_c + r\|\mathbf{a}_i\|_2 \leq b_i$$

对 $\mathbf{x}_c$ 和 r 是 linear constraint (线性约束)。

Chebyshev center problem 就是:

$$\begin{aligned} & \max_{\mathbf{x}_c, r} \quad r \\ & \text{subject to} \quad \mathbf{a}_i^T \mathbf{x}_c + r\|\mathbf{a}_i\|_2 \leq b_i, \quad i = 1, \dots, m, \\ & \quad \quad \quad r \geq 0. \end{aligned}$$

这就是 LP:

```
linear objective: maximise r
linear constraints: one margin constraint per halfspace
```

所以“多胞体最大球”不是说把一个球变成多胞体，而是：

```
inside a polytope/polyhedron,
find the largest ball that fits inside it.
```

9.1.1.3 One-Sentence Memory (记忆)

```
Chebyshev center:
the point with the largest minimum distance
to all constraint boundaries.
```

9.2 Quadratic Program (二次规划, QP)

QP:

$$\begin{aligned} \min_x \quad & \frac{1}{2}x^T Px + c^T x + d \\ \text{subject to} \quad & Gx \leq h \\ & Ax = b \end{aligned}$$

特征：

- quadratic cost (二次目标函数)；
- linear constraints (线性约束)。

例子：

- constrained least squares (带约束最小二乘)；
- portfolio optimisation (投资组合优化)；
- simple MPC (简单模型预测控制)。

9.2.1 Contour Geometry: LP vs QP (等高线几何：LP vs QP)

Roy 这里的图像很重要：LP 和 QP 可以画在同一个 polytopic feasible region (多胞体可行域) 上，但 objective contours (目标函数等高线) 形状不同。

LP: straight contour lines (LP: 等高线是直线)

LP objective:

$$c^T x + d.$$

固定一个 objective value α :

$$c^T x + d = \alpha.$$

这是一条 hyperplane。在二维里就是一条直线。不同 α 的 contours 是一组 parallel lines (平行直线)。

Minimisation 可以想成:

slide the contour line in the decreasing direction
until it first touches the feasible polytope.

通常情况下, first touch (第一次碰到) 会发生在一个 vertex (顶点)。如果 contour line 正好和一条 edge (边) 平行, 也可能整条 edge 都是 optimal, 解不唯一。

QP: curved contour lines (QP: 等高线是曲线)

QP objective:

$$\frac{1}{2}x^T P x + c^T x + d.$$

如果 $P \succ 0$, 它的 contours 通常是 ellipses (椭圆):

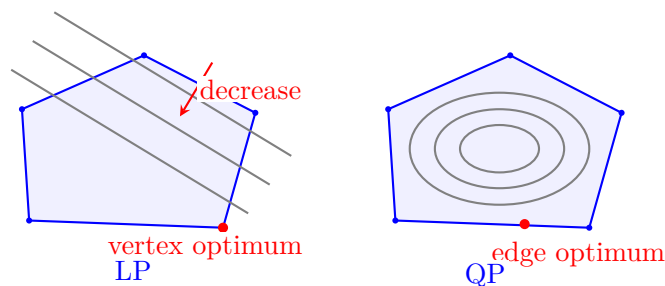
$$\frac{1}{2}x^T P x + c^T x + d = \alpha.$$

所以 QP 可以想成:

grow/shrink a curved contour
until it first touches the feasible polytope.

这次 first touch 不一定在 vertex。它可能在:

- feasible region 内部;
- 某条 edge 的中间;
- 某个 vertex;
- 多个点组成的 face 上。



这就是 Roy 说 “LP 以前很多算法沿着 vertices 找, QP 的 curved contours 不一定落在 vertex” 的意思。

9.2.1.1 Why This Matters (为什么这重要)

同样是 linear constraints:

$$Gx \leq h, \quad Ax = b,$$

feasible region 都可以是 polytope / polyhedron。

差别在 objective:

Problem	Contours	Typical optimum location
LP	parallel lines / hyperplanes	usually a vertex; sometimes an edge/face
QP	ellipses / curved level sets	interior, edge, or vertex

所以不要把 “linear constraints” 和 “LP” 混同。

QP 也可以有 linear constraints; 它之所以是 QP, 是因为 objective 是 quadratic。

9.2.2 How OLS Becomes a QP (OLS 如何变成 QP)

Roy 这里说的例子是 least squares:

$$\min_{\theta} \|\Phi\theta - y\|_2^2.$$

这就是 ordinary least squares (OLS) 的 optimisation form。

把平方展开:

$$\begin{aligned} \|\Phi\theta - y\|_2^2 &= (\Phi\theta - y)^T(\Phi\theta - y) \\ &= \theta^T \Phi^T \Phi \theta - 2y^T \Phi \theta + y^T y. \end{aligned}$$

QP 的标准 objective 是:

$$\frac{1}{2}x^T Px + c^T x + d.$$

把 x 换成 θ , OLS 可以写成:

$$\frac{1}{2}\theta^T P\theta + c^T \theta + d$$

其中:

$$P = 2\Phi^T\Phi, \quad c = -2\Phi^Ty, \quad d = y^Ty.$$

所以:

OLS is an unconstrained QP.

更准确地说, OLS 是:

quadratic cost
no constraints

如果 $\Phi^T\Phi$ invertible (可逆), 它有 closed-form solution (闭式解):

$$\hat{\theta} = (\Phi^T\Phi)^{-1}\Phi^Ty.$$

这就是 normal equations 给出的 OLS 解。

9.2.2.1 Constrained Least Squares (带约束最小二乘)

Roy 接着说, 可以给 θ 加 lower / upper bounds:

$$\mathbf{L} \leq \theta \leq \mathbf{U}.$$

这表示每个 parameter (参数) 必须落在允许范围内。例如:

parameter 1 cannot be negative
parameter 2 must be less than a physical maximum
parameter 3 must stay within an engineering tolerance

这时问题变成:

$$\begin{aligned} \min_{\theta} \quad & \|\Phi\theta - y\|_2^2 \\ \text{subject to} \quad & \mathbf{L} \leq \theta \leq \mathbf{U}. \end{aligned}$$

Bounds 可以写成 linear inequalities:

$$\theta \leq \mathbf{U}, \quad -\theta \leq -\mathbf{L}.$$

所以 constrained least squares 是:

quadratic cost + linear constraints -> QP

这就是它和 QP 的联系。

9.2.2.2 Why No Normal Equation Closed Form? (为什么没有普通正规方程闭式解)

Unconstrained OLS 的最优点满足：

$$\nabla_{\theta} \|\Phi\theta - y\|_2^2 = 0.$$

这给出 normal equations：

$$\Phi^T \Phi \theta = \Phi^T y.$$

但 constrained LS 不能只让 gradient 等于 0，因为最优点可能卡在 boundary（边界）上。

例如一维问题：

$$\min_{\theta} (\theta - 3)^2 \quad \text{subject to} \quad 0 \leq \theta \leq 2.$$

没有约束时，最优解是：

$$\theta = 3.$$

但 $\theta = 3$ 不满足 $\theta \leq 2$ 。所以 constrained optimum 是：

$$\theta^* = 2.$$

注意在 $\theta = 2$ ：

$$\frac{d}{d\theta} (\theta - 3)^2 = 2(\theta - 3) = -2 \neq 0.$$

所以 constrained optimum 不一定满足 “gradient = 0”。它满足的是：

no feasible direction can improve the objective.

这就是为什么 constrained least squares 一般不能直接用 ordinary normal equations 闭式求解，而要用 QP solver / active-set / interior-point / projected methods。

9.2.2.3 Cost vs Constraint (目标二次 vs 约束二次)

Roy 口头说 “quadratic constraint” 这里容易误解。Least squares 本身通常是：

quadratic cost

不是 quadratic constraint。

分类上:

Problem	Objective	Constraints	Class
ordinary least squares	quadratic	none	unconstrained QP
bounded least squares	quadratic	linear bounds	QP
constrained least squares with $G\theta \leq h$	quadratic	linear inequalities	QP
least squares with constraint $\ \Phi\theta - y\ _2^2 \leq \rho$	maybe linear/other	quadratic constraint	QCQP/SOCP form

所以记忆:

Least-squares objective is quadratic in theta.
 Adding linear bounds turns OLS into a constrained QP.
 Adding quadratic constraints would move the problem toward QCQP.

9.2.3 Portfolio Optimisation as QP (投资组合优化为什么是 QP)

Roy 提到 portfolio optimisation (投资组合优化) 时, 关键是这句话:

risk can be modelled by variance,
 and variance enters as a quadratic term.

9.2.3.1 Decision Variables (决策变量)

假设有 n 个 assets / stocks (资产 / 股票)。令:

x_i = fraction of money invested in asset i .

把所有权重写成 vector:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}.$$

例如:

x1 = 0.30 -> 30% of money in stock 1
 x2 = 0.20 -> 20% of money in stock 2
 ...

常见 constraints 是 linear constraints:

$$\sum_{i=1}^n x_i = 1, \quad x_i \geq 0.$$

意思是:

- all money is allocated (钱全部分配出去);
- no short selling (不做空);
- 也可以加 upper bounds, 例如 $x_i \leq 0.3$, 表示任何单一股票最多 30%。

这些都是 Roy 说的 polytopic constraints (多胞体约束), 因为它们是 linear equalities / inequalities。

9.2.3.2 Expected Return (预期回报)

令 μ_i 是 asset i 的 expected return (预期收益率)。Portfolio 的 expected return 是加权平均:

$$\mathbb{E}[R_p] = \mu^T \mathbf{x}.$$

这是 linear expression (线性表达式)。

所以如果你想 maximize expected return:

$$\max_{\mathbf{x}} \mu^T \mathbf{x},$$

这是 linear objective。

9.2.3.3 Risk as Variance (用方差衡量风险)

真正让 portfolio optimisation 变成 QP 的, 是 risk (风险)。

令 random return vector (随机收益向量) 为:

$$\mathbf{R} = \begin{bmatrix} R_1 \\ \vdots \\ R_n \end{bmatrix}.$$

Portfolio return 是:

$$R_p = \mathbf{x}^T \mathbf{R}.$$

Risk 通常用 portfolio return 的 variance (方差) 表示:

$$\text{Var}(R_p) = \text{Var}(\mathbf{x}^T \mathbf{R}).$$

如果 Σ 是 asset returns 的 covariance matrix (协方差矩阵):

$$\Sigma = \text{Cov}(\mathbf{R}),$$

那么:

$$\text{Var}(R_p) = \mathbf{x}^T \Sigma \mathbf{x}.$$

这就是 quadratic term (二次项)。所以 Roy 说 variance comes in as a quadratic term。

9.2.3.4 Why Covariance Matters (为什么相关性很重要)

如果两个 stocks 独立, 风险比较容易理解。但现实里很多 returns 不是 independent (独立) 的, 而是 correlated (相关的)。

例如 Roy 的石油公司例子:

oil company A and oil company B may move together
because both are affected by fuel prices or geopolitical events.

数学上, 相关性进入 covariance matrix 的 off-diagonal entries (非对角项):

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{12} & \sigma_2^2 \end{bmatrix}.$$

两资产 portfolio 的 variance 是:

$$\text{Var}(R_p) = x_1^2 \sigma_1^2 + x_2^2 \sigma_2^2 + 2x_1 x_2 \sigma_{12}.$$

最后一项:

$$2x_1 x_2 \sigma_{12}$$

就是 correlation / covariance 带来的 coupling term (耦合项)。

如果 $\sigma_{12} > 0$, 两只股票倾向于一起涨跌, diversification benefit (分散化收益) 较弱。

如果 $\sigma_{12} < 0$, 一只涨时另一只可能跌, 组合风险可能被抵消一部分。

9.2.3.5 Standard Mean-Variance Form (标准均值-方差形式)

一个常见 portfolio QP 是:

$$\begin{aligned} \min_{\mathbf{x}} \quad & \mathbf{x}^T \Sigma \mathbf{x} \\ \text{subject to} \quad & \mu^T \mathbf{x} \geq r_{\min} \\ & \mathbf{1}^T \mathbf{x} = 1 \\ & \mathbf{x} \geq 0. \end{aligned}$$

解释:

Term	Meaning
$\mathbf{x}^T \Sigma \mathbf{x}$	portfolio variance / risk
$\mu^T \mathbf{x} \geq r_{\min}$	expected return at least target
$\mathbf{1}^T \mathbf{x} = 1$	invest all money
$\mathbf{x} \geq 0$	no short selling

这就是 QP:

quadratic objective: portfolio variance
linear constraints: return target, budget, bounds

也可以写成 risk-return tradeoff (风险-收益权衡):

$$\min_{\mathbf{x}} \quad \mathbf{x}^T \Sigma \mathbf{x} - \lambda \mu^T \mathbf{x} \quad \text{subject to} \quad \mathbf{1}^T \mathbf{x} = 1, \mathbf{x} \geq 0.$$

这里 λ 控制你有多重视 return:

- larger λ : 更追求 expected return;
- smaller λ : 更保守, 更重视 low variance。

9.2.3.6 Engineering Reading (工程理解)

Portfolio optimisation 的核心不是“预测哪只股票一定涨”, 而是:

allocate weights under constraints
while balancing expected return and correlated risk.

这和 Roy 的其他例子很像:

Optimisation example	Decision variables	Constraints	Quadratic part
constrained LS	parameters θ	bounds / linear constraints	squared residual
portfolio optimisation	investment weights $\mathbf{x}$	budget / no-short / bounds	return variance
MPC	future control inputs	dynamics / actuator limits	tracking error + control effort

一句话记忆:

Portfolio QP = choose investment weights;
 risk is variance $\mathbf{x}^T \Sigma \mathbf{x}$;
 budget and allocation limits are linear constraints.

9.3 Second-order Cone Problem (二阶锥规划, SOCP)

SOCP 就是 optimisation problem 里允许出现 norm cone / second-order cone (范数锥 / 二阶锥) 约束。

SOCP 典型 constraint (约束):

$$\|A_i x + b_i\|_2 \leq c_i^T x + d_i$$

它和前面的 basic norm cone:

$$\|\mathbf{x}\|_2 \leq t$$

是同一结构, 只是把 $\mathbf{x}$ 换成 affine expression (仿射表达式) $A_i x + b_i$, 把 t 换成 affine scalar expression (仿射标量表达式) $c_i^T x + d_i$ 。

特征:

- linear objective (线性目标) 常见;
- feasible region (可行域) 是 second-order cones (二阶锥) 的 intersection (交集);
- QP 有时可以 reformulate (重写) 成 SOCP。

9.4 QCQP (二次约束二次规划)

QCQP:

$$\begin{aligned} \min_x \quad & \frac{1}{2}x^T P_0 x + q_0^T x + r_0 \\ \text{subject to} \quad & \frac{1}{2}x^T P_i x + q_i^T x + r_i \leq 0 \end{aligned}$$

如果 P_i 满足相应 convexity (凸性) 条件, feasible region (可行域) 可以理解为 ellipsoids (椭球) 的 intersection (交集)。

9.5 SDP and LMI (半定规划与线性矩阵不等式)

Semidefinite program / SDP (半定规划) 的核心是 matrix-valued affine constraint (矩阵值仿射约束):

$$x_1 F_1 + \cdots + x_N F_N + G \preceq 0$$

这就是 Linear Matrix Inequality / LMI (线性矩阵不等式)。

考试分类技巧:

linear cost + linear constraints	-> LP (线性规划)
quadratic cost + linear constraints	-> QP (二次规划)
norm cone constraints	-> SOCP (二阶锥规划)
quadratic constraints	-> QCQP (二次约束二次规划)
PSD matrix constraints	-> SDP/LMI (半定规划 / 线性矩阵不等式)
finite horizon dynamic optimisation	-> MPC (模型预测控制), often QP

10 LMI and Schur Complement

10.1 Epigraph Trick: 为什么 “linear cost is universal”

Roy 在 Lec 9 反复用一个小技巧: 几乎所有 problem class 写出来都是 **linear cost** (线性目标), 即使你真正关心的 cost $J(x)$ 是 quadratic 甚至更怪。原因是任何 cost 都能用一个辅助变量 γ 搬进约束里:

$$\min_x J(x) \iff \min_{x, \gamma} \gamma \quad \text{subject to} \quad J(x) \leq \gamma$$

直觉: γ 是 cost 的”上界”, 最小化 γ 就是把这个上界压到 $J(x)$ 的最小值。这叫 **epigraph form** (上图形式, epigraph = 函数图像上方的区域)。

为什么重要:

- solver (尤其商业 solver) 常假设 **linear cost**。epigraph trick 让你把任意 convex cost 塞进它能处理的形式。

- 它把”最小化一个复杂 cost”变成”最小化 γ + 一条约束 $J(x) \leq \gamma$ “。如果那条约束本身能写成 LMI, 整个问题就是 SDP/LMI。
- 后面 minimise spectral norm、minimise LQR cost 全都用这个套路: $\min \quad \text{s.t.}$ (某个 LMI in x and)。

记忆钩子:

```
Want to minimise a hard cost J(x)?
-> minimise gamma, subject to J(x) <= gamma.
-> if "J(x) <= gamma" is an LMI, you have an SDP.
```

10.2 What Is an LMI? (什么是线性矩阵不等式)

10.2.1 Generalized Inequality 与 Cone 视角

Lec 9 把 inequality (不等式) 推广了: 约束不再只是”标量函数 ≤ 0 “, 而可以是 **向量值或矩阵值**, 并用一个 cone (锥) K 表达:

$$f(x) \preceq_K y \iff y - f(x) \in K$$

也可以写成:

$$y - f(x) \succeq_K 0.$$

这里的 “ $\succeq_K 0$ ” 就是 cone-relative nonnegative (相对于锥 K 的非负):

```
greater than zero with respect to K
= belongs to K
```

两个最有用的 cone:

- **positive orthant (正卦限)** $K = \mathbb{R}_+^N$: $x \in K \iff x_i \geq 0$, 逐元素非负。
- **positive semidefinite cone (正半定锥)** $S_+^N = \{X \in S^N \mid X \succeq 0\}$: X 是 symmetric 且所有 eigenvalue (特征值) ≥ 0 。

关键区别: matrix 在 PSD cone 里, **不是**”每个元素 ≥ 0 “, 而是”所有特征值 ≥ 0 “。这比 element-wise (逐元素) 强大得多, 也是 LMI 表达力的来源。

10.2.2 LMI 定义

semidefinite program (SDP, 半定规划) 的核心约束就是 LMI:

$$\min_x c^T x \quad \text{subject to} \quad x_1 F_1 + \cdots + x_N F_N + G \preceq 0, \quad Ax = b$$

其中 F_i, G 是给定的 symmetric matrices。约束

$$F(x) = F_0 + x_1 F_1 + \cdots + x_n F_n \succeq 0$$

里，矩阵 $F(x)$ 对变量 x 是 **affine (仿射)** 的：变量只以 $x_i F_i$ 这种 linear 方式进入矩阵。

它重要是因为很多看起来 **nonlinear (非线性)** 的 constraint 可以写成这种 PSD matrix constraint (正半定矩阵约束) —— LMI 把困难的 scalar nonlinear inequalities 转成 convex matrix inequality, 而 convex 就意味着 solver 可靠、local optimum 即 global optimum。Roy 的原话大意：如果你是 control engineer, 你学过的几乎所有控制理论都能写成 LMI。

10.2.3 Multiple LMIs $\rightarrow$ 堆成对角块

如果有多条 LMI 要同时满足，可以堆到一个 block-diagonal (分块对角) 矩阵的对角线上：

$$F_i(x) \preceq 0, \quad i = 1, \dots, N \quad \Leftrightarrow \quad \begin{bmatrix} F_1(x) & & \\ & \ddots & \\ & & F_N(x) \end{bmatrix} \preceq 0$$

因为 block-diagonal 矩阵的 eigenvalue 就是各块 eigenvalue 的并集。**概念上**它是一条大约束；**实践中** solver 会把它拆开分别处理 (大矩阵里全是零，单独算更快)。所以”一条 LMI”和”多条 LMI”没有本质区别。

10.3 Block Matrix Basics for LMI (矩阵分块基础)

LMI 里大量出现这种写法：

$$\begin{bmatrix} Q & S \\ S^T & R \end{bmatrix}, \quad \begin{bmatrix} -P & A^T P \\ P A & -P \end{bmatrix}, \quad \begin{bmatrix} tI & y \\ y^T & t \end{bmatrix}.$$

这些不是新的矩阵类型，只是把一个大矩阵按行列切成 smaller matrices (小矩阵块)。这样做的目的：

large matrix algebra becomes ordinary algebra on blocks,
as long as dimensions match.

10.3.1 1. Block Dimensions Must Match (先看维度)

假设

$$Q \in \mathbb{R}^{m \times m}, \quad R \in \mathbb{R}^{n \times n}, \quad S \in \mathbb{R}^{m \times n}.$$

那么

$$\begin{bmatrix} Q & S \\ S^T & R \end{bmatrix}$$

整体是 $(m+n) \times (m+n)$ 。为什么 S^T 在左下角？因为左下角必须是 $n \times m$ ，这样整体矩阵才 symmetric。

具体例子：如果 $x \in \mathbb{R}^2$ ，quadratic LMI

$$\begin{bmatrix} c + b^T x & x^T \\ x & -Q^{-1} \end{bmatrix} \preceq 0$$

就是 3×3 ：

Block	Size
$c + b^T x$	1×1
x^T	1×2
x	2×1
$-Q^{-1}$	2×2

所以分块第一步永远是问：

each block has what size?

can these blocks sit next to each other?

10.3.2 2. Block Addition (分块加法)

同样分块方式的矩阵可以逐块相加：

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} + \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} A+E & B+F \\ C+G & D+H \end{bmatrix}.$$

前提：对应 block 尺寸一致。

10.3.3 3. Block Transpose (分块转置)

转置会做两件事：

1. 每个 block 自己转置；

2. block 的位置沿主对角线交换。

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix}^T = \begin{bmatrix} A^T & C^T \\ B^T & D^T \end{bmatrix}.$$

所以

$$\begin{bmatrix} Q & S \\ S^T & R \end{bmatrix}^T = \begin{bmatrix} Q^T & S \\ S^T & R^T \end{bmatrix}.$$

如果 $Q = Q^T$ 且 $R = R^T$, 整体就是 symmetric。注意 S 本身不需要 symmetric, 因为它可能甚至不是 square matrix。

10.3.4 4. Block Multiplication (分块乘法)

分块乘法和普通矩阵乘法一样, 只是把 scalar multiplication 换成 matrix multiplication:

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{bmatrix}.$$

每一项都必须维度匹配。比如 AE 能乘, BG 也能乘, 而且 AE 和 BG 的结果尺寸要一样, 才能相加。

这个公式就是后面 congruence transformation 手算的基础。

10.3.5 5. Block Diagonal Matrix (分块对角矩阵)

如果非对角 block 都是 zero:

$$\begin{bmatrix} A & 0 \\ 0 & D \end{bmatrix},$$

这叫 block diagonal matrix。它的好处是很多性质可以逐块看:

$$\begin{bmatrix} A & 0 \\ 0 & D \end{bmatrix} \succeq 0 \iff A \succeq 0, \quad D \succeq 0.$$

多个 LMI 堆成一条大 LMI, 就是利用这个性质:

$$F_1(x) \preceq 0, \quad F_2(x) \preceq 0 \iff \begin{bmatrix} F_1(x) & 0 \\ 0 & F_2(x) \end{bmatrix} \preceq 0.$$

10.3.6 6. Multiplying by a Block Diagonal Matrix

LMI 里经常乘

$$T = \begin{bmatrix} I & 0 \\ 0 & P \end{bmatrix}.$$

它的作用很直观：左上 block 不变，右下方向被 P 缩放。

例如：

$$\begin{bmatrix} -P & A^T \\ A & -P^{-1} \end{bmatrix} \begin{bmatrix} I & 0 \\ 0 & P \end{bmatrix} = \begin{bmatrix} -P & A^T P \\ A & -I \end{bmatrix}.$$

再从左边乘一次同样的 block diagonal matrix：

$$\begin{bmatrix} I & 0 \\ 0 & P \end{bmatrix} \begin{bmatrix} -P & A^T P \\ A & -I \end{bmatrix} = \begin{bmatrix} -P & A^T P \\ PA & -P \end{bmatrix}.$$

这就是 Roy 在 congruence transformation 里做的逐块运算。

10.3.7 7. Block Matrix Quadratic Form

如果

$$M = \begin{bmatrix} Q & S \\ S^T & R \end{bmatrix}, \quad z = \begin{bmatrix} u \\ v \end{bmatrix},$$

那么

$$z^T M z = \begin{bmatrix} u^T & v^T \end{bmatrix} \begin{bmatrix} Q & S \\ S^T & R \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = u^T Q u + 2u^T S v + v^T R v.$$

这解释了为什么 off-diagonal block 会产生 cross term（交叉项）。标量二次函数里 bt 要写成两个 $\frac{b}{2}t$ ，本质就是这个 $2u^T S v$ 。

10.3.8 8. Why Schur Complement Is a Block Operation

Schur complement 从

$$\begin{bmatrix} Q & S \\ S^T & R \end{bmatrix}$$

里”消去”右下角 block R ，得到：

$$Q - SR^{-1}S^T.$$

所以它天然依赖 block partitioning。你必须先决定：

```
which block is Q?
which block is S?
which block is R?
which block will be inverted/eliminated?
```

这也是为什么 LMI 题常常先考你会不会把矩阵正确分块。

10.3.9 9. Common Mistakes (常见错误)

1. Dimension mismatch (维度不匹配)

比如 S 是 $m \times n$ ，那么左下角必须是 S^T ，尺寸 $n \times m$ 。

2. Forgetting block transpose (忘记分块转置)

$\begin{bmatrix} A & B \\ C & D \end{bmatrix}^T$ 不是 $\begin{bmatrix} A^T & B^T \\ C^T & D^T \end{bmatrix}$ ，而是 $\begin{bmatrix} A^T & C^T \\ B^T & D^T \end{bmatrix}$ 。

3. Treating blocks as scalars when multiplication order matters (把矩阵块当标量)

一般 $AB \neq BA$ 。所以 $A^T P$ 和 PA 不是同一个东西。

4. Ignoring symmetry (忽略对称性)

LMI/PSD 约束里的矩阵必须 symmetric。写 block matrix 时要检查右上和左下是否互为 transpose。

5. Confusing block diagonal with diagonal matrix (分块对角不等于普通对角)

block diagonal 的每个 block 本身可以是矩阵，不必是 scalar。

10.4 Schur Complement (舒尔补)

Schur complement 是把 LMI 里”看起来二次 / 带逆”的东西线性化的 **核心技巧**，考试出现频率很高。

典型形式 (symmetric block matrix, Q, R symmetric, S 不必):

$$\begin{bmatrix} Q & S \\ S^T & R \end{bmatrix} \succ 0 \iff R \succ 0, \quad Q - SR^{-1}S^T \succ 0$$

而且对 **affine combinations (仿射组合)** 也成立：如果 Q, S, R 都是优化变量 x 的仿射函数，

$$\begin{bmatrix} Q(x) & S(x) \\ S(x)^T & R(x) \end{bmatrix} \succ 0 \iff R(x) \succ 0, \quad Q(x) - S(x)R(x)^{-1}S(x)^T \succ 0$$

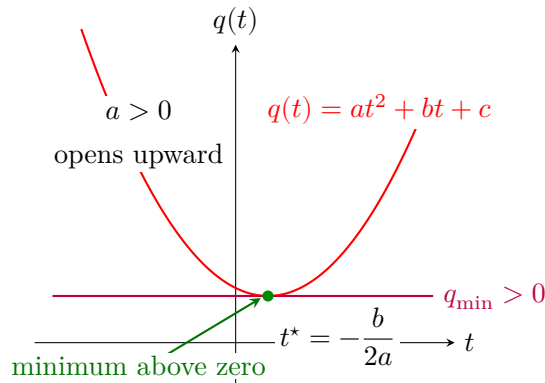
注意右边 $Q - SR^{-1}S^T$ 对 x 是 **非线性**的（有乘积、有逆），但左边的 block matrix 对 x 是 **线性**的。所以 Schur complement 让我们把一条非线性约束”升维”成一条线性 matrix 约束——这正是能交给 SDP solver 的形式。

10.4.1 标量直觉：它到底在算什么

Roy 用一个一维例子解释 Schur complement 的本质。考虑标量二次函数

$$q(t) = at^2 + bt + c, \quad a > 0$$

问：它对所有 t 都 ≥ 0 吗？Roy 画的就是下面这种图：开口向上的 convex quadratic（凸二次函数），但它是否处处非负，取决于最低点有没有掉到 0 以下。



做法是求最小值： $q'(t) = 2at + b = 0 \Rightarrow t^* = -\frac{b}{2a}$ ，代回得

$$q(t^*) = c - \frac{b^2}{4a}$$

所以“ $q(t) \geq 0$ 对所有 t ” $\iff a \geq 0$ 且 $c - \frac{b^2}{4a} \geq 0$ 。

把它写成 block matrix ($Q = c$, $S = \frac{b}{2}$, $R = a$):

$$\begin{bmatrix} c & \frac{b}{2} \\ \frac{b}{2} & a \end{bmatrix} \succeq 0 \iff a \geq 0, \quad c - \frac{b}{2} a^{-1} \frac{b}{2} \geq 0$$

为什么这里是 $S = \frac{b}{2}$ ，不是 b ？因为 block matrix 对应的是 quadratic form：

$$\begin{bmatrix} 1 & t \end{bmatrix} \begin{bmatrix} c & \frac{b}{2} \\ \frac{b}{2} & a \end{bmatrix} \begin{bmatrix} 1 \\ t \end{bmatrix} = c + 2 \left(\frac{b}{2} \right) t + at^2 = at^2 + bt + c$$

交叉项会出现两次：左上到右下一次，右下到左上一次。所以 bt 在矩阵里要拆成两个 $\frac{b}{2}t$ 。

右边正是”求导、找最小、代回”得到的同一个判据。**Schur complement 就是”先对一部分变量做最优消去”的矩阵版**： $Q - SR^{-1}S^T$ 是把 R 那部分变量最优化掉之后剩下的 effective term（有效项）。把这个标量论证推广到矩阵 / 向量，就得到一般 Schur complement。

10.4.2 How to Map Q, R, S into a QP Constraint

这里最容易混淆的是 notation（记号）。Roy 在 Schur complement 里写的

$$\begin{bmatrix} Q & S \\ S^T & R \end{bmatrix}$$

只是 block names（分块名字）。但 QP 里很多教材也用 Q 表示 Hessian / quadratic-cost matrix。这两个 Q 不一定是同一个东西。为了减少混淆，下面把 QP 的二次矩阵写成 H 。

实用判断法：

Schur block	怎么找	直觉
R	放你想通过 R^{-1} 产生的 curvature / Hessian block	被消去变量的二次项
S	放 cross term（交叉项）的一半或对应线性表达式	因为 S 会在 $SR^{-1}S^T$ 里平方起来
Q	放剩下的 constant / affine part	Schur 后留下 $Q - SR^{-1}S^T$

标量例子：

$$at^2 + bt + c \geq 0 \quad \text{for all } t$$

对应

$$R = a, \quad S = \frac{b}{2}, \quad Q = c$$

因为 Schur complement 给出

$$Q - SR^{-1}S^T = c - \frac{b}{2}a^{-1}\frac{b}{2} = c - \frac{b^2}{4a}$$

这正是凸二次函数的 minimum value（最小值）。

QP constraint 里常见的是

$$x^T H x + b^T x + c \leq 0, \quad H \succ 0$$

要把它写成 LMI，先把它改写成

$$-(x^T H x + b^T x + c) \geq 0$$

现在我们希望 Schur complement 产生 $-x^T H x$ 。选择

$$R = H^{-1}, \quad S = x^T, \quad Q = -(b^T x + c)$$

则

$$Q - SR^{-1}S^T = -(b^T x + c) - x^T H x \geq 0$$

所以得到一条 LMI:

$$\begin{bmatrix} -(b^T x + c) & x^T \\ x & H^{-1} \end{bmatrix} \succeq 0$$

等价地，也可以写成 guide 后面表格里的 negative semidefinite 形式:

$$\begin{bmatrix} c + b^T x & x^T \\ x & -H^{-1} \end{bmatrix} \preceq 0$$

这里 off-diagonal 写 x 或 $-x$ 都会给出同一个二次项，因为 Schur complement 里它以 $S(\cdot)^{-1}S^T$ 的形式成对出现。

一句话记忆: **先展开 block matrix 的 quadratic form, 看交叉项有没有少一半; 再做 Schur complement, 检查 $Q - SR^{-1}S^T$ 是否正好还原原来的二次不等式。**

重要提醒: 上面的“ Q 放剩下的 constant / affine part”是针对 **scalar quadratic constraint** 的读法。更一般地, Q, S, R 是你构造的 block matrix 的三个块; 它们的角色要由

$$Q - SR^{-1}S^T$$

能否还原目标 inequality 来判断。也就是说, **不要先问原式里有没有常数项; 要先问 Schur 后想得到什么式子。** Spectral norm 例子就是这种情况。

考试常用场景: 把 norm constraint、quadratic inequality 写成 LMI; 处理 stability 和 controller synthesis 里的 matrix inequalities。

易错点: 用 Schur complement 必须先确认被消去那块 (R 或 Q) 的 definiteness, 且整个 block 必须 symmetric。

10.5 Worked LMI #1: Minimise the Spectral Norm (最小化最大奇异值)

矩阵的 2-norm (spectral norm, 谱范数) $\|A\|_2$ 就是最大 singular value (最大奇异值), 也就是矩阵作为线性映射的“最大增益”。控制里它常代表 disturbance $\rightarrow$ error 的最坏放大 (例如风力 $\rightarrow$ 自动驾驶转向误差), 我们想把它压小。

问题: $\min_x \|A(x)\|_2$, 其中 $A(x)$ 对 x 仿射。

用 epigraph trick 引入 γ : 约束 $\|A\|_2 < \gamma$ 。这里的意思是:

$$\text{largest singular value of } A(x) < \gamma$$

因为 singular values 的平方是 $A(x)A(x)^T$ 的 eigenvalues, 所以

$$\|A(x)\|_2 < \gamma \iff \lambda_{\max}(A(x)A(x)^T) < \gamma^2 \iff \gamma^2 I - A(x)A(x)^T \succ 0$$

这个式子还是有 $A(x)A(x)^T$, 对 x 是 quadratic / nonlinear。Schur complement 的目的, 就是把它改写成一个更大的 **linear matrix inequality**:

$$\begin{bmatrix} \gamma I & A(x) \\ A(x)^T & \gamma I \end{bmatrix} \succ 0$$

这时按 Schur complement 的 block notation:

$$Q = \gamma I, \quad S = A(x), \quad R = \gamma I$$

注意: 这里 Q 不是 0。因为 Q 是 **block matrix** 的左上角, 而我们主动把左上角设计成 γI 。为什么要这样设计? 因为 Schur complement 会给出

$$Q - SR^{-1}S^T = \gamma I - A(x)(\gamma I)^{-1}A(x)^T = \gamma I - \frac{1}{\gamma}A(x)A(x)^T$$

再加上 $R = \gamma I \succ 0$, 也就是 $\gamma > 0$, 两边乘以正数 γ :

$$\gamma \left(\gamma I - \frac{1}{\gamma}A(x)A(x)^T \right) = \gamma^2 I - A(x)A(x)^T \succ 0$$

这就回到了 $\|A(x)\|_2 < \gamma$ 。

这里确实有“cross term” (交叉项), 只是它不是 x 和常数的交叉项, 而是 block matrix quadratic form 里的两个 test vectors 之间的交叉项:

$$\begin{bmatrix} u \\ v \end{bmatrix}^T \begin{bmatrix} \gamma I & A \\ A^T & \gamma I \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \gamma \|u\|_2^2 + 2u^T A v + \gamma \|v\|_2^2$$

中间的 $2u^T A v$ 就来自 off-diagonal block $S = A$ 和 $S^T = A^T$ 。Schur complement 把这个交叉项”消去”，留下 AA^T 。

一维数值例子最清楚。令 $A = [a]$ 是一个标量，则 $\|A\|_2 = |a|$ 。约束 $|a| < \gamma$ 可以写成：

$$\begin{bmatrix} \gamma & a \\ a & \gamma \end{bmatrix} \succ 0$$

对右下角 $R = \gamma$ 做 Schur complement：

$$\gamma > 0, \quad \gamma - a\gamma^{-1}a > 0 \iff \gamma^2 - a^2 > 0 \iff |a| < \gamma$$

例如 $a = 3, \gamma = 5$ ：

$$\begin{bmatrix} 5 & 3 \\ 3 & 5 \end{bmatrix}$$

的 eigenvalues 是 8 和 2，所以 positive definite；这对应 $|3| < 5$ 。

如果 $a = 6, \gamma = 5$ ，矩阵

$$\begin{bmatrix} 5 & 6 \\ 6 & 5 \end{bmatrix}$$

的 eigenvalues 是 11 和 -1 ，不是 positive definite；这对应 $|6| < 5$ 不成立。

所以等价的 LMI 问题是：

$$\min_{x, \gamma} \gamma \quad \text{subject to} \quad \begin{bmatrix} \gamma I & A(x) \\ A(x)^T & \gamma I \end{bmatrix} \succ 0$$

这个约束对 x 和 γ 都 **线性**。解出来的 γ 就是最小的最大奇异值，对应的 x 就是达成它的设计。

10.6 Worked LMI #2: Point Inside an Ellipsoid (点在椭圆内)

椭圆 $E = \{x \mid (x - x_c)^T P^{-1} (x - x_c) \leq 1\}$, $P \succ 0$ 。判断 $x \in E$ 看似要算 P^{-1} 的二次型，但用 Schur complement 就是一条 LMI：

$$x \in E \iff \begin{bmatrix} 1 & (x_c - x)^T \\ (x_c - x) & P \end{bmatrix} \succeq 0$$

(对右下块 $P \succ 0$ 做 Schur: top-left $1 - (x_c - x)^T P^{-1} (x_c - x) \geq 0$, 正是椭球定义。)

威力在于: 这条约束对 x 仿射, **同时对 center x_c 和 shape P 也仿射**。所以”给定若干点, 找一个包住它们的椭球”——优化 x_c, P ——就是一个 SDP。这正是 convex exercise 里 minimum-volume ellipsoid 那类题的内核。

10.7 What Does “Linear in x and γ ” Mean? (矩阵里怎么看 linear)

Roy 多次说:

```
this constraint is linear in x and linear in gamma
it is linear in the centers and linear in the scaling P
```

这里的 linear 不是说 Schur complement 展开后的式子是线性的。展开后经常会出现:

$$\gamma^2 I - A(x)A(x)^T, \quad 1 - (x - x_c)^T P^{-1} (x - x_c)$$

这些明显不是 linear。Roy 的意思是: **看 LMI 左边那个 matrix 的每一个 entry (矩阵元素) 是不是 decision variables 的 affine expression (仿射表达式)**。

Affine / linear in optimisation language 通常允许:

$$\text{constant} + \alpha_1 z_1 + \alpha_2 z_2 + \cdots + \alpha_k z_k$$

不允许:

$$z_1 z_2, \quad z_1^2, \quad \frac{1}{z_1}, \quad P^{-1}.$$

所以判断步骤是:

1. 写出 LMI matrix。
2. 把每个 block 展开成 matrix entries。
3. 看每个 entry 是否只是变量的一次项加常数。
4. 如果没有变量乘变量、平方、倒数、矩阵逆, 就叫 affine / linear in those variables。

10.7.1 Example 1: Spectral Norm LMI

$$\begin{bmatrix} \gamma I & A(x) \\ A(x)^T & \gamma I \end{bmatrix} \succ 0$$

如果

$$A(x) = A_0 + x_1 A_1 + x_2 A_2 + \cdots + x_k A_k,$$

那么 $A(x)$ 的每个元素都是:

$$a_{ij}(x) = a_{ij,0} + x_1 a_{ij,1} + x_2 a_{ij,2} + \cdots + x_k a_{ij,k}.$$

这是 affine in x 。而 γI 的元素是:

$$(\gamma I)_{ij} = \begin{cases} \gamma, & i = j, \\ 0, & i \neq j. \end{cases}$$

这也是 affine in γ 。所以整个 block matrix 对 x 和 γ 都是 affine。

注意对比 Schur 展开后的式子:

$$\gamma^2 I - A(x)A(x)^T \succ 0$$

这里有 γ^2 和 $A(x)A(x)^T$, 所以它不是 linear。**Schur complement** 的价值正是: 把展开后非线性的约束, 改写成 **entry-wise affine** 的 **LMI**。

一个具体标量例子:

$$A(x) = 2 + x, \quad \gamma \text{ is a variable.}$$

原来的 norm constraint 是:

$$|2 + x| < \gamma$$

平方后是:

$$\gamma^2 - (2 + x)^2 > 0$$

这对 x, γ 都不是 linear。但 LMI 写成:

$$\begin{bmatrix} \gamma & 2+x \\ 2+x & \gamma \end{bmatrix} \succ 0$$

矩阵元素分别是 γ 、 $2+x$ 、 $2+x$ 、 γ ，每个都是变量的一次表达式。所以这是 linear in x and γ 。

10.7.2 Example 2: Point Inside an Ellipsoid

$$\begin{bmatrix} 1 & (x_c - x)^T \\ (x_c - x) & P \end{bmatrix} \succeq 0$$

如果 x 是变量， x_c 也是变量，那么 entry 是：

$$(x_c - x)_i = x_{c,i} - x_i$$

这是 affine in both x and x_c 。如果 P 也是变量，那么右下角每个元素就是 P_{ij} 本身，也是 linear in P_{ij} 。

所以 Roy 说它 linear in x 、linear in the centers、linear in scaling P ，意思是：

the LMI matrix entries contain x , x_c , and P only to first order.

对比原始 ellipsoid condition:

$$(x - x_c)^T P^{-1} (x - x_c) \leq 1$$

这里同时有 P^{-1} 和 $(x - x_c)$ 的二次型。如果直接看它，就很难说它对 x_c 或 P 是 linear。Schur complement 把它换成 LMI 后，solver 看到的是一个 entry-wise affine matrix。

一句话记忆：

In LMI, "linear" means the matrix entries are affine in the decision variables.

Do not judge linearity after expanding the Schur complement back into products/inverses.

10.8 Generality of SDP: 把别的约束都写成 LMI

Roy 给了一张“翻译表”，说明前面的 problem class 都能塞进 LMI（这也是为什么我们集中精力在 LMI 上）：

Constraint	等价 LMI
linear $Ax \leq b$	$\text{diag}(Ax) \preceq \text{diag}(b)$ （放到对角线）
convex quadratic $x^T Q x + b^T x + c \leq 0$, $Q \succ 0$	$\begin{bmatrix} c + b^T x & x^T \\ x & -Q^{-1} \end{bmatrix} \preceq 0$

Constraint	等价 LMI
second-order cone $\ Ax + b\ _2 \leq c^T x + d$	$\begin{bmatrix} (c^T x + d)I & Ax + b \\ (Ax + b)^T & c^T x + d \end{bmatrix} \succeq 0$

记忆: **LP / QP / SOCP / QCQP 都是 SDP 的特例**, 反过来不一定。所以分类题里如果一个约束能写成“仿射矩阵表达式 $\succeq 0$ ”, 它就是 LMI / SDP。

10.8.1 Practice: Convert Standard Constraints to LMIs (考试风格练习)

这张 slide 不适合死记。考试更可能给你一个具体约束, 让你把它改写成 LMI。下面题目先给 constraint, 答案放后面; 建议先自己写 block matrix, 再核对。

10.8.1.1 Practice A: Linear Constraints

A1. Let $x = (x_1, x_2)^T$. Write the following linear inequalities as one diagonal LMI:

$$2x_1 - x_2 \leq 3, \quad -x_1 + 4x_2 \leq 5.$$

A2. Write the box constraints as a diagonal LMI:

$$-1 \leq x_1 \leq 2, \quad x_2 \leq 4.$$

答案:

For A1,

$$\begin{bmatrix} 2x_1 - x_2 & 0 \\ 0 & -x_1 + 4x_2 \end{bmatrix} \preceq \begin{bmatrix} 3 & 0 \\ 0 & 5 \end{bmatrix}$$

也可以写成:

$$\begin{bmatrix} 3 - (2x_1 - x_2) & 0 \\ 0 & 5 - (-x_1 + 4x_2) \end{bmatrix} \succeq 0.$$

For A2, 先把 lower bound 改成 upper-bound style:

$$-1 \leq x_1 \iff -x_1 \leq 1.$$

所以:

$$\begin{bmatrix} x_1 & 0 & 0 \\ 0 & -x_1 & 0 \\ 0 & 0 & x_2 \end{bmatrix} \preceq \begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 4 \end{bmatrix}.$$

10.8.1.2 Practice B: Quadratic Constraints

Use the slide formula:

$$x^T Q x + b^T x + c \leq 0, \quad Q \succ 0 \quad \Longleftrightarrow \quad \begin{bmatrix} c + b^T x & x^T \\ x & -Q^{-1} \end{bmatrix} \preceq 0.$$

B1. Convert the scalar quadratic constraint to an LMI:

$$2x^2 - 3x - 1 \leq 0.$$

B2. Let $x = (x_1, x_2)^T$. Convert the following quadratic constraint to an LMI:

$$x^T \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} x + \begin{bmatrix} -4 & 2 \end{bmatrix} x + 1 \leq 0.$$

答案:

For B1,

$$Q = 2, \quad b = -3, \quad c = -1, \quad Q^{-1} = \frac{1}{2}.$$

所以:

$$\begin{bmatrix} -1 - 3x & x \\ x & -\frac{1}{2} \end{bmatrix} \preceq 0.$$

For B2,

第一步, 先把题目对齐到标准形式:

$$x^T Q x + b^T x + c \leq 0.$$

题目里:

$$Q = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \quad Q^{-1} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & 1 \end{bmatrix}, \quad b^T x = -4x_1 + 2x_2.$$

这里 x 是 2-vector, 所以 Schur LMI 会是 $1 + 2 = 3$ 维的 block matrix:

$$\begin{bmatrix} \text{scalar block} & \text{row block} \\ \text{column block} & \text{matrix block} \end{bmatrix}$$

按照公式:

$$\begin{bmatrix} c + b^T x & x^T \\ x & -Q^{-1} \end{bmatrix} \preceq 0.$$

逐块填进去:

$$c + b^T x = 1 - 4x_1 + 2x_2, \quad x^T = \begin{bmatrix} x_1 & x_2 \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix},$$

and

$$-Q^{-1} = -\begin{bmatrix} \frac{1}{2} & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} -\frac{1}{2} & 0 \\ 0 & -1 \end{bmatrix}.$$

所以最终 LMI 是:

$$\begin{bmatrix} 1 - 4x_1 + 2x_2 & x_1 & x_2 \\ x_1 & -\frac{1}{2} & 0 \\ x_2 & 0 & -1 \end{bmatrix} \preceq 0.$$

为什么这等价? 对右下角 $-Q^{-1} \prec 0$ 做 Schur complement。对 negative definite block 的版本, 条件等价于:

$$c + b^T x - x^T (-Q^{-1})^{-1} x \leq 0.$$

因为

$$(-Q^{-1})^{-1} = -Q,$$

所以:

$$c + b^T x - x^T (-Q)x = c + b^T x + x^T Qx \leq 0.$$

这正好回到原来的 quadratic constraint。

10.8.1.3 Practice C: Second-Order Cone Constraints

Use the slide formula:

$$\|Ax + b\|_2 \leq c^T x + d.$$

Let

$$y = Ax + b, \quad t = c^T x + d.$$

Then the LMI is:

$$\begin{bmatrix} tI & y \\ y^T & t \end{bmatrix} \succeq 0.$$

Some slides write the negative version with $\preceq 0$; both are equivalent after multiplying the whole matrix by -1 . The important condition is $t \geq 0$ and $\|y\|_2 \leq t$.

C1. Convert this SOCP constraint to an LMI:

$$\left\| \begin{bmatrix} x_1 + 1 \\ 2x_2 \end{bmatrix} \right\|_2 \leq 3 - x_1.$$

C2. Convert this scalar absolute-value constraint to an LMI:

$$|2 + x| \leq \gamma.$$

答案:

For C1,

$$y = \begin{bmatrix} x_1 + 1 \\ 2x_2 \end{bmatrix}, \quad t = 3 - x_1.$$

Because $y \in \mathbb{R}^2$, tI is 2×2 :

$$\begin{bmatrix} 3 - x_1 & 0 & x_1 + 1 \\ 0 & 3 - x_1 & 2x_2 \\ x_1 + 1 & 2x_2 & 3 - x_1 \end{bmatrix} \succeq 0.$$

For C2, this is the one-dimensional version of SOC:

$$y = 2 + x, \quad t = \gamma.$$

Thus:

$$\begin{bmatrix} \gamma & 2+x \\ 2+x & \gamma \end{bmatrix} \succeq 0.$$

This is the same scalar example used above for the spectral norm: it enforces $\gamma^2 - (2+x)^2 \geq 0$, with $\gamma \geq 0$.

10.9 CVX 实务 (disciplined convex)

CVX (MATLAB / Python 都有, Python 版是 `cvxpy`) 让你几乎照着数学写 LMI。Roy 给的最小距离例子 (两椭圆间最短距离):

```
cvx_begin sdp
    variable x1(n), x2(n)
    minimize( norm(x1 - x2) )
    subject to
        [1, (xc1 - x1)'];
        (xc1 - x1), P1] >= 0
        [1, (xc2 - x2)'];
        (xc2 - x2), P2] >= 0
cvx_end
```

要点:

- **sdp flag 很关键**: 它让 `>=` 被解释成“矩阵在 PSD cone 里”, 而不是 MATLAB 默认的 **element-by-element (逐元素)**。忘了它, 矩阵约束含义就错了。
- CVX 只接受 **disciplined convex**: 它有一套语法规则检查你写的东西能否被证明 convex (convex 的单调函数、affine 等)。覆盖范围很大, 但确实有些 convex 问题它的语法写不出来。
- **别随便平方**: CVX 建议保留 norm / conic 形式, 不要写成 squared。原因 (转录里 Roy 解释): solver 内部用 conic / cutting-plane 方法, norm 在零点的不可微对它没影响, 反而平方有时数值更差。只有当你要手算 gradient / normal equations / Lagrangian 时, 平方才是“对的写法”。

10.10 Stability LMIs (稳定性线性矩阵不等式)

控制理论里“系统稳定”可以直接写成 LMI 的 feasibility problem (可行性问题): 找不到 $P \succ 0$ 满足某条 LMI, 就不稳定。

10.10.1 Background: What Is a Lyapunov Equation? (无控制背景版)

控制里最基本的 linear system (线性系统) 可以写成:

$$\dot{x}(t) = Ax(t)$$

这里:

- $x(t)$ 是 state vector (状态向量), 例如位置、速度、温度、电压等;
- $\dot{x}(t)$ 是 state 的变化率;
- A 描述系统自身如何演化;
- 没有 input 时, 这叫 autonomous system (自治系统)。

稳定性想问的是:

If the state is slightly away from zero, will it naturally decay back to zero?

如果会回到零, 系统稳定; 如果越跑越大, 系统不稳定。

Lyapunov 的想法很像 energy method (能量法)。不要直接解微分方程, 而是构造一个像”能量”一样的函数:

$$V(x) = x^T P x, \quad P \succ 0.$$

为什么要求 $P \succ 0$? 因为这样 $V(x) > 0$ for $x \neq 0$, 而且 $V(0) = 0$ 。也就是说, $V(x)$ 可以像 energy / distance-to-zero 一样衡量 state 离零有多远。

稳定的直觉是:

energy should keep decreasing along the system trajectory.

对 continuous-time 系统 $\dot{x} = Ax$, 沿轨迹求导:

$$\begin{aligned} \dot{V}(x) &= \frac{d}{dt}(x^T P x) \\ &= \dot{x}^T P x + x^T P \dot{x} \\ &= (Ax)^T P x + x^T P (Ax) \\ &= x^T (A^T P + P A) x. \end{aligned}$$

如果

$$A^T P + P A \prec 0,$$

那么对所有 $x \neq 0$ 都有 $\dot{V}(x) < 0$, energy 一直下降, 所以 state 会往零走。这就是 Lyapunov stability condition (Lyapunov 稳定性条件)。

Lyapunov equation 和 Lyapunov inequality 的区别:

Name	Form	Meaning
Lyapunov inequality	$A^T P + P A \prec 0$	找某个 $P \succ 0$ 证明系统稳定
Lyapunov equation	$A^T P + P A = -Q, Q \succ 0$	给定 Q , 求 P ; 若解 $P \succ 0$, 也证明稳定

所以 Roy 说 “write it as equals and you are solving a Lyapunov equation”, 意思是:

Instead of only asking whether $A^T P + P A$ is negative, choose a positive Q and solve $A^T P + P A = -Q$.

最常见选择是 $Q = I$:

$$A^T P + P A = -I.$$

如果这个方程解出来的 $P \succ 0$, 那就说明系统稳定。

一个一维例子:

$$\dot{x} = ax, \quad V(x) = px^2, \quad p > 0.$$

则

$$\dot{V} = 2apx^2.$$

要 $\dot{V} < 0$, 需要 $2ap < 0$. 因为 $p > 0$, 所以必须 $a < 0$. 这正是一维系统 $\dot{x} = ax$ 稳定的条件: $a < 0$ 时 $x(t)$ 会指数衰减。

对应 Lyapunov equation, 如果取 $Q = 1$:

$$2ap = -1 \quad \Rightarrow \quad p = -\frac{1}{2a}.$$

只有当 $a < 0$ 时, $p > 0$. 这和稳定条件完全一致。

Discrete-time (离散时间) 系统是:

$$x_{k+1} = Ax_k.$$

这时不是看 derivative, 而是看下一步 energy 是否变小:

$$V(x_{k+1}) - V(x_k) = x_k^T (A^T P A - P) x_k.$$

所以离散时间的 Lyapunov condition 是:

$$A^T P A - P \prec 0.$$

一句话记忆:

Lyapunov method proves stability by finding a positive energy function $V(x)=x^T P x$ that decreases along the system trajectory.

在本课里, 关键不是控制理论证明本身, 而是: 这些 stability conditions 可以写成 matrix inequalities, 然后用 LMI / SDP solver 去找 P 。

Continuous-time (连续时间) $\dot{x}(t) = Ax(t)$:

$$\text{stable} \iff \exists P \succ 0 : A^T P + P A \prec 0$$

(这是 Lyapunov 不等式; 写成等式 $A^T P + P A = -$ 给定, 就是 Lyapunov equation。)

Discrete-time (离散时间) $x_{k+1} = Ax_k$:

$$\text{stable} \iff \exists P \succ 0 : A^T P A - P \prec 0$$

离散情形里 $A^T P A$ 对 A 是二次的。要把它变成对 P 的 LMI, 用 Schur complement:

$$A^T P A - P \prec 0 \iff \begin{bmatrix} -P & A^T \\ A & -P^{-1} \end{bmatrix} \prec 0 \iff \begin{bmatrix} -P & A^T P \\ P A & -P \end{bmatrix} \prec 0$$

最后一式对 P 是线性的 (对 A 也是线性的 —— 所以”给 A 求 P “或”给 P 求 A “都能当 LMI 解)。

10.10.2 Background: Congruence Transformation (合同变换)

Roy 说的 congruence transformation 是 LMI 推导里非常常用的一步。形式是:

$$M \mapsto T^T M T$$

其中 M 是 symmetric matrix, T 是 invertible matrix (可逆矩阵)。

关键性质:

$$M \succ 0 \iff T^T M T \succ 0$$

同理也保持 $\prec 0, \succeq 0, \preceq 0$ 。为什么? 因为对任意非零向量 z ,

$$z^T (T^T M T) z = (Tz)^T M (Tz).$$

如果 T 可逆, 那么 $z \neq 0 \Rightarrow Tz \neq 0$ 。所以 M 对所有非零向量给正值, $T^T M T$ 也对所有非零向量给正值。负定同理。

一维直觉更简单: 如果 $m < 0$, 且 $t \neq 0$, 那么

$$tmt = t^2 m < 0.$$

因为 $t^2 > 0$, 所以符号不变。矩阵版就是同一个想法。

注意: congruence transformation 不是随便左乘或右乘。必须是:

$$T^T M T$$

也就是左右配套地乘, 才能保证 definiteness 不变。

10.10.2.1 Why Roy Uses It Here

在 discrete-time stability 里, Schur complement 先给出:

$$\begin{bmatrix} -P & A^T \\ A & -P^{-1} \end{bmatrix} \prec 0.$$

这个矩阵里有 P^{-1} , 不是 linear in P 。Roy 的做法是用

$$T = \begin{bmatrix} I & 0 \\ 0 & P \end{bmatrix}$$

做 congruence transformation:

$$T^T \begin{bmatrix} -P & A^T \\ A & -P^{-1} \end{bmatrix} T \prec 0.$$

这里 $P = P^T$ 不是额外假设, 而是来自 $P \succ 0$ 的约定: 在 real matrix LMI 里, $P \succ 0$ 表示 P 是 symmetric positive definite (对称正定) 矩阵。因此 Lyapunov matrix P 本来就是 symmetric。

所以：

$$T^T = \begin{bmatrix} I^T & 0 \\ 0 & P^T \end{bmatrix} = \begin{bmatrix} I & 0 \\ 0 & P \end{bmatrix} = T.$$

然后逐块乘：

$$\begin{bmatrix} I & 0 \\ 0 & P \end{bmatrix} \begin{bmatrix} -P & A^T \\ A & -P^{-1} \end{bmatrix} \begin{bmatrix} I & 0 \\ 0 & P \end{bmatrix} = \begin{bmatrix} -P & A^T P \\ PA & -P \end{bmatrix} \prec 0.$$

你可以看到右下角：

$$P(-P^{-1})P = -P.$$

这就是 Roy 说 “the first P cancels that one, and then this P makes a P ” 的意思。这样 P^{-1} 消失了，矩阵元素变成 linear in P 。

10.10.2.2 Second Use: Change P into $Y = P^{-1}$

状态反馈设计里会出现：

$$\begin{bmatrix} -P & (A+BK)^T P \\ P(A+BK) & -P \end{bmatrix} \prec 0.$$

这里的问题不是 P^{-1} ，而是 P 和 K 乘在一起：

$$P(A+BK) = PA + PBK.$$

如果 P 和 K 都未知，这不是 LMI。于是用另一个 congruence transformation，把 P 换成 $Y = P^{-1}$ 。令

$$T = \begin{bmatrix} P^{-1} & 0 \\ 0 & P^{-1} \end{bmatrix} = \begin{bmatrix} Y & 0 \\ 0 & Y \end{bmatrix}.$$

对上面的矩阵做 $T^T M T$ ，会得到等价形式（差一个整体负号和 off-diagonal 的符号；这些不改变核心可行性）：

$$\begin{bmatrix} Y & Y(A+BK)^T \\ (A+BK)Y & Y \end{bmatrix} \succ 0.$$

现在乘积变成：

$$(A+BK)Y = AY + BK Y.$$

这时令

$$V = KY,$$

就得到 $AY + BV$ ，对新变量 Y, V 是 linear。解完之后再还原：

$$K = VY^{-1}.$$

一句话记忆：

Congruence transformation is a legal way to pre/post multiply an LMI without changing positive/negative definiteness. It is used to remove inverses or prepare a change of variables.

10.11 State Feedback Design via LMI (状态反馈设计)

这是 Lec 9 的高潮，也是最可能考的推导。系统

$$x_{k+1} = Ax_k + Bu_k, \quad u_k = Kx_k \Rightarrow x_{k+1} = (A + BK)x_k$$

目标：设计 feedback gain K 使闭环稳定。把离散稳定性 LMI 里的 A 换成 $A + BK$ ：

$$\begin{bmatrix} -P & (A + BK)^T P \\ P(A + BK) & -P \end{bmatrix} \prec 0$$

问题来了： P 和 K 都未知，而 $P(A + BK) = PA + PBK$ 里出现了 P 与 K 的乘积——**非线性**，不能直接当 LMI。

关键技巧（两步）：

1. **Congruence transformation + 换元** $Y = P^{-1}$ 。用 $\text{diag}(P^{-1}, P^{-1})$ 左右同乘（congruence，不改变 definiteness），把条件改写成只含 $Y = P^{-1}$ 的形式：

$$\begin{bmatrix} Y & Y(A + BK)^T \\ (A + BK)Y & Y \end{bmatrix} \succ 0$$

2. **换元** $V = KY$ 。现在 KY 整体出现，令 $V = KY$ ，得到一条 **线性** LMI：

$$\begin{bmatrix} Y & YA^T + V^T B^T \\ AY + BV & Y \end{bmatrix} \succ 0$$

优化变量是 Y 和 V （都不含乘积）。解出后 **还原** gain：

$$K = VY^{-1}$$

记忆钩子:

```
P*K is nonlinear -> change variables.
Y = P^{-1}, V = KY => the LMI becomes linear in (Y, V).
Recover the controller: K = V Y^{-1}.
```

10.12 LQR via LMI (线性二次型调节器)

加上 performance (不只是稳定)。系统 $x_{k+1} = Ax_k + Bu_k + Fw_k$, cost

$$J(x) = \sum_{k=0}^{\infty} (x_k^T Q x_k + u_k^T R u_k) = \sum_k x_k^T \begin{bmatrix} Q^{1/2} & K^T R^{1/2} \end{bmatrix} \begin{bmatrix} Q^{1/2} \\ R^{1/2} K \end{bmatrix} x_k$$

定理 (LMI 形式): $J(x) < \gamma$ 当且仅当存在 symmetric W, P 使

$$\text{trace}(W) < \gamma, \quad \begin{bmatrix} W & \begin{bmatrix} Q^{1/2} \\ R^{1/2} K \end{bmatrix} P \\ P \begin{bmatrix} Q^{1/2} & K^T R^{1/2} \end{bmatrix} & P \end{bmatrix} \succ 0,$$

并且

$$\begin{bmatrix} P & (A+BK)P & F \\ P(A+BK)^T & P & 0 \\ F^T & 0 & I \end{bmatrix} \succ 0.$$

同样地, $(A+BK)P$ 和 KP 含 $K \cdot P$ 乘积 —— 用同一招 $V = KP$ 线性化 (注意: 这里这个 P 扮演的就是上一节 $Y = P^{-1}$ 的角色, 是 inverse Lyapunov matrix; Roy 的 slides 在两页之间换了记号, 别被绕进去)。最终 design optimisation:

$$\min_{W, P, V} \gamma \quad \text{subject to} \quad \text{trace}(W) < \gamma,$$

$$\begin{bmatrix} W & \begin{bmatrix} Q^{1/2} \\ R^{1/2} V \end{bmatrix} \\ \begin{bmatrix} Q^{1/2} & V^T R^{1/2} \end{bmatrix} & P \end{bmatrix} \succ 0, \quad \begin{bmatrix} P & AP + BV & F \\ PA^T + V^T B^T & P & 0 \\ F^T & 0 & I \end{bmatrix} \succ 0$$

变量是 W, P, V ($W, P \succ 0$, V 不必), 解完用 $K = VP^{-1}$ 得到最优状态反馈。这整套可以几乎照抄成 CVX 代码 (trace 能直接进 CVX)。

考试理解重点:

- LQR 你可能已从别的课见过 (解 Riccati equation)。这里展示的是 **另一条路**: 写成 **LMI / SDP** 求解。
- 核心永远是同一个把戏: **乘积 $K \cdot P$ 非线性 $\rightarrow$ 换元 $V = KP \rightarrow$ LMI $\rightarrow$ 解完 $K = VP^{-1}$ 还原。**
- 这种方法的真正价值是 **可扩展**: 把目标换成别的 LMI 就能做更多设计 (见下)。

10.13 LMI-based Control Design (还能做什么)

Roy 列出 LMI 框架能统一处理的控制设计 / 分析问题 (不需要手推, 知道” 这些都能写成 LMI” 即可):

- pole placement (极点配置, 在 s-平面 / z-平面的某个 region 内);
- 最小化闭环增益 (H_∞ norm 或 H_2 norm);
- Kalman filter 设计 (从测量最优估计 state 的 filter);
- robust stability / robust performance 分析 (A, B 有 $\pm 10\%$ 变动还稳不稳);
- robust control design (系统带 uncertainty 的设计)。

一句话: **经典控制理论的大半都能翻译成 LMI 约束, 再交给 SDP solver。** 把一个新的控制目标第一次写成 LMI, 本身就能发论文。

11 Model Predictive Control

11.1 Core Idea

MPC 的基本循环:

1. measure or estimate current state;
2. optimise future state/input trajectory over horizon M ;
3. apply only the first input;
4. move one time step and repeat。

一句话记忆: **plan a full future, execute the first move, then replan.**

11.2 Mathematical Formulation

Dynamics:

$$x_{k+1} = Ax_k + Bu_k + Fw_k$$

Finite horizon cost:

$$J(u, x) = \sum_{i=0}^{M-1} (x_i^T Q x_i + u_i^T R u_i) + x_M^T Q_{\text{terminal}} x_M$$

Roy 说 “Deviation from the desired state, quadratic is usually a pretty good choice” 的意思是：如果你真正关心的是 state x_i 离 desired state / reference state（期望状态） x_{ref} 有多远，cost 通常写成 tracking error（跟踪误差）的二次型：

$$(x_i - x_{\text{ref}})^T Q (x_i - x_{\text{ref}}).$$

如果目标状态就是 zero，例如希望系统回到平衡点 $x = 0$ ，那它就退化成：

$$x_i^T Q x_i.$$

为什么二次型常用？

- 偏差为 0 时 cost 最小；
- 正负偏差同样惩罚，例如 +2 和 -2 都是 error magnitude 2；
- 大偏差惩罚更重，例如 error 从 1 到 2，cost 从 1 到 4；
- $Q \succeq 0$ 可以给不同 state 不同权重，例如位置误差比速度误差更重要；
- quadratic cost + linear dynamics / linear constraints 通常得到 QP，solver 很成熟。

但 Roy 也提醒：不是所有 cost 都必须 quadratic。比如 energy / money 这类 economic cost（经济成本）可能更像 linear cost；steering input 有时只需要加 hard bound，不一定需要在 objective 里惩罚。

Typical MPC problem:

$$\begin{aligned} \min_{x,u} \quad & J(u, x) \\ \text{subject to} \quad & x_{k+1} = Ax_k + Bu_k + Fw_k \\ & u_{\text{lb}} \leq u_k \leq u_{\text{ub}} \\ & x_k \in X_{\text{operating}} \\ & x_M \in X_{\text{terminal}} \end{aligned}$$

重要理解：

- dynamics 是 constraints；
- decision variables 通常包括 future inputs 和 future states；
- disturbance forecast 可以作为 feedforward information；
- repeated measurement/re-estimation 提供 feedback。

11.3 Hard, Soft, and Terminal Constraints

Hard constraints:

- 必须满足；
- 例子：actuator limits、safety limits。

Soft constraints:

- 想满足，但允许违反；
- 通过 penalty cost 处理；
- 例子：building temperature comfort band。

Terminal constraints:

- 规定 horizon 末端状态；
- 用于 recursive feasibility 和 closed-loop stability；
- 常见选择是 invariant terminal set。

11.4 Explicit MPC

Online MPC 每个 time step 求解 optimisation。Explicit MPC 先离线解 parametric optimisation，把 state space 切成 regions：

- 每个 region 有一个对应的 affine feedback law；
- online 时只判断当前 state 落在哪个 region；
- 很快，但 regions 数量可能指数增长。

适用场景：

- low-dimensional system；
- extremely high-speed control；
- embedded controller。

不适合：

- 高维系统；
- long horizon；
- constraints 很多导致 regions 爆炸。

11.4.1 Worked Example: One-Step Explicit MPC via KKT

Roy 用一个 one-step-ahead example 展示 explicit MPC 为什么会得到 piecewise affine controller（分段仿射控制器）。这里 horizon 只有一步，input 是 scalar（单输入）：

$$x_{k+1} = Ax_k + Bu_k.$$

目标是选 u_k ，让下一步 state 和 input cost 小，同时满足 input bounds：

$$u_{\text{lbnd}} \leq u_k \leq u_{\text{ubnd}}.$$

对应 optimisation problem:

$$\begin{aligned} \min_{u_k} \quad & x_{k+1}^T Q x_{k+1} + u_k^T R u_k \\ \text{subject to} \quad & x_{k+1} = A x_k + B u_k, \\ & u_k - u_{\text{ubnd}} \leq 0, \\ & u_{\text{lbnd}} - u_k \leq 0. \end{aligned}$$

11.4.1.1 Physical Meaning of A, B, Q, R (这些符号通常代表什么)

题目里给 A, B, Q, R 时，它们首先是数学对象；但在控制工程里，它们通常不是随便来的。

Dynamics matrices:

- A 是 state transition matrix (状态转移矩阵)。它描述 **如果不额外控制，当前状态 x_k 会怎样自然演化到下一步**。例如位置、速度、温度、库存量、电压等状态之间的自然耦合，都藏在 A 里。
- B 是 input matrix / control matrix (输入矩阵、控制矩阵)。它描述 **control input u_k 能怎样影响下一步状态**。例如电机电压对转速的影响、加速度命令对位置和速度的影响、加热功率对室温的影响。

Cost matrices:

- Q 是 state penalty / tracking-error weight (状态误差权重)。它回答：**哪些 state deviation 更重要？** 如果某个状态不能偏离太多，就给它更大的权重。
- R 是 input penalty / control-effort weight (输入努力权重)。它回答：**我们愿不愿意用很大的控制动作？** R 越大，controller 越保守，越不愿意猛踩油门、猛加热、猛打方向。

Input bounds:

- $u_{\text{lbnd}}, u_{\text{ubnd}}$ 是 actuator limits (执行器限制)。它们通常来自真实硬件：最大电压、最大力矩、最大阀门开度、最大充放电功率等。

所以这个 one-step MPC problem 的工程直觉是：

在系统动力学 $x_{k+1} = A x_k + B u_k$ 和 actuator limits 允许的范围内，选择一个控制动作 u_k ，让下一步状态尽量好，同时不要用太激烈的控制。

一个常见 trade-off:

- Q 大、 R 小：更 aggressive，愿意用大输入快速压低 state error；
- Q 小、 R 大：更 conservative，动作平滑，但 state error 可能消得慢；
- bounds 很紧：即使 Q 很大，controller 也不能超过硬件极限。

Roy 笔记里的 typo 要注意：

- dynamics 里应是 x_{k+1} ，不是 x_{k+t} ；
- 展开 quadratic term 时应是 $B^T Q B + R$ ，不是漏掉 Q 的 $B^T B + R$ 。

11.4.1.2 KKT Sign Convention (为什么约束要这样写)

KKT 里最常见的 inequality convention 是:

$$g_i(u) \leq 0, \quad \lambda_i \geq 0,$$

然后 Lagrangian 写成:

$$L(u, \lambda) = f(u) + \sum_i \lambda_i g_i(u).$$

所以这里必须先把 input bounds 改写成 “ ≤ 0 ” 的形式。

Upper bound:

$$u_k \leq u_{\text{ubnd}} \iff g_u(u_k) = u_k - u_{\text{ubnd}} \leq 0.$$

Lower bound:

$$u_k \geq u_{\text{lbnd}} \iff u_{\text{lbnd}} - u_k \leq 0 \iff g_l(u_k) = u_{\text{lbnd}} - u_k \leq 0.$$

因此 Lagrangian 里出现:

$$+\lambda_u(u_k - u_{\text{ubnd}}) + \lambda_l(u_{\text{lbnd}} - u_k), \quad \lambda_u, \lambda_l \geq 0.$$

注意: 在 feasible region 里面, $g_i(u) \leq 0$ 确实通常是负数或 0。active constraint (活跃约束) 时 $g_i(u) = 0$; inactive 时 $g_i(u) < 0$, 由 complementarity 得 $\lambda_i = 0$ 。

如果你换另一种 convention, 把约束写成 $h_i(u) \geq 0$, 那 Lagrangian 通常要写成 $f(u) - \mu_i h_i(u)$ 且 $\mu_i \geq 0$ 。两套写法等价, 但不要混用。本 guide 统一使用:

```
constraint <= 0, multiplier >= 0, L = f + lambda * constraint
```

11.4.1.3 Step 1: Write the Lagrangian

对 upper bound $u_k - u_{\text{ubnd}} \leq 0$ 加 multiplier $\lambda_u \geq 0$; 对 lower bound $u_{\text{lbnd}} - u_k \leq 0$ 加 multiplier $\lambda_l \geq 0$:

$$L(x, u, \lambda) = x_{k+1}^T Q x_{k+1} + u_k^T R u_k + \lambda_u(u_k - u_{\text{ubnd}}) + \lambda_l(u_{\text{lbnd}} - u_k).$$

把 dynamics $x_{k+1} = Ax_k + Bu_k$ 代入:

$$\begin{aligned}
x_{k+1}^T Q x_{k+1} &= (Ax_k + Bu_k)^T Q (Ax_k + Bu_k) \\
&= x_k^T A^T Q Ax_k + 2x_k^T A^T Q Bu_k + u_k^T B^T Q Bu_k.
\end{aligned}$$

所以 Lagrangian 变成:

$$\begin{aligned}
L &= x_k^T A^T Q Ax_k + 2x_k^T A^T Q Bu_k + u_k^T (B^T Q B + R) u_k \\
&\quad + \lambda_u (u_k - u_{\text{ubnd}}) + \lambda_l (u_{\text{lbnd}} - u_k).
\end{aligned}$$

为了让后面式子短一点, 定义

$$H = B^T Q B + R, \quad g = B^T Q A.$$

单输入时 H 是 scalar, 且通常 $H > 0$; $g x_k$ 是 scalar。

11.4.1.4 Step 2: KKT Conditions

KKT conditions 包括四类:

1. Stationarity (对 u_k 求导为 0)

$$\frac{\partial L}{\partial u_k} = 2B^T Q Ax_k + 2(B^T Q B + R)u_k + \lambda_u - \lambda_l = 0.$$

用 H, g 记:

$$2g x_k + 2H u_k + \lambda_u - \lambda_l = 0.$$

2. Primal feasibility (原约束满足)

$$u_k - u_{\text{ubnd}} \leq 0, \quad u_{\text{lbnd}} - u_k \leq 0.$$

也就是:

$$u_{\text{lbnd}} \leq u_k \leq u_{\text{ubnd}}.$$

3. Dual feasibility (乘子非负)

$$\lambda_u \geq 0, \quad \lambda_l \geq 0.$$

4. Complementarity (互补松弛)

$$\lambda_u(u_k - u_{\text{ubnd}}) = 0, \quad \lambda_l(u_{\text{lbnd}} - u_k) = 0.$$

互补松弛的意思是：

either the constraint is active and its multiplier may be positive,
or the constraint is inactive and its multiplier must be zero.

因为 u_k 不可能同时等于 upper bound 和 lower bound（除非两个 bound 相等），所以只有三种 case：

1. upper bound active;
2. lower bound active;
3. neither active, so input is inside the bounds。

11.4.1.5 Case 1: Upper Bound Active

Upper bound active means:

$$u_k = u_{\text{ubnd}}, \quad \lambda_u \geq 0, \quad \lambda_l = 0.$$

代入 stationarity:

$$2gx_k + 2Hu_{\text{ubnd}} + \lambda_u = 0.$$

所以：

$$\lambda_u = -2gx_k - 2Hu_{\text{ubnd}}.$$

因为 $\lambda_u \geq 0$ ，得到 region condition:

$$-2gx_k - 2Hu_{\text{ubnd}} \geq 0 \iff gx_k \leq -Hu_{\text{ubnd}}.$$

因此在这个 state-space region 里，optimal control 是：

$$u_k^* = u_{\text{ubnd}}.$$

这就是一个 saturation region（饱和区域）：unconstrained control 想超过 upper bound，所以最优解被卡在上界。

11.4.1.6 Case 2: Lower Bound Active

Lower bound active means:

$$u_k = u_{\text{lbnd}}, \quad \lambda_u = 0, \quad \lambda_l \geq 0.$$

代入 stationarity:

$$2gx_k + 2Hu_{\text{lbnd}} - \lambda_l = 0.$$

所以:

$$\lambda_l = 2gx_k + 2Hu_{\text{lbnd}}.$$

因为 $\lambda_l \geq 0$, 得到:

$$2gx_k + 2Hu_{\text{lbnd}} \geq 0 \quad \Longleftrightarrow \quad gx_k \geq -Hu_{\text{lbnd}}.$$

因此在这个 region 里:

$$u_k^* = u_{\text{lbnd}}.$$

11.4.1.7 Case 3: No Bound Active (Interior Region)

No bound active means:

$$\lambda_u = 0, \quad \lambda_l = 0, \quad u_{\text{lbnd}} < u_k < u_{\text{ubnd}}.$$

Stationarity 变成:

$$2gx_k + 2Hu_k = 0.$$

所以:

$$u_k^* = -H^{-1}gx_k.$$

这就是一个 linear state feedback law:

$$u_k^* = Kx_k, \quad K = -H^{-1}g = -(B^TQB + R)^{-1}B^TQA.$$

它只在满足 input bounds 时有效:

$$u_{\text{lbnd}} \leq -H^{-1}gx_k \leq u_{\text{ubnd}}.$$

因为 $H > 0$, 可以写成 state-space 条件:

$$-Hu_{\text{ubnd}} \leq gx_k \leq -Hu_{\text{lbnd}}.$$

11.4.1.8 Final Explicit Controller

把三种情况合在一起:

$$u_k^*(x_k) = \begin{cases} u_{\text{ubnd}}, & gx_k \leq -Hu_{\text{ubnd}}, \\ -H^{-1}gx_k, & -Hu_{\text{ubnd}} \leq gx_k \leq -Hu_{\text{lbnd}}, \\ u_{\text{lbnd}}, & gx_k \geq -Hu_{\text{lbnd}}. \end{cases}$$

这就是 explicit MPC 的核心形状:

state space is split into regions;
inside each region, the optimal input is either constant or affine in the state.

在这个简单例子里只有 3 个 regions:

Region	Condition	Optimal input
upper saturation	$gx_k \leq -Hu_{\text{ubnd}}$	u_{ubnd}
interior feedback	$-Hu_{\text{ubnd}} \leq gx_k \leq -Hu_{\text{lbnd}}$	$-H^{-1}gx_k$
lower saturation	$gx_k \geq -Hu_{\text{lbnd}}$	u_{lbnd}

这也是 Roy 说的: online 时不必每一步重新跑 optimiser; 只需要判断当前 x_k 落在哪个 region, 然后套对应公式。

12 Tutorial and Exercise Route

12.1 LS Exercises

重点题型:

1. BLUE proof

- unbiased condition: $Z^T\Phi = I$;

- covariance: $\text{Cov}(\hat{\theta}) = Z^T R Z$;
- BLUE estimator:

$$\hat{\theta}_Z = (\Phi^T R^{-1} \Phi)^{-1} \Phi^T R^{-1} Y$$

2. Polynomial and basis-function fitting

- 根据 basis functions 建立 design matrix;
- 判断 $X^T X$ 是否 nonsingular;
- radial basis functions 与 kernel matrix。

3. Conic section fitting

- 建立 parameter vector;
- 固定 scaling, 例如令 $f = 1$;
- 判断 ellipse/parabola/hyperbola。

12.2 Statistics Exercises

重点题型:

- true/false 判断:
 - regularised estimate 通常 biased;
 - inverse variance estimate unbiased;
 - inverse variance weighting 是 BLUE;
 - regularised estimate 可以有更低 MSE。
- 概念解释:
 - BLUE minimises variance only among unbiased linear estimators;
 - minimum MSE may accept bias to reduce variance;
 - confidence interval measures coefficient uncertainty;
 - coefficient significance tests whether 0 is plausible for that coefficient;
 - data mining 中的 underfitting / overfitting 是 bias-variance tradeoff 的预测版本。
- 计算:
 - ordinary LS estimate;
 - covariance matrix;
 - standard error / confidence interval / t-statistic;
 - non-identically distributed noise 下的 BLUE。

12.3 Lecture 6–7 High-frequency LS Practice Set

Use this section actively: read the question, cover the solution, derive it yourself, then compare.

12.3.1 Problem 1: Build Φ and Derive Normal Equations

Question. Suppose the model is:

$$y_i \approx \theta_1 + \theta_2 x_i + \theta_3 x_i^2, \quad i = 1, \dots, N.$$

1. Write y , θ , and Φ .
2. Write the LS objective.
3. Derive the normal equations.
4. State the closed-form solution and the rank condition.

Solution.

Stack the observations:

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}, \quad \theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}.$$

The design matrix is:

$$\Phi = \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & \vdots & \vdots \\ 1 & x_N & x_N^2 \end{bmatrix}.$$

The LS problem is:

$$\min_{\theta} \|y - \Phi\theta\|_2^2.$$

Let:

$$J(\theta) = (y - \Phi\theta)^T (y - \Phi\theta).$$

Expand:

$$J(\theta) = y^T y - 2\theta^T \Phi^T y + \theta^T \Phi^T \Phi \theta.$$

Differentiate:

$$\nabla_{\theta} J = -2\Phi^T y + 2\Phi^T \Phi \theta.$$

Set to zero:

$$\Phi^T \Phi \theta = \Phi^T y.$$

If Φ has full column rank, then $\Phi^T \Phi$ is invertible and:

$$\hat{\theta} = (\Phi^T \Phi)^{-1} \Phi^T y.$$

Exam comments:

- If $N < 3$, full column rank is impossible.
- If all x_i values do not vary enough, columns can become dependent or nearly dependent.
- In practice, use QR/SVD/backslash rather than explicitly inverting $\Phi^T \Phi$.

12.3.2 Problem 2: Projection Geometry Gives Normal Equations

Question. Explain geometrically why the LS residual is orthogonal to the columns of Φ , and derive the normal equations from that fact.

Solution.

The fitted vector is:

$$\hat{y} = \Phi \hat{\theta}.$$

All possible fitted vectors lie in the column space:

$$\text{col}(\Phi).$$

OLS chooses the point in $\text{col}(\Phi)$ closest to y . Therefore the residual:

$$r = y - \Phi \hat{\theta}$$

must be orthogonal to every direction in $\text{col}(\Phi)$.

The columns of Φ span this space, so:

$$\Phi^T r = 0.$$

Substitute $r = y - \Phi\hat{\theta}$:

$$\Phi^T(y - \Phi\hat{\theta}) = 0.$$

Expand:

$$\Phi^T y - \Phi^T \Phi \hat{\theta} = 0.$$

Therefore:

$$\Phi^T \Phi \hat{\theta} = \Phi^T y.$$

This is the same normal equation obtained by differentiating the cost.

Exam sentence:

Least squares projects y onto $\text{col}(\Phi)$; the residual is orthogonal to that subspace, giving $\Phi^T r = 0$.

12.3.3 Problem 3: Prove OLS Is Unbiased and Find Its Covariance

Question. Suppose:

$$y = \Phi\theta_0 + \epsilon, \quad E[\epsilon] = 0, \quad \text{Cov}(\epsilon) = \sigma^2 I,$$

and Φ has full column rank. Show that ordinary LS is unbiased and compute its covariance.

Solution.

OLS estimator:

$$\hat{\theta} = (\Phi^T \Phi)^{-1} \Phi^T y.$$

Substitute the data-generating model:

$$\begin{aligned} \hat{\theta} &= (\Phi^T \Phi)^{-1} \Phi^T (\Phi\theta_0 + \epsilon) \\ &= \theta_0 + (\Phi^T \Phi)^{-1} \Phi^T \epsilon. \end{aligned}$$

Take expectation:

$$\begin{aligned} E[\hat{\theta}] &= \theta_0 + (\Phi^T \Phi)^{-1} \Phi^T E[\epsilon] \\ &= \theta_0. \end{aligned}$$

So OLS is unbiased.

For covariance, the error is:

$$\hat{\theta} - \theta_0 = (\Phi^T \Phi)^{-1} \Phi^T \epsilon.$$

Use:

$$\text{Cov}(A\epsilon) = A\text{Cov}(\epsilon)A^T.$$

Here:

$$A = (\Phi^T \Phi)^{-1} \Phi^T.$$

Therefore:

$$\begin{aligned} \text{Cov}(\hat{\theta}) &= (\Phi^T \Phi)^{-1} \Phi^T (\sigma^2 I) \Phi (\Phi^T \Phi)^{-1} \\ &= \sigma^2 (\Phi^T \Phi)^{-1}. \end{aligned}$$

Exam comments:

- zero-mean error gives unbiasedness;
- $\Phi^T \Phi$ controls estimator variance;
- weak or nearly dependent columns make covariance large.

12.3.4 Problem 3B: Standard Error, Confidence Interval, and Significance

Question. Suppose an OLS fit gives:

$$\hat{\theta}_j = -0.40, \quad \widehat{\text{se}}(\hat{\theta}_j) = 0.15, \quad N - d = 30.$$

Use $t_{0.975,30} \approx 2.04$.

1. Compute a 95% confidence interval for θ_j .
2. Is the coefficient significant at $\alpha = 0.05$?
3. Interpret this in Roy's Φ framework.

Solution.

A 95% confidence interval is:

$$\hat{\theta}_j \pm t_{0.975,30} \widehat{\text{se}}(\hat{\theta}_j).$$

Substitute:

$$-0.40 \pm 2.04(0.15).$$

Compute the margin:

$$2.04(0.15) = 0.306.$$

So:

$$\theta_j \in [-0.40 - 0.306, -0.40 + 0.306].$$

Therefore:

$$\theta_j \in [-0.706, -0.094].$$

Since 0 is not inside the interval, the coefficient is significant at $\alpha = 0.05$.

Equivalent t-statistic:

$$t_j = \frac{-0.40}{0.15} \approx -2.67.$$

Since:

$$|t_j| > 2.04,$$

we reject:

$$H_0 : \theta_j = 0$$

at the 5% level.

Roy-framework interpretation:

```
theta_hat_j is not just a number;
it is a random estimate with standard error.
```

The confidence interval asks whether the data identify that column of Φ strongly enough to distinguish its coefficient from 0. If the interval excludes 0, that column has statistically detectable linear contribution, conditional on the other columns in Φ .

Common mistakes:

- saying significance proves causal importance;
- forgetting that MLR significance is conditional on other features;
- ignoring multicollinearity, which can make standard errors large;
- interpreting one realised CI as assigning probability to fixed θ_j .

12.3.5 Problem 4: Derive Inverse-variance Weighting

Question. Suppose $\hat{\theta}_i$ are independent unbiased estimators of θ with:

$$\text{Var}(\hat{\theta}_i) = \sigma_i^2.$$

Find weights w_i such that:

$$\hat{\theta}_w = \sum_i w_i \hat{\theta}_i$$

is unbiased and has minimum variance.

Solution.

Unbiasedness:

$$\begin{aligned} E[\hat{\theta}_w] &= \sum_i w_i E[\hat{\theta}_i] \\ &= \theta \sum_i w_i. \end{aligned}$$

Therefore:

$$\sum_i w_i = 1.$$

Variance:

$$\text{Var}(\hat{\theta}_w) = \sum_i w_i^2 \sigma_i^2.$$

Optimisation problem:

$$\min_w \sum_i w_i^2 \sigma_i^2 \quad \text{s.t.} \quad \sum_i w_i = 1.$$

Lagrangian:

$$L = \sum_i w_i^2 \sigma_i^2 + \lambda \left(\sum_i w_i - 1 \right).$$

First-order condition:

$$\frac{\partial L}{\partial w_i} = 2w_i \sigma_i^2 + \lambda = 0.$$

Hence:

$$w_i = -\frac{\lambda}{2\sigma_i^2}.$$

Let:

$$c = -\frac{\lambda}{2}.$$

Then:

$$w_i = \frac{c}{\sigma_i^2}.$$

Use the constraint:

$$\sum_i w_i = 1.$$

Substitute:

$$\sum_i \frac{c}{\sigma_i^2} = 1.$$

Factor out c :

$$c \sum_i \frac{1}{\sigma_i^2} = 1.$$

So:

$$c = \frac{1}{\sum_j 1/\sigma_j^2}.$$

Therefore:

$$w_i = \frac{1/\sigma_i^2}{\sum_j 1/\sigma_j^2}.$$

Interpretation:

small variance $\rightarrow$ large inverse variance $\rightarrow$ large weight
 large variance $\rightarrow$ small inverse variance $\rightarrow$ small weight

12.3.6 Problem 5: BLUE with Correlated Noise

This is a medium/high probability exam-style problem.

Question. Suppose:

$$y = \Phi\theta + \epsilon, \quad E[\epsilon] = 0, \quad \text{Cov}(\epsilon) = R,$$

where $R \succ 0$ is known and Φ has full column rank. Consider a linear estimator:

$$\hat{\theta} = Z^T y.$$

1. Find the condition on Z for $\hat{\theta}$ to be unbiased.
2. Compute $\text{Cov}(\hat{\theta})$.
3. State the BLUE.
4. Show that it reduces to ordinary LS when $R = \sigma^2 I$.
5. Explain why knowing R matters in practice.

Solution.

Start with:

$$\hat{\theta} = Z^T y.$$

Substitute the model:

$$\begin{aligned} \hat{\theta} &= Z^T(\Phi\theta + \epsilon) \\ &= Z^T\Phi\theta + Z^T\epsilon. \end{aligned}$$

Take expectation:

$$\begin{aligned} E[\hat{\theta}] &= E[Z^T\Phi\theta + Z^T\epsilon] \\ &= Z^T\Phi\theta + Z^T E[\epsilon] \\ &= Z^T\Phi\theta. \end{aligned}$$

For unbiasedness, we need:

$$E[\hat{\theta}] = \theta$$

for every possible θ . Therefore:

$$Z^T \Phi \theta = \theta \quad \text{for all } \theta.$$

So:

$$Z^T \Phi = I.$$

This is the unbiasedness constraint.

Now compute the covariance:

$$\hat{\theta} - \theta = Z^T \epsilon$$

when $Z^T \Phi = I$. Therefore:

$$\begin{aligned} \text{Cov}(\hat{\theta}) &= E[(\hat{\theta} - \theta)(\hat{\theta} - \theta)^T] \\ &= E[Z^T \epsilon \epsilon^T Z] \\ &= Z^T E[\epsilon \epsilon^T] Z \\ &= Z^T R Z. \end{aligned}$$

So among all linear unbiased estimators, the goal is:

$$\min_Z Z^T R Z \quad \text{subject to} \quad Z^T \Phi = I.$$

The solution is:

$$Z^T = (\Phi^T R^{-1} \Phi)^{-1} \Phi^T R^{-1}.$$

Therefore the BLUE is:

$$\hat{\theta}_{\text{BLUE}} = (\Phi^T R^{-1} \Phi)^{-1} \Phi^T R^{-1} y.$$

Its covariance is:

$$\text{Cov}(\hat{\theta}_{\text{BLUE}}) = (\Phi^T R^{-1} \Phi)^{-1}.$$

Now check the white-noise special case. If:

$$R = \sigma^2 I,$$

then:

$$R^{-1} = \frac{1}{\sigma^2} I.$$

Substitute into BLUE:

$$\begin{aligned} \hat{\theta}_{\text{BLUE}} &= \left(\Phi^T \frac{1}{\sigma^2} I \Phi \right)^{-1} \Phi^T \frac{1}{\sigma^2} I y \\ &= \left(\frac{1}{\sigma^2} \Phi^T \Phi \right)^{-1} \frac{1}{\sigma^2} \Phi^T y \\ &= \sigma^2 (\Phi^T \Phi)^{-1} \frac{1}{\sigma^2} \Phi^T y \\ &= (\Phi^T \Phi)^{-1} \Phi^T y. \end{aligned}$$

So when the noise is white, BLUE reduces to ordinary LS.

Interpretation:

ordinary LS:

assumes $R = \sigma^2 I$

correlated-noise BLUE:

uses R^{-1} to account for variance and correlation structure

Practical issue: to compute the best linear unbiased estimator, you need to know or estimate R . In an exam, R may be given. In real data, estimating R can be difficult.

Common mistakes:

- writing ordinary LS when the question gives nontrivial R ;
- forgetting the unbiasedness constraint $Z^T \Phi = I$;
- saying correlated noise makes LS biased. Ordinary LS can still be unbiased, but it is generally not best;
- forgetting that R^{-1} disappears only when $R = \sigma^2 I$.

12.3.7 Problem 6: Explain Bias-variance Tradeoff and Regularisation

Question. Explain why an unbiased LS estimator is not necessarily the minimum-MSE estimator. Then explain how ridge regularisation changes bias and variance.

Solution.

The key identity is:

$$\text{MSE} = \text{Bias}^2 + \text{Variance}.$$

An unbiased estimator satisfies:

$$E[\hat{\theta}] = \theta_0,$$

so:

$$\text{Bias} = 0.$$

Then:

$$\text{MSE} = \text{Variance}.$$

But this only says that MSE equals variance **inside the unbiased case**. It does not prove that no biased estimator can have smaller total MSE.

Ridge solves:

$$\min_{\theta} \|y - \Phi\theta\|_2^2 + \eta\|\theta\|_2^2.$$

The solution is:

$$\hat{\theta}_{\eta} = (\Phi^T\Phi + \eta I)^{-1}\Phi^T y.$$

For $\eta > 0$, the estimate is shrunk toward zero. Usually:

```
eta increases
-> variance decreases
-> bias increases
```

This can improve MSE if the variance reduction is larger than the bias increase:

$$\Delta\text{Variance} > \Delta\text{Bias}^2.$$

Exam sentence:

BLUE minimises variance among unbiased linear estimators; ridge may be biased but can have lower MSE

Connection to validation:

training error usually decreases as model flexibility increases
 validation error estimates future prediction error
 choose eta by validation/CV, not by training residual alone

12.3.8 Problem 7: Nonlinear LS as Repeated Linear LS

Question. Suppose:

$$y_{\text{obs}} = H(\theta) + \epsilon.$$

Explain how Gauss-Newton turns nonlinear LS into repeated linear LS problems.

Solution.

The nonlinear least-squares objective is:

$$\min_{\theta} \frac{1}{2} \|y_{\text{obs}} - H(\theta)\|_2^2.$$

At current estimate θ_k , linearise H :

$$H(\theta_k + \delta\theta) \approx H(\theta_k) + J_k \delta\theta,$$

where:

$$J_k = \left. \frac{\partial H}{\partial \theta} \right|_{\theta_k}.$$

Define current residual:

$$r_k = y_{\text{obs}} - H(\theta_k).$$

Then:

$$y_{\text{obs}} - H(\theta_k + \delta\theta) \approx r_k - J_k \delta\theta.$$

So the local problem is:

$$\min_{\delta\theta} \|r_k - J_k \delta\theta\|_2^2.$$

This is an ordinary linear LS problem with:

```
output vector      = r_k
regressor matrix   = J_k
unknown parameter  = delta theta
```

The normal equations are:

$$J_k^T J_k \delta\theta = J_k^T r_k.$$

Solve for $\delta\theta$, then update:

$$\theta_{k+1} = \theta_k + \delta\theta.$$

Then re-linearise and repeat.

Exam sentence:

Gauss-Newton solves nonlinear least squares by repeatedly linearising the model and solving a local LS problem.

Common mistakes:

- treating one linearisation as the global nonlinear solution;
- forgetting that J_k changes after each update;
- confusing gradient descent with Gauss-Newton. Gauss-Newton solves a local LS problem, not just a fixed-size step along the negative gradient.

12.3.9 Problem 8: Match Regularisation Tools to Modelling Goals

Question. For each situation, choose Ridge, LASSO, ElasticNet, AIC/BIC, or validation/CV, and explain why.

1. Many predictors are correlated, and most are expected to have small but real effects.
2. You have many candidate predictors, but expect only a few to matter.
3. Predictors are correlated, but you also want sparsity.
4. You have several candidate models with different numbers of parameters and want an information criterion.
5. You want to choose the actual value of the regularisation hyperparameter.

Solution.

1. Use Ridge. Ridge shrinks coefficients and stabilises collinearity, but usually keeps all predictors.

2. Use LASSO. LASSO's L_1 penalty can set coefficients exactly to zero, so it performs feature selection.
3. Use ElasticNet. It combines L_2 stability with L_1 sparsity.
4. Use AIC/BIC. These compare model fit while penalising number of parameters.
5. Use validation or cross-validation. The best $\eta/\lambda/\alpha$ depends on unknown noise, signal, and data structure, so estimate prediction performance on held-out data.

Exam sentence:

Ridge shrinks, LASSO selects, ElasticNet compromises, AIC/BIC compare models, CV chooses hyperparameter

12.3.10 Problem 9: Why Parameterise the Tikhonov Matrix?

Question. In Tikhonov regularisation, suppose:

$$\min_{\theta} \|y - \Phi\theta\|_2^2 + \theta^T R \theta, \quad R \succ 0.$$

Why does Roy say that choosing an arbitrary positive definite R can become a huge computational/statistical problem? What is the practical alternative?

Solution.

If $\theta \in \mathbb{R}^d$, a symmetric $d \times d$ matrix R has:

$$\frac{d(d+1)}{2}$$

free parameters.

So if we allow arbitrary R , we are not just estimating θ anymore. We are also trying to choose many hyperparameters inside R .

This causes:

high-dimensional hyperparameter search
too little validation data
large computational burden
risk of overfitting the regulariser itself

The practical alternative is to choose a low-dimensional structured family:

$$R = R(\gamma_1, \gamma_2, \dots, \gamma_k), \quad k \ll d^2.$$

Example:

$$R(\eta) = \eta I$$

is ridge with one hyperparameter.

Another example:

$$R(\eta_1, \eta_2) = \eta_1 I + \eta_2 L^T L$$

uses only two hyperparameters but can encode smoothness or other physical structure.

Exam sentence:

Tikhonov regularisation is useful when the penalty matrix encodes structure; an arbitrary positive de

12.4 Convex Optimisation Exercises

重点题型：

- prove convexity (证明凸性) :
 - $C = \{x \mid Ax \leq b\}$;
 - intersection of convex sets (凸集交集);
 - norm cone (范数锥) $\{(r, x) \mid \|x\| \leq r\}$ 。
- formulate as LP (建模成线性规划) :
 - $\min \|Ax - b\|_\infty$;
 - $\min \|Ax - b\|_1$;
 - use epigraph variables (上图变量 / 辅助变量) and linear inequalities (线性不等式)。

Lower priority:

- S-procedure minimum-volume ellipsoid (最小体积椭球) problem is beyond examination detail, but the idea that ellipsoid containment (椭球包含关系) can become an SDP (半定规划) is useful.

13 Exam Templates and Common Mistakes

13.1 Template: Form a Least-squares Problem

1. Identify output vector y .
2. Choose parameter vector θ .
3. Build each row of Φ from the model and data.
4. Write residual $r = y - \Phi\theta$.
5. Minimise $\|r\|_2^2$.
6. If full column rank, use $(\Phi^T \Phi)^{-1} \Phi^T y$.

Common mistakes:

- using $\Phi\Phi^T$ instead of $\Phi^T\Phi$;
- forgetting the dimensions of θ ;
- treating noisy x and noisy y as ordinary LS without comment.

13.2 Template: Show OLS Is Unbiased

Use this when the question asks whether ordinary least squares is unbiased.

Assume:

$$y = \Phi\theta_0 + \epsilon, \quad E[\epsilon] = 0, \quad \Phi^T\Phi \text{ invertible.}$$

Then:

$$\begin{aligned} \hat{\theta} &= (\Phi^T\Phi)^{-1}\Phi^Ty \\ &= (\Phi^T\Phi)^{-1}\Phi^T(\Phi\theta_0 + \epsilon) \\ &= \theta_0 + (\Phi^T\Phi)^{-1}\Phi^T\epsilon. \end{aligned}$$

Therefore:

$$\begin{aligned} E[\hat{\theta}] &= \theta_0 + (\Phi^T\Phi)^{-1}\Phi^TE[\epsilon] \\ &= \theta_0. \end{aligned}$$

So:

$$\text{Bias}(\hat{\theta}) = E[\hat{\theta}] - \theta_0 = 0.$$

Exam sentence:

Under fixed Φ , full column rank, and zero-mean y -error, OLS is unbiased.

13.3 Template: Answer a BLUE Question

1. Start from $\hat{\theta} = Z^Ty$.
2. Use $y = \Phi\theta + \epsilon$.
3. For unbiasedness:

$$E[\hat{\theta}] = Z^T\Phi\theta = \theta \Rightarrow Z^T\Phi = I$$

4. Covariance:

$$\text{Cov}(\hat{\theta}) = Z^T R Z$$

5. State the BLUE:

$$\hat{\theta} = (\Phi^T R^{-1} \Phi)^{-1} \Phi^T R^{-1} y$$

Common mistakes:

- assuming $R = I$ when the question gives correlated noise;
- saying BLUE minimises MSE rather than variance among unbiased linear estimators.

13.4 Template: Derive Inverse-variance Weights

Use this for combining independent unbiased estimates.

Given:

$$E[\hat{\theta}_i] = \theta, \quad \text{Var}(\hat{\theta}_i) = \sigma_i^2, \quad \hat{\theta}_w = \sum_i w_i \hat{\theta}_i.$$

Unbiasedness:

$$E[\hat{\theta}_w] = \theta \sum_i w_i \Rightarrow \sum_i w_i = 1.$$

Variance:

$$\text{Var}(\hat{\theta}_w) = \sum_i w_i^2 \sigma_i^2.$$

Optimisation:

$$\min_w \sum_i w_i^2 \sigma_i^2 \quad \text{s.t.} \quad \sum_i w_i = 1.$$

Lagrangian:

$$L = \sum_i w_i^2 \sigma_i^2 + \lambda \left(\sum_i w_i - 1 \right).$$

First-order condition:

$$2w_i\sigma_i^2 + \lambda = 0 \quad \Rightarrow \quad w_i = \frac{c}{\sigma_i^2}.$$

Use $\sum_i w_i = 1$:

$$c \sum_i \frac{1}{\sigma_i^2} = 1 \quad \Rightarrow \quad c = \frac{1}{\sum_j 1/\sigma_j^2}.$$

Therefore:

$$w_i = \frac{1/\sigma_i^2}{\sum_j 1/\sigma_j^2}.$$

Exam sentence:

The minimum-variance unbiased linear combination weights each estimate in proportion to inverse variance.

13.5 Template: Explain Regularisation

Use this structure:

Regularisation adds a penalty term to the fitting objective.

It shrinks the estimate, which increases bias but decreases variance.

This can reduce MSE and improve prediction, especially when the problem is ill-conditioned.

The hyperparameter eta is usually selected by validation or cross-validation.

13.6 Template: Prove Convexity of a Set

1. Take arbitrary x_1, x_2 in the set.
2. Let $x_\gamma = \gamma x_1 + (1 - \gamma)x_2$, with $\gamma \in [0, 1]$.
3. Show x_γ also satisfies the set condition.
4. Conclude the set is convex.

For $C = \{x \mid Ax \leq b\}$:

$$A(\gamma x_1 + (1 - \gamma)x_2) = \gamma Ax_1 + (1 - \gamma)Ax_2 \leq \gamma b + (1 - \gamma)b = b$$

13.7 Template: Rewrite a Constraint as an LMI (Schur Complement)

Use this when asked to express a quadratic / norm / “minimise the gain” condition as an LMI.

1. **Epigraph if needed.** If the goal is $\min J(x)$, introduce γ : $\min \gamma$ s.t. $J(x) \leq \gamma$.

2. **Get a single scalar / matrix inequality**, e.g. $\gamma^2 I - A(x)A(x)^T \succ 0$, or $x^T Q x + b^T x + c \leq 0$.
3. **Identify the** $\begin{bmatrix} Q & S \\ S^T & R \end{bmatrix}$ **blocks** so that $Q - SR^{-1}S^T$ reproduces your inequality. Pick R to be the easy-to-invert block ($\gamma I, -Q^{-1}, P, \dots$).
4. **Write the block matrix** and state the sign ($\succ 0$ / $\prec 0$) and which block must be definite.
5. **Check linearity in the variables** — that is the whole point: the block matrix must be affine in x (and γ).

Worked patterns to quote:

$$\|A(x)\|_2 < \gamma \iff \begin{bmatrix} \gamma I & A(x) \\ A(x)^T & \gamma I \end{bmatrix} \succ 0, \quad x \in E \iff \begin{bmatrix} 1 & (x_c - x)^T \\ (x_c - x) & P \end{bmatrix} \succeq 0$$

Controller-synthesis trap: a product like $P(A + BK)$ or $K \cdot P$ is **not** linear — change variables ($Y = P^{-1}$, $V = KY$ or $V = KP$), solve the LMI, then recover $K = VY^{-1}$ (resp. $K = VP^{-1}$).

Common mistakes:

- forgetting the definiteness condition on the block being eliminated ($R \succ 0$);
- leaving a product of two unknowns (PK) and calling it an LMI;
- in CVX, omitting the `sdp` flag so `>=` is read element-wise instead of as a PSD cone.

13.8 Template: Formulate MPC

1. State current state x_0 .
2. Define decision variables:
 - $u_0, \dots, u_{M-1}$;
 - $x_1, \dots, x_M$.
3. Write dynamics constraints.
4. Add hard constraints.
5. Add soft constraints as penalties if needed.
6. Add terminal cost or terminal constraint.
7. Solve, apply only u_0 , then replan.

Common mistakes:

- forgetting that only the first input is applied;
- treating forecasts as feedback;
- omitting state estimation even though state is usually estimated, not directly measured;
- writing terminal constraints without explaining their role.

14 Appendix: Practical EDA for Candidate Φ

这部分不是 Roy Smith least-squares 章节的考试重点，而是帮助把 DATA423 的 feature workflow 和 Φ 的矩阵视角连接起来。

实践中，在正式拟合之前，我们通常会对 candidate Φ 做 EDA。核心问题是：

`Do these candidate columns have useful, independent variation for explaining y?`

更具体地说：

- 每一列自己是否有足够变化；
- columns 之间是否高度相关；
- columns 是否和 y 有合理关系；
- Φ 是否 full rank；
- Φ 是否 ill-conditioned；
- 是否存在 outliers 或 high leverage points；
- 是否有 domain reason 支持这些 columns。

14.1 R: From Formula to Φ

在 R 里，`lm()` 公式背后会自动构造 design matrix，也就是这里的 Φ 。

例如：

```
model <- lm(y ~ x1 + x2 + x1:x2, data = df)
X <- model.matrix(model)
```

这里：

$$X = \Phi$$

如果公式是：

```
y ~ x1 + x2 + x1:x2
```

那么 Φ 的 columns 大致是：

$$1, \quad x_1, \quad x_2, \quad x_1x_2$$

如果写：

```
y ~ poly(x, 3)
```

则 R 会构造三阶 polynomial 相关的 columns。实际生成的列是否是原始 powers 或 orthogonal polynomial, 取决于 `poly()` 的设置。

因此, 一个很实用的习惯是:

```
X <- model.matrix(model)
head(X)
dim(X)
```

先看清楚模型真正送进 LS 的 Φ 长什么样。

14.2 Check 1: Single-Column Variation

第一步看每个 feature 自己有没有变化。几乎常数的 column 不会提供多少信息, 还可能导致数值问题。

```
summary(df)
sapply(df[predictors], sd, na.rm = TRUE)
```

如果使用 `caret`:

```
caret::nearZeroVar(df[predictors])
```

矩阵语言:

```
near-zero variance column
-> almost no direction in data
-> weak information for estimating its coefficient
```

注意: variance 大不等于一定 useful。它还需要和 y 有关系, 并且不能只是其他 columns 的重复。

14.3 Check 2: Pairwise Correlation Between Features

相关矩阵检查 columns 是否两两接近线性相关:

```
cor(df[predictors], use = "complete.obs")
```

可视化:

```
pairs(df[predictors])
```

如果:

$$\text{corr}(x_1, x_2) \approx 1$$

那么 Φ 的两列方向几乎一样, 容易造成 multicollinearity。

矩阵语言:

```
highly correlated columns
-> Phi columns nearly linearly dependent
-> larger condition number
-> unstable coefficient estimates
```

但 pairwise correlation 不是完整诊断。两两相关不高，也可能多个 columns 合在一起造成 dependence。

14.4 Check 3: Relationship Between Each Feature and y

单独看 feature 和 output 的关系：

```
library(ggplot2)

ggplot(df, aes(x1, y)) +
  geom_point() +
  geom_smooth()
```

这类图主要看：

- 是否有大致线性关系；
- 是否有 nonlinear pattern；
- 是否需要 transformation；
- 是否存在 outliers；
- 是否不同区间有不同趋势；
- 是否应该加入 interaction。

例如如果 x_1 和 y 的关系随 x_2 改变，就可能考虑：

$$x_1 x_2$$

作为 interaction column。

14.5 Check 4: Multicollinearity Beyond Pairwise Correlation

VIF 检查一个 predictor 能否被其他 predictors 解释。

```
model <- lm(y ~ x1 + x2 + x3, data = df)
car::vif(model)
```

直觉：

```
high VIF for x_j
-> x_j is predictable from other features
-> coefficient for x_j is unstable
```

这比 pairwise correlation 更强，因为它检查的是 one column vs all other columns。

14.6 Check 5: Rank and Condition Number of Φ

这是最直接对应 Roy Smith 这里的检查。

```
model <- lm(y ~ x1 + x2 + x1:x2, data = df)
X <- model.matrix(model)

qr(X)$rank
ncol(X)
kappa(X)
```

解释：

```
qr(X)$rank == ncol(X)
-> full column rank

qr(X)$rank < ncol(X)
-> lose rank
-> some coefficients not identifiable

kappa(X) large
-> ill-conditioned
-> coefficients sensitive to noise
```

如果想看 singular values：

```
svd(X)$d
```

如果最小 singular value 很接近 0：

$$\sigma_{\min}(\Phi) \approx 0$$

说明某些 feature direction 几乎没有被数据激发出来。

14.7 Check 6: Residual Diagnostics After Fitting

前面的 EDA 是拟合前检查 Φ 。拟合后还要看 residuals：

```
par(mfrow = c(2, 2))
plot(model)
```

对应检查：

- Residuals vs Fitted: 是否有系统性模式;
- Q-Q Residuals: residual 是否近似 normal;
- Scale-Location: 是否 heteroscedastic;
- Residuals vs Leverage: 是否有 high leverage / influential points。

这一步回答的是:

`Given this Phi, does the fitted residual look like reasonable noise?`

如果 residual 有明显结构, 可能说明 Φ 的 columns 还没有表达出重要关系, 需要重新做 feature engineering 或 extraction。

14.8 Check 7: Anscombe Warning and High-Dimensional Φ

Roy 用 Anscombe dataset 提醒我们: 仅看 summary statistics 会误导。四组数据可以有相同的:

- mean of x ;
- mean of y ;
- variance of x ;
- variance of y ;
- fitted line slope and intercept;
- correlation-like summary;

但图像结构完全不同。只有一组适合线性拟合; 其他可能是 nonlinear relation、outlier-driven relation, 或者根本没有关系。

这对 Φ 的教训是:

summary statistics are not enough; the geometry of the data matters.

这也连接到主章节里的 data-driven optimisation warning: 如果 optimisation 的 objective、constraint 或 parameters 来自 fitted model, 那么没有检查 data geometry 就直接优化, 会把建模错误放大成决策错误。

当 $d \leq 2$ 或 $d \leq 3$ 时, 我们可以比较直接地画出 data cloud 或 column space 的几何图像。但当 Φ 有很多 columns 时, 完整可视化不现实。这时不要放弃可视化, 而是改成分层、切片、投影和诊断图。

14.8.1 What To Do When $d > 3$

高维时, 实践策略不是“画一张完整的高维图”, 而是:

`visualise useful 2D/3D summaries of the high-dimensional structure`

推荐操作如下。

第一, 画 pairwise feature plots。目标是发现 obvious nonlinear patterns、clusters、outliers 和 collinearity。


```
pairs(df[predictors])
```

如果 predictors 很多，不要画全部。先画：

- domain-important variables;
- 与 y correlation 较高的 variables;
- VIF 高的 variables;
- suspected interaction variables。

第二，画每个重要 feature 对 y 的关系：

```
ggplot(df, aes(x1, y)) +  
  geom_point() +  
  geom_smooth()
```

这不是为了证明模型正确，而是为了发现：

- line 是否合理;
- 是否更像 polynomial;
- 是否被 outlier 驱动;
- 是否存在 threshold / saturation;
- 是否需要 interaction。

第三，画 residual vs fitted:

```
plot(model, which = 1)
```

如果 residual vs fitted 有曲线、漏斗形或分层，说明 Φ 可能没有表达出重要结构。

第四，画 residual vs important predictors:

```
df$resid <- residuals(model)  
  
ggplot(df, aes(x1, resid)) +  
  geom_point() +  
  geom_smooth()
```

如果 residual 对某个 predictor 仍有模式，说明这个 predictor 的关系没有被当前 Φ 正确表达。可能需要：

- nonlinear term;
- transformation;
- interaction;
- 分组模型;
- 不同 model class。

第五，用 partial residual / component-plus-residual plots 看某个 predictor 在控制其他 predictors 后的形状。

```
car::crPlots(model)
```

这比单独画 x_j vs y 更接近多元回归语境，因为它部分控制了其他 columns 的影响。

第六，检查 leverage 和 influential points:

```
plot(model, which = 5)
cooks.distance(model)
hatvalues(model)
```

Anscombe 的一个重要教训就是：一两个点可能制造出看似合理的 slope。高维时更要检查 high leverage，因为某些点在单变量图里不显眼，但在 Φ 的 column space 中非常极端。

第七，用 low-dimensional projection 看整体结构。比如 PCA:

```
X <- model.matrix(model)[, -1]
X_scaled <- scale(X)
pca <- prcomp(X_scaled)

plot(pca$x[, 1], pca$x[, 2])
```

如果想看 y 或 residual 在 projection 上的结构:

```
plot(pca$x[, 1], pca$x[, 2], col = cut(df$y, 4))
```

或:

```
plot(pca$x[, 1], pca$x[, 2], col = cut(residuals(model), 4))
```

这回答的是:

```
In the main variation directions of Phi, do y or residuals show structure?
```

第八，做 sample slicing。高维关系常常只在某些区间里显现。可以按某个变量分组，再画局部关系:

```
ggplot(df, aes(x1, y)) +
  geom_point() +
  geom_smooth() +
  facet_wrap(~ group_var)
```

如果不同 slice 的 slope 不同，就可能需要 interaction:

$$x_1 x_2$$

第九，把 visual checks 和 validation 结合。高维时不可能靠眼睛确认全部结构，所以最后要看:

```
resampling / cross-validation / test performance
```

视觉检查帮助我们发现 model misspecification; validation 帮助我们判断新的 Φ 是否真的提升泛化。

14.8.2 High-Dimensional EDA Checklist

当 Φ 很宽时，可以按这个顺序做：

1. Check column variance and missingness.
2. Check pairwise correlations for important predictors.
3. Check VIF / rank / condition number.
4. Plot y vs key predictors.
5. Fit baseline model.
6. Plot residuals vs fitted.
7. Plot residuals vs key predictors.
8. Check leverage and Cook's distance.
9. Use PCA/low-dimensional projection of Φ .
10. Validate candidate Φ designs by cross-validation.

核心态度是：

Do not trust statistics alone; use plots to find structure, and use validation to test whether the structure helps prediction.

14.9 Practical Workflow

一个实际但不复杂的 workflow：

1. Start from domain variables.
2. Build candidate features: transformations, interactions, basis functions.
3. Inspect each column: variance, missingness, units, scale.
4. Inspect relationships: feature-feature correlation and feature- y plots.
5. Build Φ via `model.matrix()`.
6. Check rank and condition number.
7. Fit model.
8. Check residual diagnostics.
9. Use validation/cross-validation for model selection.
10. Revise Φ if needed.

这个流程的重点不是“机械删除所有高相关变量”，而是理解：

useful information for estimating θ lives in independent, relevant variation of the columns of Φ .

因此，EDA 不是替代 LS，而是在问 LS 所依赖的 design matrix 是否真的含有足够信息。

15 Appendix: Φ as Hidden Structure and Dynamics

这部分把前面关于 Φ 的直觉再提升一层： Φ 不只是一个 data table，也不只是 R 里 `model.matrix()` 生成的矩阵。它是模型把世界压缩成可计算结构的方式。

核心句子：

choosing Φ means choosing what kinds of variation the model is allowed to explain.

也可以说：

every column of Φ is an implicit hypothesis about the mechanism behind y .

在 LS 里：

$$y \approx \Phi\theta$$

这句话不只是代数式。它隐含地说：

y mainly varies along the directions represented by the columns of Phi.

因此， Φ 的列既是数学 basis，也是 assumptions about structure, dynamics, physics, behaviour, or data generation。

15.1 Common Φ Constructions and Their Implicit Assumptions

Φ construction	Example columns	Implicit assumption
Linear features	1, x_1 , x_2	y 对输入的响应近似线性
Polynomial basis	1, x , x^2 , x^3	关系是平滑曲线，可由局部弯曲近似
Chebyshev basis	$T_0(x)$, $T_1(x)$, $T_2(x)$	关系在区间内平滑，并希望数值更稳定
Interaction terms	x_1x_2	一个变量的影响取决于另一个变量
Log features	$\log x$	边际效应递减，或关系更像比例变化
Sin/cos seasonal terms	$\sin(2\pi t/T)$, $\cos(2\pi t/T)$	数据有周期性，例如日、周、年
Lag features	y_{t-1} , y_{t-2} , u_{t-1}	当前状态受过去状态/输入影响
Conic section basis	x^2 , xy , y^2 , x , y , 1	轨迹近似服从二体中心力运动
PTDF / sensitivity matrix	PTDF p	潮流可由 DC power flow 线性近似
Jacobian / linearisation	$H_\theta(\theta_i)$	非线性系统在局部可线性化
PCA components	$z_1, z_2, \dots$	主要信息在少数 latent directions 中
One-hot categories	indicator columns	不同类别有不同 baseline effect

Φ construction	Example columns	Implicit assumption
Splines	piecewise basis	关系平滑，但不同区间曲率不同

这张表的重点不是背例子，而是看出模式：

```
choose columns
-> choose allowed variation directions
-> choose implicit model structure
```

15.2 Orbit Fitting: Polynomial vs Conic Section

Roy 讲 conic section 时，重点不是“圆锥曲线比多项式更高级”，而是：

conic section encodes the physics of ideal two-body motion.

如果用 polynomial：

$$y \approx a_0 + a_1x + a_2x^2 + a_3x^3$$

隐含假设是：

the trajectory is a smooth graph $y=f(x)$ that can be locally bent by polynomial terms.

如果用 conic section：

$$Ax^2 + Bxy + Cy^2 + Dx + Ey + F = 0$$

隐含假设变成：

the trajectory is close to the solution of a two-body central-force problem.

其物理线索是：

```
inverse-square central force
+ conservation of energy
+ conservation of angular momentum
-> conic sections
```

所以 conic basis：

$$x^2, xy, y^2, x, y, 1$$

不是任意的 feature engineering。它把二体轨道的几何结构写进了 Φ 。

这就是为什么短时间 polynomial 也许能 fit 一段轨迹，但如果要 extrapolate orbit 或解释 physical orbital parameters, conic section 更自然。

15.3 Industrial Examples

下面这些例子帮助建立直觉：工业中的 Φ 往往来自物理模型、工程近似、过程知识或行为假设。

15.3.1 Power Systems: DC Power Flow / OPF

可能有：

$$f \approx \text{PTDF } p$$

这里 Φ 来自 PTDF 或 sensitivity matrix。

隐含假设：

```
voltage magnitudes near 1 p.u.
angle differences small
reactive power ignored
network topology fixed
line flows approximately linear in injections
```

所以 Φ 编码的是 DC power flow 近似，而不是任意矩阵。

15.3.2 Refinery / Chemical Process Scheduling

可能有：

$$\text{product output} \approx \Phi \cdot \text{feedstock mix}$$

Φ 的列可能是不同 crude oil 的 yield profile，例如某种原油能炼出多少 petrol、diesel、jet fuel。

隐含假设：

```
yield coefficients approximately stable
blending is roughly linear
process units operate in known regimes
capacity constraints known
```

这类 Φ 来自过程工程知识和历史 plant data。

15.3.3 Manufacturing: Production Time / Throughput Model

例如:

$$\text{production time} \approx \theta_0 + \theta_1(\text{batch size}) + \theta_2(\text{setup}) + \theta_3(\text{shift})$$

隐含假设:

batch size affects time predictably
 setup cost adds fixed overhead
 shift/operator effects can be separated
 relationship stable across observations

如果加入 interaction:

$$\text{batch size} \times \text{shift}$$

则是在假设 batch-size effect depends on shift/operator context。

15.3.4 Structural Engineering: Modal Basis

在结构动力学或有限元降阶里, 可能写:

$$u \approx \Phi q$$

其中 Φ 的列是 mode shapes, q 是 modal coordinates。

隐含假设:

deformation can be represented by selected modes
 high-frequency modes are negligible
 linear elasticity approximately holds

这里 Φ 的列是结构系统“自然振动方式”的方向。

15.3.5 Control System Identification

线性 state-space identification 常写:

$$x_{t+1} \approx Ax_t + Bu_t$$

写成 LS 时, Φ 包含:

$$[x_t, u_t]$$

隐含假设:

```
next state depends linearly on current state and input
dynamics are time-invariant over the dataset
unmodelled effects are noise
```

如果系统是 nonlinear, 则可能用 Jacobian linearisation 得到 local Φ 。

15.3.6 Predictive Maintenance

Φ 可能包含:

```
temperature
vibration RMS
load
operating hours
recent trend
```

隐含假设:

```
failure risk is related to measurable degradation signals
recent history contains predictive information
sensor features summarize machine condition
```

如果 residual 仍有结构, 可能说明某些 degradation mechanism 没有被当前 Φ 捕捉。

15.3.7 Traffic and Network Flow

Φ 可能包含:

```
previous traffic volume
speed
time of day
day of week
weather
event indicator
```

隐含假设:

```
traffic has temporal dependence
daily/weekly cycles matter
weather/events perturb baseline flow
```


这里 lag features 和 seasonal features 都是在把动态结构写入 Φ 。

15.3.8 Finance / Risk Factor Models

Factor model:

$$r_i \approx \beta_{i1}f_1 + \beta_{i2}f_2 + \cdots + \epsilon_i$$

Φ 是 market factors。

隐含假设:

```
asset returns are driven by common latent/economic factors  
idiosyncratic noise is residual  
factor exposure is stable over fitting window
```

15.3.9 Healthcare / Biostatistics

Φ 可能包含:

```
age  
BMI  
blood pressure  
biomarkers  
treatment indicator  
interaction terms
```

隐含假设:

```
outcome varies systematically with physiological covariates  
treatment effect can be separated from baseline risk  
interactions may represent subgroup effects
```

15.3.10 Demand Forecasting / Supply Chain

Φ 可能包含:

```
price  
promotion indicator  
holiday  
season  
lagged demand  
competitor price
```

隐含假设:

```
demand responds to price and promotion
seasonality matters
past demand carries information about future demand
```

15.4 The Main Intuition

把所有例子压缩成一句:

Φ is the model's account of why y varies.

所以构造 Φ 时, 不只是在做数据预处理, 而是在回答:

```
What mechanisms, dynamics, cycles, interactions, or physical laws
do I believe generate the observed variation in  $y$ ?
```

如果 Φ 选得好, least squares 是在正确的 structural directions 上投影。如果 Φ 选得差, least squares 仍然会给出最小 residual, 但可能是在错误的假设空间里找到的最优解。

16 Appendix: Math Background - SVD and Pseudo-inverse

这部分是 rank、condition number、pseudo-inverse 和 regularisation 的共同数学背景。考试不一定要求完整证明 SVD, 但理解它会让 Roy 提到的 singular、rank deficient、pseudo-inverse 更自然。

16.1 What SVD Says

Singular value decomposition, SVD, 写成:

$$\Phi = U\Sigma V^T$$

它的几何直觉是:

```
V^T      : rotate / re-express input directions
Sigma    : stretch or compress each special direction
U        : rotate / re-express output directions
```

所以任何矩阵 Φ 的作用都可以理解成:

```
input vector
-> rotate to special coordinates
-> scale each direction by singular values
-> rotate into output space
```

其中最重要的是 Σ 。它通常形如：

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & \cdots \\ 0 & \sigma_2 & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

对角线上的：

$$\sigma_1, \sigma_2, \dots$$

叫 singular values。

16.2 Geometric Picture: Circle to Ellipse

在二维直觉里，可以想象一个单位圆。矩阵 Φ 作用在这个圆上后，通常会变成椭圆：

```
circle -> ellipse
```

SVD 告诉我们：

- V^T 找到输入空间里哪些方向会变成椭圆主轴；
- Σ 给出这些主轴被拉伸或压缩的长度；
- U 告诉我们这个椭圆在输出空间里朝哪个方向。

因此 singular values 可以理解成椭圆半轴长度。

如果：

$$\sigma_i \text{ large}$$

说明这个方向被数据/矩阵强烈看见。

如果：

$$\sigma_i \text{ small}$$

说明这个方向被压得很扁，估计会对噪声敏感。

如果：

$$\sigma_i = 0$$

说明这个方向被完全压没了，信息丢失。

16.3 Singular and Rank Deficient

一个 square matrix 如果 singular，意思是它不可逆：

$$A^{-1} \text{ does not exist}$$

几何上，它把至少一个非零方向压成了零：

$$Ax = 0, \quad x \neq 0$$

所以不同输入可能给出同一个输出，无法反向唯一恢复。

对 LS 来说，如果 Φ not full column rank，则存在：

$$v \neq 0$$

使得：

$$\Phi v = 0$$

这意味着：

$$\Phi(\theta + \alpha v) = \Phi\theta$$

任意 α 都给出同样 prediction，因此参数不唯一。这就是 rank deficient 的本质。

用 SVD 看，rank 就是非零 singular values 的数量：

$$\text{rank}(\Phi) = \#\{i : \sigma_i > 0\}$$

16.4 Condition Number Through SVD

Condition number 可以写成：

$$\kappa(\Phi) = \frac{\sigma_{\max}}{\sigma_{\min}}$$

如果 $\sigma_{\min}$ 很小，说明存在某个方向几乎看不见。此时：

```
small data noise
-> large coefficient uncertainty
```

Normal equations 会形成:

$$\Phi^T \Phi$$

它的 singular/eigen values 对应大致会平方, 所以:

$$\kappa(\Phi^T \Phi) = \kappa(\Phi)^2$$

这就是为什么直接用 normal-equation inverse 在数值上可能更糟。

16.5 Pseudo-inverse from SVD

普通 inverse 的思想是: 每个方向都反向缩放。

如果某方向被乘以:

$$\sigma_i$$

反解时要乘以:

$$\frac{1}{\sigma_i}$$

但如果:

$$\sigma_i = 0$$

就不能取倒数。Pseudo-inverse 的做法是:

$$\sigma_i^+ = \begin{cases} 1/\sigma_i, & \sigma_i \neq 0 \\ 0, & \sigma_i = 0 \end{cases}$$

因此如果:

$$\Phi = U \Sigma V^T$$

则:

$$\Phi^+ = V\Sigma^+U^T$$

直觉:

directions with information: invert the scaling
directions with zero information: do not guess

所以 pseudo-inverse 承认有些方向数据没有告诉我们, 不强行恢复那些方向。

16.6 Meaning in Least Squares

在:

$$y \approx \Phi\theta$$

中, SVD 告诉我们:

which parameter directions are strongly visible in the data
which directions are weakly visible
which directions are invisible

如果 full column rank 且 well-conditioned, ordinary LS 很稳定。

如果 rank deficient, 有无限多组 θ 可能给出同样 prediction。Pseudo-inverse 选择其中 norm 最小的:

$$\hat{\theta} = \arg \min_{\Phi\theta=y} \|\theta\|_2$$

如果没有 exact solution, pseudo-inverse 给 least-squares best fit:

$$\hat{\theta} = \arg \min_{\theta} \|\Phi\theta - y\|_2$$

更完整地说, pseudo-inverse 在多解时选 minimum-norm solution。

16.7 Why Small Singular Values Are Dangerous

如果:

$$\sigma_i = 0.001$$

pseudo-inverse 会用:

$$\frac{1}{\sigma_i} = 1000$$

这会把该方向上的噪声也放大 1000 倍。

所以 small singular values 虽然不是严格 zero，但仍然危险。它们对应：

`nearly unidentifiable directions`

这就是 regularisation 和 truncated SVD 的动机：

- pseudo-inverse: invert all nonzero directions;
- truncated SVD: ignore very small directions;
- ridge regularisation: shrink unstable directions instead of fully inverting them.

16.8 One-Line Summary

SVD 是看 Φ 的“方向显微镜”：

```
large singular value -> data strongly identifies that direction
small singular value -> direction is weak and noise-sensitive
zero singular value  -> direction is invisible / rank deficient
```

Pseudo-inverse 是基于这个显微镜做的 generalized inverse：

```
invert what can be inverted,
ignore what has been completely lost,
choose the minimum-norm solution when many solutions exist.
```

17 Appendix: Projection Geometry and Instrumental Variables

这部分补充 Roy 在 Lecture 6 里用几何解释导出 normal equations，并顺势引出 instrumental variable methods 的思路。它不是新的 LS 公式，而是同一个问题的另一种视角。

17.1 Projection View of Least Squares

Least-squares problem 是：

$$\min_{\theta} \|y - \Phi\theta\|_2^2$$

其中：

$$y \in \mathbb{R}^N, \quad \Phi \in \mathbb{R}^{N \times d}, \quad \theta \in \mathbb{R}^d$$

$\Phi\theta$ 是 Φ 的 columns 的 linear combination。因此所有可能的 fitted values:

$$\hat{y} = \Phi\theta$$

都落在 Φ 的 column space 里:

$$\text{col}(\Phi)$$

几何上, OLS 做的是:

把 y 投影到 $\text{col}(\Phi)$ 上, 找到离 y 最近的点 $\hat{y} = \Phi\theta^*$ 。

残差是:

$$\epsilon = y - \Phi\theta^*$$

投影 theorem 告诉我们, 最近点处的 residual 必须垂直于这个 subspace:

$$\epsilon \perp \text{col}(\Phi)$$

也就是说 residual 不仅垂直于 $\Phi\theta^*$, 更强地说, 它垂直于 Φ 的每一列:

$$\phi_i^T \epsilon = 0, \quad i = 1, \dots, d$$

把这些条件叠成矩阵形式:

$$\Phi^T \epsilon = 0$$

这就是 normal equations 的几何来源。

17.2 Inner Product Notation

Roy 板书里写:

$$\langle \Phi\theta^*, \epsilon \rangle = 0$$

这里的 inner product 对实向量就是:

$$\langle a, b \rangle = a^T b$$

所以：

$$\langle \Phi\theta^*, \epsilon \rangle = (\Phi\theta^*)^T \epsilon$$

而 residual 是：

$$\epsilon = y - \Phi\theta^*$$

代入：

$$\langle \Phi\theta^*, y - \Phi\theta^* \rangle = 0$$

利用 inner product 的线性：

$$\langle a, b - c \rangle = \langle a, b \rangle - \langle a, c \rangle$$

得到：

$$\langle \Phi\theta^*, y \rangle - \langle \Phi\theta^*, \Phi\theta^* \rangle = 0$$

这对应你截图里的展开：

$$\langle \Phi\theta^*, y \rangle - \langle \Phi\theta^*, \Phi\theta^* \rangle = 0$$

17.3 Expanding the Inner Products Step by Step

第一项：

$$\langle \Phi\theta^*, y \rangle = (\Phi\theta^*)^T y$$

用转置规则：

$$(AB)^T = B^T A^T$$

所以：

$$(\Phi\theta^*)^T = (\theta^*)^T\Phi^T$$

因此：

$$\langle \Phi\theta^*, y \rangle = (\theta^*)^T\Phi^T y$$

第二项：

$$\langle \Phi\theta^*, \Phi\theta^* \rangle = (\Phi\theta^*)^T(\Phi\theta^*)$$

继续用同样的转置规则：

$$(\Phi\theta^*)^T(\Phi\theta^*) = (\theta^*)^T\Phi^T\Phi\theta^*$$

所以：

$$(\theta^*)^T\Phi^T y - (\theta^*)^T\Phi^T\Phi\theta^* = 0$$

提取共同的 $(\theta^*)^T$ ：

$$(\theta^*)^T (\Phi^T y - \Phi^T\Phi\theta^*) = 0$$

为了让这个 orthogonality condition 对 optimal projection 成立，括号中的 normal-equation residual 应该为 0：

$$\Phi^T y - \Phi^T\Phi\theta^* = 0$$

所以：

$$\Phi^T\Phi\theta^* = \Phi^T y$$

这就是 normal equations。

更直接、更标准的写法是从 “residual 垂直于整个 column space” 出发：

$$\Phi^T \epsilon = 0$$

代入 $\epsilon = y - \Phi\theta^*$ ：

$$\Phi^T(y - \Phi\theta^*) = 0$$

展开：

$$\Phi^T y - \Phi^T \Phi \theta^* = 0$$

所以：

$$\Phi^T \Phi \theta^* = \Phi^T y$$

这个版本更严谨，因为它直接表达了 residual 对 Φ 的每一列都正交，而不仅仅是对某一个 fitted vector $\Phi\theta^*$ 正交。

17.4 What This Has to Do with Instrumental Variables

OLS 的核心 moment condition 是：

$$\Phi^T \epsilon = 0$$

换句话说：

residual should be orthogonal to the regressors.

在 sample form 里，如果 Φ 的第 i 列是 regressor ϕ_i ，那么：

$$\sum_{j=1}^N \phi_{ji} \epsilon_j = 0$$

这表示 regressor 和 residual 没有剩余相关性。

但如果 regressors 和 true error 相关，例如：

$$\text{Cov}(\Phi, \epsilon) \neq 0$$

那么 OLS 的正交条件不再可信。此时强行要求：

$$\Phi^T \epsilon = 0$$

会让 estimator 吸收错误的相关性，导致 biased 或 inconsistent estimates。

Instrumental variables 的出发点就是换一个正交对象。我们找一个工具变量矩阵：

$$Z$$

它满足两个直觉条件：

1. Z 和 problematic regressors Φ 有关系；
2. Z 和 structural error ϵ 没关系。

对应写成：

$$\text{Cov}(Z, \Phi) \neq 0$$

和：

$$\text{Cov}(Z, \epsilon) = 0$$

于是 IV 不再要求 residual 对 Φ 正交，而是要求 residual 对 Z 正交：

$$Z^T \epsilon = 0$$

代入：

$$\epsilon = y - \Phi\theta$$

得到：

$$Z^T(y - \Phi\theta) = 0$$

展开：

$$Z^T y - Z^T \Phi \theta = 0$$

所以：

$$Z^T \Phi \theta = Z^T y$$

如果 $Z^T \Phi$ 可逆，则：

$$\hat{\theta}_{IV} = (Z^T \Phi)^{-1} Z^T y$$

这就是 Roy 说 instrumental variable methods 可以从 orthogonality point of view 出发的意思。

17.5 OLS vs IV as Orthogonality Conditions

可以把它们放在同一个框架里：

Method	Orthogonality condition	Meaning
OLS	$\Phi^T \epsilon = 0$	residual orthogonal to regressors
IV	$Z^T \epsilon = 0$	residual orthogonal to instruments

OLS 的几何图像是：

```
project y onto column space of Phi
residual perpendicular to col(Phi)
```

IV 的直觉是：

```
Phi may be contaminated
so use Z as clean directions
require residual to be orthogonal to Z instead
```

所以 Roy 这里的重点不是让你多背一个算法，而是让你看到：

least squares, normal equations, and instrumental variables all come from choosing which directions the residual must be orthogonal to.

18 Appendix: Math Background - Variance and Covariance Formula Sheet

这部分是 least-squares statistics、BLUE、standard error、confidence interval 的基础公式参考。读 Roy 的 Lecture 7 时，最常用的是最后的 linear transformation rule。

18.1 Scalar Variance

对 scalar random variable X ：

$$\text{Var}(X) = E[(X - E[X])^2].$$

等价展开式:

$$\text{Var}(X) = E[X^2] - (E[X])^2.$$

常数平移不改变 variance:

$$\text{Var}(X + b) = \text{Var}(X).$$

常数缩放会平方:

$$\text{Var}(aX) = a^2 \text{Var}(X).$$

18.2 Scalar Covariance

对两个 scalar random variables X, Y :

$$\text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])].$$

等价展开式:

$$\text{Cov}(X, Y) = E[XY] - E[X]E[Y].$$

特殊情况:

$$\text{Cov}(X, X) = \text{Var}(X).$$

线性缩放:

$$\text{Cov}(aX, bY) = ab \text{Cov}(X, Y).$$

如果 X 和 Y independent, 则:

$$\text{Cov}(X, Y) = 0.$$

但反过来不一定成立:

$$\text{Cov}(X, Y) = 0 \nRightarrow X, Y \text{ independent}.$$

18.3 Variance of a Sum

一般情况：

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2 \text{Cov}(X, Y).$$

如果 X, Y uncorrelated 或 independent：

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y).$$

带权重的一般形式：

$$\text{Var}(aX + bY) = a^2 \text{Var}(X) + b^2 \text{Var}(Y) + 2ab \text{Cov}(X, Y).$$

对多个 independent / uncorrelated variables：

$$\text{Var}\left(\sum_i w_i X_i\right) = \sum_i w_i^2 \text{Var}(X_i).$$

如果不独立也不 uncorrelated，则要加 covariance terms：

$$\text{Var}\left(\sum_i w_i X_i\right) = \sum_i \sum_j w_i w_j \text{Cov}(X_i, X_j).$$

18.4 Covariance Matrix of a Random Vector

对 random vector $X \in \mathbb{R}^n$ ：

$$\text{Cov}(X) = E[(X - E[X])(X - E[X])^T].$$

如果：

$$X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_n \end{bmatrix},$$

那么：

$$\text{Cov}(X) = \begin{bmatrix} \text{Var}(X_1) & \text{Cov}(X_1, X_2) & \cdots \\ \text{Cov}(X_2, X_1) & \text{Var}(X_2) & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}.$$

记忆方式:

diagonal entries $\rightarrow$ variances
off-diagonal entries $\rightarrow$ covariances

Covariance matrix is symmetric:

$$\text{Cov}(X)^T = \text{Cov}(X).$$

It is positive semidefinite:

$$z^T \text{Cov}(X) z \geq 0 \quad \forall z.$$

18.5 Linear Transformation Rule

这是 least-squares 里最重要的 covariance 公式。

如果:

$$Y = AX,$$

其中 A 是 deterministic matrix, 那么:

$$E[Y] = AE[X],$$

且:

$$\text{Cov}(Y) = A \text{Cov}(X) A^T.$$

如果再加常数:

$$Y = AX + b,$$

则:

$$E[Y] = AE[X] + b,$$

但 covariance 不受常数 b 影响:

$$\text{Cov}(Y) = A \text{Cov}(X) A^T.$$

18.6 How It Is Used in Least Squares

在 ordinary LS 里:

$$\hat{\theta} = (\Phi^T \Phi)^{-1} \Phi^T y.$$

如果:

$$y = \Phi \theta + \epsilon,$$

则:

$$\hat{\theta} - \theta = (\Phi^T \Phi)^{-1} \Phi^T \epsilon.$$

令:

$$A = (\Phi^T \Phi)^{-1} \Phi^T.$$

那么:

$$\hat{\theta} - \theta = A \epsilon.$$

使用 linear transformation rule:

$$\text{Cov}(\hat{\theta}) = A \text{Cov}(\epsilon) A^T.$$

如果 white-noise assumption 成立:

$$\text{Cov}(\epsilon) = \sigma^2 I,$$

则:

$$\begin{aligned}\text{Cov}(\hat{\theta}) &= (\Phi^T \Phi)^{-1} \Phi^T (\sigma^2 I) \Phi (\Phi^T \Phi)^{-1} \\ &= \sigma^2 (\Phi^T \Phi)^{-1}.\end{aligned}$$

如果 noise covariance 是一般矩阵:

$$\text{Cov}(\epsilon) = R,$$

那么 ordinary LS 的 covariance 是:

$$\text{Cov}(\hat{\theta}_{\text{OLS}}) = (\Phi^T \Phi)^{-1} \Phi^T R \Phi (\Phi^T \Phi)^{-1}.$$

而 correlated-noise BLUE 使用:

$$\hat{\theta}_{\text{BLUE}} = (\Phi^T R^{-1} \Phi)^{-1} \Phi^T R^{-1} y,$$

其 covariance 为:

$$\text{Cov}(\hat{\theta}_{\text{BLUE}}) = (\Phi^T R^{-1} \Phi)^{-1}.$$

18.7 Connection to Standard Error

如果:

$$\text{Cov}(\hat{\theta}) = C_{\theta},$$

那么第 j 个 coefficient 的 variance 是:

$$\text{Var}(\hat{\theta}_j) = (C_{\theta})_{jj}.$$

Standard error 是:

$$\text{se}(\hat{\theta}_j) = \sqrt{(C_{\theta})_{jj}}.$$

在 ordinary LS white-noise case:

$$C_{\theta} = \sigma^2 (\Phi^T \Phi)^{-1}.$$

如果 σ 未知, 用 $\hat{\sigma}$ 估计:

$$\widehat{\text{se}}(\hat{\theta}_j) = \hat{\sigma} \sqrt{((\Phi^T \Phi)^{-1})_{jj}}.$$

19 Self Check

You should be able to answer these without notes:

1. What are the normal equations and where do they come from?
2. Why is the LS residual orthogonal to the column space of Φ ?
3. What is the difference between ordinary LS, weighted LS, and BLUE?
4. Why does minimum variance among unbiased estimators not imply minimum MSE?
5. What is the difference between algebraic consistency and statistical consistency?
6. How do standard errors lead to confidence intervals and coefficient significance?
7. How does regularisation change bias and variance?
8. When would you use Ridge, LASSO, ElasticNet, AIC/BIC, or cross-validation?
9. Why does Roy prefer a low-dimensional parameterisation of the Tikhonov penalty matrix?
10. What is a convex set (凸集)?
11. Why are affine equality constraints (仿射等式约束) allowed in convex optimisation (凸优化)?
12. What is the difference between LP (线性规划), QP (二次规划), SOCP (二阶锥规划), QCQP (二次约束二次规划), and SDP (半定规划)?
13. What is an LMI (线性矩阵不等式)?
14. What does the Schur complement (舒尔补) help with?
15. How do you write an MPC optimisation problem?
16. Why does MPC apply only the first input?
17. What is explicit MPC, and why does it not scale well?

20 Minimum Memory List

- LS problem:

$$\min_{\theta} \|y - \Phi\theta\|_2^2$$

- Normal equations:

$$\Phi^T \Phi \theta = \Phi^T y$$

- Ordinary LS:

$$\hat{\theta} = (\Phi^T \Phi)^{-1} \Phi^T y$$

- OLS unbiased condition:

$$y = \Phi \theta_0 + \epsilon, \quad E[\epsilon] = 0 \quad \Rightarrow \quad E[\hat{\theta}] = \theta_0$$

- BLUE:

$$\hat{\theta} = (\Phi^T R^{-1} \Phi)^{-1} \Phi^T R^{-1} y$$

- MSE:

$$\text{MSE} = \text{bias}^2 + \text{variance}$$

- Regularised LS:

$$\hat{\theta} = (\Phi^T \Phi + \eta I)^{-1} \Phi^T y$$

- Convex set (凸集) :

$$\gamma x_1 + (1 - \gamma)x_2 \in C$$

- PSD (positive semidefinite, 正半定) :

$$X \succeq 0 \Leftrightarrow z^T X z \geq 0 \quad \forall z$$

- MPC loop (模型预测控制循环) :

```
estimate state -> optimise horizon -> apply first input -> re-estimate -> repeat
```